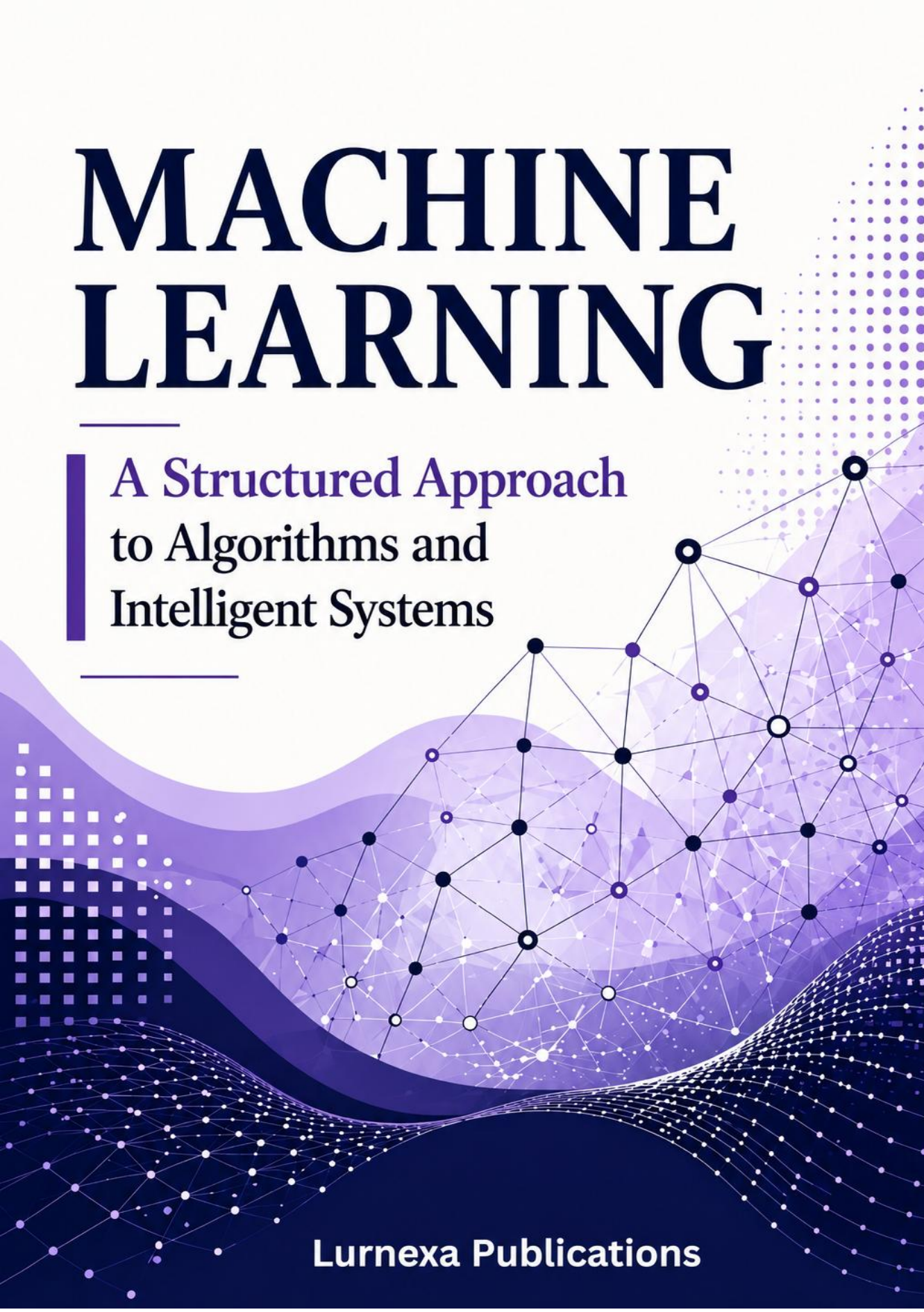
MACHINE LEARNING

A Structured Approach to Algorithms and Intelligent Systems



Lurnexa Publications

MACHINE LEARNING: A STRUCTURED APPROACH TO ALGORITHMS AND INTELLIGENT SYSTEMS

- Dr. H Balaji
Ms. J Saritha
Ms. B Pallavi

LURNEXA PUBLICATIONS

Title: *MACHINE LEARNING: A STRUCTURED APPROACH TO ALGORITHMS AND INTELLIGENT SYSTEMS*

Authors: Dr. H Balaji, Ms. J Saritha, Ms. B Pallavi

Edition: First Edition, 2026

Published by:
Lurnexa Publications Private Limited
(An ISO 9001:2015 Certified Organization)
130–187, Ramulavari Gudi Centre,
Gorantla, Guntur – 522034, Andhra Pradesh, India
Email: lurnexapublication@gmail.com
Website: www.lurnexa.in

ISBN: 978-81-685077-3-9

Cataloguing-in-Publication Data: Available on request

Printed and bound in India

ENDORSEMENT

Machine Learning is a valuable academic contribution to the field of Artificial Intelligence and intelligent computing systems. The authors, **Dr. H. Balaji**, **J. Saritha**, and **Pallavi B**, have successfully combined academic expertise, teaching excellence, and research insight to create a book that is both informative and practically relevant.

The book presents fundamental and advanced concepts of Machine Learning in a clear, systematic, and learner-friendly manner. Topics such as learning paradigms, feature engineering, model selection, model evaluation, and predictive analytics are explained with clarity and academic depth.

A major strength of this publication is its balance between theoretical understanding and practical application. Through the examples, illustrations, and analytical discussions, the authors have simplified complex concepts, making the book highly useful for students, faculty members, researchers, and industry professionals. The inclusion of review questions and practical perspectives further enhances its academic value.

I congratulate the authors and publishers for this commendable scholarly effort and extend my best wishes for the book's success and impact in the academic and professional community.

Dr. T V RAJINI KANTH
Professor of Emerging Technologies, Convener R&D,
Mahatma Gandhi Institute of Technology, Hyderabad.

FOREWORD

It is a privilege to present this insightful work on **Machine Learning**. In today's data-driven world, the ability to extract meaningful insights from data has become essential across business, governance, healthcare, finance, manufacturing, and public administration. This book is therefore both timely and highly relevant.

The authors **Dr. H. Balaji, J. Saritha**, and **Pallavi B** bring together strong academic expertise, teaching excellence, and research-oriented insight. Their combined experience reflects both scholarly rigor and a deep commitment to learner development.

The book explains fundamental and advanced concepts of Machine Learning in a clear, structured, and application-oriented manner. It presents Machine Learning not only as a technical discipline but also as an interdisciplinary field with growing importance in industry, research, and decision-making processes. By balancing theory with practical understanding, the authors make complex concepts accessible to students, faculty members, researchers, and professionals alike.

This work will serve as a valuable resource for learners and practitioners seeking a strong foundation in Machine Learning and Data Mining. It meaningfully contributes to the development of analytical and problem-solving skills required in an increasingly data-centric world.

I congratulate the authors and publishers for this commendable academic contribution and extend my best wishes for its success and impact in the academic and professional community.

Dr. Easwar Krishna Iyer
Professor & Director,
Indus Business Academy - IBA, Bangalore.

PREFACE

It is a matter of great academic satisfaction to present this book on **Machine Learning**, a valuable contribution to the growing field of Artificial Intelligence and intelligent computing systems.

Machine Learning has become a key driver of technological advancement in across the healthcare and finance and education, manufacturing and cybersecurity, and scientific research. As industries increasingly rely on data-driven solutions, there is a strong need for educational resources that combine theoretical understanding with practical application. This book successfully fulfills that need through its systematic and learner-friendly approach.

The text provides clear coverage of essential topics including learning paradigms, feature engineering, data representation, model selection, model learning, and evaluation techniques.

The authors have balanced academic depth with practical relevance by explaining complex concepts through examples, diagrams, and review questions, making the book suitable for both classroom learning and self-study.

Particular emphasis has been placed on the complete Machine Learning pipeline, from data acquisition and preprocessing to model evaluation and prediction. This integrated approach helps learners understand Machine Learning as a comprehensive problem-solving framework.

This publication will serve as a valuable resource for students, faculty members, researchers, and professionals seeking a strong foundation in Machine Learning. I congratulate the authors and publishers for their sincere efforts and extend my best wishes for the book's success and academic impact.

Dr. Chinmaya Kumar Swain
Assistant Professor, IT Systems & Analytics
Indian Institute of Management Jammu (IIM Jammu)

AUTHORS

Dr. Halavath Balaji

Dr. Halavath Balaji is a dedicated academician and Professor at Sreenidhi Institute of Science and Technology. He is recognized for his expertise in teaching, research guidance, and student mentorship. His commitment to academic excellence, continuous learning, and creating an intellectually stimulating environment has earned him respect among students and colleagues. Through his guidance and mentorship, he continues to inspire aspiring engineers and researchers.

Jogu Saritha

Jogu Saritha is an Assistant Professor at Sreenidhi Institute of Science and Technology and a Research Scholar at National Institute of Technology Jalandhar. She has strong expertise in teaching, mentoring, and research, with a commitment to academic excellence and continuous learning. Her dedication to quality education and student development has made her a respected educator. She dedicates her work to her parents and expresses sincere gratitude to her husband for his constant support and encouragement throughout her professional journey.

Pallavi B

Pallavi B is an experienced academician serving as an Assistant Professor at Sreenidhi Institute of Science and Technology and a Research Scholar at SR University. With 18 years of teaching experience, she has developed strong expertise in delivering quality education, mentoring students, and supporting their academic and career growth. She creates a positive learning environment that encourages curiosity, critical thinking, and continuous learning. Her dedication to both teaching and research reflects her strong commitment to education and professional excellence.

ACKNOWLEDGEMENTS

The completion of this **Machine Learning** textbook has been possible through the support and contributions of many individuals and institutions. We sincerely thank everyone who contributed directly and indirectly to this work.

We express our heartfelt gratitude to **Dr. H. Balaji**, **J. Saritha**, and **B. Pallavi** for their dedicated efforts in literature review, dataset compilation, manuscript refinement, and research validation. **Dr. H. Balaji**'s attention to detail and ability to simplify complex concepts greatly enriched the manuscript. **J. Saritha**'s analytical insights and structured academic approach strengthened the quality of the work, while **B. Pallavi**'s timely research support and proactive involvement were invaluable throughout the project.

We are especially grateful to **Prof. A. Govardhan**, Vice-Chancellor of International Institute of Information Technology Basara, for his visionary guidance, subject expertise, and continuous academic mentorship that significantly enhanced this textbook.

We also thank our supervisors for their constant encouragement and support in keeping us aligned with emerging technologies. Our sincere appreciation goes to **Lurnexa Publications**, especially **Narendra Kumar**, for their professionalism and dedication in transforming the manuscript into its polished final form.

We gratefully acknowledge the valuable feedback provided by peer reviewers, domain experts, and seminar participants from India and abroad, whose suggestions helped refine this work.

This textbook is lovingly dedicated to our families and parents for their unwavering love, patience, sacrifices, and encouragement throughout this journey. Their support remains the foundation of this achievement.

Finally, any remaining errors or omissions that are here solely our responsibility. We hope this textbook serves as a valuable resource for students, educators, researchers, and practitioners alike.

— Dr. H Balaji, J Saritha, Pallavi B

CONTENTS

CHAPTER I
Introduction to Machine Learning

Chapter Overview

This chapter introduces the field of Machine Learning from its historical roots to the core concepts and workflows that govern modern practice. We trace the intellectual evolution of the discipline, examine the major learning paradigms, characterise the types of data encountered in real-world problems, and describe the end-to-end pipeline from raw data acquisition through feature engineering, model selection, training, evaluation, and deployment that every practitioner must understand.

1.1 Evaluation of Machine Learning

Machine Learning (ML) is a branch of artificial intelligence concerned with constructing systems that learn from experience rather than from explicit, hand-crafted rules. Its development did not occur overnight; it is the product of decades of cross-disciplinary fertilization involving statistics, neuroscience, computer science, control theory, and cognitive psychology.

1.1.1 Historical Milestones

The intellectual antecedents of ML reach back to the seventeenth century with the foundations of probability theory. However, the modern story begins in the mid-twentieth century with three pivotal developments.

The first is Alan Turing's landmark 1950 paper, which posed the question of whether machines could think and introduced the imitation game later called the Turing Test as an operational criterion for machine intelligence. This framing shifted attention from hardware to behavior, suggesting that learning was the appropriate path toward intelligent machines.

The second milestone is the Perceptron, introduced by Frank Rosenblatt in 1958. The Perceptron was a computational model inspired by the biological neuron; it demonstrated that a machine could adjust its own internal parameters called weights so as to correctly classify simple patterns. For the first time, adaptation from data was demonstrated in hardware.

The third pivotal event is the publication of Minsky and Paper's Perceptrons in 1969, which rigorously established the representational limitations of single-layer networks. Although the book was often misread as condemning neural networks altogether, its mathematical analysis stimulated the search for deeper architectures.

The 1980s brought two transformative contributions. The popularisation of the Backpropagation algorithm (Rumelhart, Hinton, and Williams, 1986) provided an efficient method for training multi-layer networks, reigniting interest in connectionism.

Simultaneously, Valiant's theory of Probably Approximately Correct (PAC) learning placed machine learning on rigorous mathematical footing, enabling formal reasoning about what can and cannot be learned from data.

The 1990s were dominated by kernel methods. Boser, Guyon, and Vapnik introduced Support Vector Machines in 1992, framing classification as a convex optimization problem with provable generalization guarantees. Ensemble methods Breiman's Bagging and Freund and Schapire's AdaBoost further demonstrated that combining simple models could achieve robustness far

beyond any individual learner.

The 2000s and 2010s witnessed the deep learning revolution. Increases in computational power, the availability of massive datasets, and algorithmic innovations dropout, batch normalization, residual connections allowed networks of extraordinary depth to surpass human performance on image recognition, speech, and natural language benchmarks. The establishment of ImageNet and the annual ILSVRC competition provided the shared benchmark that accelerated progress measurably.

Today, machine learning permeates every domain of science and industry: autonomous vehicles, drug discovery, financial risk modelling, climate simulation, personalised medicine, and human-computer interaction. Understanding its principles has become a fundamental literacy requirement for professionals across disciplines.

Era	Key Development	Significance
1950s	Turing Test; Perceptron (Rosenblatt)	Established learning as a criterion for intelligence
1960s–70s	Nearest-Neighbour; Decision Trees	Non-parametric and symbolic learning methods
1980s	Backpropagation; PAC Learning Theory	Multi-layer training; formal learnability
1990s	SVMs; Ensemble Methods; Kernel Theory	Principled generalization; strong theoretical base
2000s	Bayesian Networks; Graphical Models	Probabilistic reasoning under uncertainty
2010s–now	Deep Learning; Transformers; LLMs	Human-level perception; general- purpose models

1.2 Paradigms for Machine Learning

A learning paradigm is a fundamental framework that specifies the nature of the feedback available to the learner during training. The four principal paradigms supervised, unsupervised, semi-supervised, and reinforcement learning differ in what information is provided and what the learner is expected to produce.

1.2.1 Supervised Learning

In supervised learning, the training dataset consists of input–output pairs: each input example x(i) is accompanied by a corresponding label y(i) supplied by an external teacher or oracle. The learner's task is to find a function f such that f(x)

approximates y for new, unseen inputs. When the label is a continuous quantity (e.g., house price), the task is called regression; when the label is a discrete category (e.g., spam versus not-spam), the task is called classification.

Definition 1.2.1 Definition & Equation

Given a training set D = {(x(1), y(1)), (x(2), y(2)), ..., (x(n), y(n))} where each x(i) is drawn from an input space X and each y(i) is drawn from an output space Y, the goal is to learn a hypothesis $h : X \rightarrow Y$ that minimizes the expected prediction error on new examples drawn from the same underlying distribution.

1.2.2 Unsupervised Learning

Unsupervised learning operates without any labels. The learner is given only the raw input data X = {x(1), x(2), ..., x(n)} and must discover latent structure clusters, manifolds, distributions, or generative factors inherent in the data. Canonical tasks include clustering (grouping similar examples), dimensionality reduction (compressing data while preserving structure), density estimation, and generative modeling.

1.2.3 Semi-Supervised Learning

In many practical settings, labelling is expensive while unlabelled data is abundant. Semi-supervised learning exploits both a small labelled set and a large unlabelled set. The underlying assumption is that the geometric structure revealed by the unlabelled data clusters, low-dimensional manifolds constrains where decision boundaries should lie.

Paradigm	Input to Learner	Output of Learner	Representative Algorithms
Supervised	Labelled pairs (x, y)	Prediction function $h : X \rightarrow Y$	Linear Regression, SVM, Neural Networks
Unsupervised	Unlabelled examples x	Structure: clusters, manifolds	k-Means, PCA, Autoencoders, GMMs
Semi-Supervised	Few labels + many unlabelled	Improved prediction function	Label Propagation, Self- Training
Reinforcement	State, reward signal	Policy $\pi : S \rightarrow A$	Q-Learning, Policy Gradient, Actor-Critic

1.2.4 Reinforcement Learning

Reinforcement learning addresses sequential decision-making. An agent interacts with an environment through a cycle: observe the current state's, select an action a, receive a scalar reward r, and transition to a new state's'. The agent's goal is to learn a policy a mapping from states to actions that maximises its expected cumulative reward over time. There are no labelled training examples; the only feedback is the delayed, sparse reward signal.

1.3 Learning By Rote

Learning by rote is the most elementary form of machine learning it corresponds to pure memorisation. A rote-learning system stores every training example verbatim and reproduces stored answers for identical future queries. No generalisation takes place: if a query is identical to a stored example, the stored answer is returned; otherwise, the system cannot respond.

Definition 1.3.1 Definition & Equation

A rote-learning system constructs a look-up table T = {(x(1), y(1)), (x(2), y(2)), ..., (x(n), y(n))}.

Given a query x*, it returns y* = y(i) if x* = x(i) for some i, and produces no answer otherwise.

Rote learning has zero generalisation error on training data by construction, but its generalisation to unseen queries is zero a property sometimes described as perfect overfitting. Despite its triviality as a learning paradigm, it occupies an important conceptual position: it defines the baseline against which more sophisticated methods must improve. Any system that genuinely learns must do more than memorise; it must induce patterns that transfer to novel inputs.

Rote learning is practically useful in narrowly defined domains where the input space is finite and small, such as hard-coded game look-up tables for board positions. In database retrieval and caching systems, rote memorisation of query results is a standard optimisation technique, though such systems are not typically described as machine learning.

Example 1.3.1 Rote Learning in a Weather Prediction System
Suppose a system is trained on three days of data:

Day	Temperature (°C)	Humidity (%)	Outcome
Day 1	28	80	Rain
Day 2	15	40	No Rain
Day 3	22	65	Rain

A rote system can correctly answer queries for Day 1, Day 2, and Day 3 by look-up. When presented with Day 4 (Temperature: 20 C, Humidity: 70%), it has no answer the exact record does not exist in its table. A generalising learner (such as a decision tree) would interpolate from similar historical records and predict Rain. The contrast illustrates the fundamental limitation of rote learning.

1.4 Learning by induction

Inductive learning is the predominant paradigm in modern machine learning. The learner is given a finite set of training examples and must produce a hypothesis a general rule or function that correctly describes not only the training examples but also future, unobserved examples drawn from the same distribution. This leap from specific observations to general rules is the essence of inductive reasoning.

Definition 1.4.1 Inductive Learning Hypothesis

Given training examples D = {(x(i), f(x(i))) | i = 1, ..., n} generated by an unknown target concept f, an inductive learner produces a hypothesis h drawn from a hypothesis class H. A good hypothesis h satisfies: h(x) = f(x) for all x in D (training accuracy) and h(x) ≈ f(x) for x not in D (generalisation).

The fundamental challenge of induction is that the number of possible hypotheses consistent with any finite training set is, in general, infinite. This is known as the underdetermination problem: data alone cannot uniquely determine which hypothesis is correct. Learners must therefore impose an inductive bias a preference for certain hypotheses over others to make learning tractable. Common inductive biases include preferring simpler hypotheses

(Occam's Razor), preferring smoother functions, or preferring linear models.

1.4.1 The Inductive Bias

The No Free Lunch Theorem (Wolpert, 1996) formalises a sobering truth: no learning algorithm outperforms every other algorithm across all possible problem distributions. Every algorithm that achieves good generalisation in some domains implicitly assumes structure that may not hold in other domains. The choice of learning algorithm encodes assumptions about the data-generating process.

In practice, inductive learning encompasses most of the canonical algorithms studied in this course: decision trees, k-Nearest Neighbours, Support Vector Machines, logistic regression, and neural networks all perform inductive learning they generalise from labelled training examples to predict labels for unseen inputs.

1.4.2 Inductive Learning and Generalization

Generalisation is the ability to correctly classify examples not seen during training. It is the defining criterion by which learning algorithms are ultimately judged. A model that achieves high accuracy on the training data but low accuracy on new data is said to overfit; a model that performs poorly even on training data is said to underfit. The art of machine learning lies in finding the sweet spot a model that captures genuine patterns in the data without also capturing noise.

Example 1.4.1 Induction in Animal Classification

Training observations: Robin feathers, wings, warm-blooded: Bird. Eagle feathers, wings, warm-blooded: Bird. Salmon gills, fins, cold-blooded: Fish. Trout gills, fins, cold- blooded: Fish.

Induced rule: IF an animal has feathers AND wings AND is warm-blooded, THEN classify as Bird.

IF an animal has gills AND fins AND is cold-blooded, THEN classify as Fish.

When a new animal Hawk (feathers, wings, warm-blooded) is presented, the learner correctly generalises: Hawk is a Bird. The system has gone beyond the training data to recognise a new instance of a learned concept.

1.5. Reinforcement Learning

Reinforcement Learning (RL) addresses a fundamentally different learning scenario from supervised or inductive learning. Rather than learning from a labelled dataset, an RL agent learns through direct interaction with an environment over time. The agent must discover, through trial and error, which sequence of actions leads to the greatest cumulative reward.

Definition 1.5.1 Reinforcement Learning Framework

An RL problem is modelled as a Markov Decision Process (MDP), defined by the tuple (S, A, T, R, gamma) where S is the set of states, A is the set of actions, T : S × A → S is the transition function (possibly stochastic), R : S × A → Real is the reward function, and gamma is in [0, 1] is the discount factor controlling how much the agent values future rewards relative to immediate rewards.

1.5.1 The Agent-Environment Interaction

At each discrete time step t, the agent observes the current state s(t), selects an action a(t) according to its policy pi(s), receives a scalar reward r(t) = R(s(t), a(t)), and transitions to a new state s(t+1). The objective is to find a policy pi* that maximises the expected discounted cumulative return:

G(t) = r(t) + gamma * r(t+1) + gamma^2 * r(t+2) + ... = sum over k from 0 to infinity of gamma^k * r(t+k)

The key challenge distinguishing RL from supervised learning is the credit assignment problem: rewards may be delayed many steps after the action that caused them, making it difficult to determine which past actions were responsible for the current outcome.

1.5.2 Key Concepts in Reinforcement Learning

- **Exploration vs. Exploitation:** The agent must balance exploring unfamiliar states to discover potentially better policies against exploiting known good actions to accumulate reward.
- **Value Function:** V(s) is the expected cumulative reward from state's following policy pi. It summarises how good it is to be in state's.
- **Q-Function:** Q(s, a) is the expected cumulative reward of taking action a in state s and then following policy pi. Q-learning and Deep Q-Networks

(DQN) learn this function directly.

- **Policy Gradient:** Rather than learning a value function, policy gradient methods directly optimise the policy parameters by estimating the gradient of expected return.

Example 1.5.1 RL in a Grid World

Consider a 3x3 grid. The agent starts at position (0,0). The goal is at position (2,2), which yields a reward of +10. Each step incurs a reward of -1. Walls block movement.

Through repeated episodes, the agent learns that moving toward (2,2) leads to higher cumulative returns than wandering randomly.

The optimal policy directs the agent along the shortest path to the goal in every state, balancing the -1 step penalty against the +10 terminal reward.

1.6 Types of Data

The nature of the data available to a learning system fundamentally determines which algorithms are applicable, what preprocessing steps are required, and what kinds of patterns can be discovered. A precise understanding of data types is therefore a prerequisite for sound machine learning practice.

1.6.1 By Measurement Scale

Data attributes are classified according to the measurement scale of their values. This classification, originating with Stevens (1946), has profound implications for the choice of distance metrics, statistical operations, and learning models.

Scale Type	Properties	Permitted Operations	Examples
Nominal	Categories with no order	Equality, mode	Blood type, city name, species
Ordinal	Ordered categories; differences meaningless	Inequality, median, rank	Pain level (mild/moderate/severe)
Interval	Ordered; differences meaningful; no true zero	Addition, mean, std deviation	Temperature (Celsius), calendar year
Ratio	Interval + true zero; ratios meaningful	All arithmetic operations	Height, weight, income, duration

1.6.2 By Structure

- **Structured Data:** Data organised in a well-defined schema, typically in tabular form where each row is an observation and each column is a feature with a consistent data type. Relational databases contain structured data. Most classical ML algorithms are designed for structured tabular data.
- **Semi-Structured Data:** Data that does not conform to a rigid schema but contains tags or markers that provide some organisational information. JSON files, XML documents, and log files are common examples.
- **Unstructured Data:** Data with no predefined schema. Raw text, images, audio waveforms, and video streams are unstructured. Deep learning methods are particularly effective at extracting features from unstructured data automatically.

1.6.3 Temporal

- **Cross-Sectional Data:** Observations collected from multiple subjects at a single point in time. Each row represents a different entity (person, item, transaction).
- **Time-Series Data:** Sequential observations of a single entity measured at regular time intervals. Stock prices, weather readings, and sensor logs are time-series data. Temporal order carries information and must be preserved during preprocessing.
- **Panel (Longitudinal) Data:** Multiple entities each observed over multiple time points. Panel data combines cross-sectional breadth with time-series depth.

1.6.4 Feature Types within a Dataset

Within a structured dataset, individual features (columns) are further classified as:

Feature Type	Definition	Example
Binary	Takes exactly two values	Is_spam: {0, 1}
Discrete / Count	Integer-valued; countable	Number of clicks, word frequency
Continuous	Real-valued; uncountably many values	Temperature, probability score
Categorical	Finite set of unordered labels	Colour: {red, green, blue}

Ordinal	Finite set of ordered labels	Rating: {1, 2, 3, 4, 5}
Text / String	Variable-length character sequences	Review text, product description
Date / Time	Calendar or timestamp values	Purchase date, event timestamp

1.7 Matching

Matching refers to the process of determining correspondence or similarity between data objects. It is a fundamental operation in many machine learning tasks: classification by similarity, information retrieval, duplicate detection, entity resolution, and recommendation all rely on some form of matching

$$d_p(x, y) = \left(\sum_{i=1}^{d} |x_i - y_i|^p \right)^{1/p}$$

Definition 1.7.1 Similarity and Distance

A distance function d : X × X → Real is a metric if it satisfies: (1)

Non-negativity: d(x, y) >= 0 for all x, y; (2) Identity: d(x, y) = 0 if and only if x = y;

(3) Symmetry: d(x, y) = d(y, x); (4) Triangle inequality: d(x, z) <= d(x, y) + d(y, z).

A similarity function s(x, y) is typically defined so that s(x, y) = 1 when x and y are identical and s(x, y) = 0 when they are completely dissimilar.

1.7.1 Distance Metrics for Numeric Data

For continuous numeric feature vectors, the Minkowski family of metrics provides a flexible family of distance measures:
Special cases of particular importance are:

- **Euclidean Distance (p = 2):** $d(\mathbf{x}, \mathbf{y}) = \sqrt{(x_1 - y_1)^2 + (x_2 - y_2)^2 + \ldots + (x_n - y_n)^2}$

The Straight-line distance between two points. Sensitive to scale; features should be normalised before use.

- **Manhattan Distance (p = 1):** $d(x, y) = \sum_{i=1}^{d} |x_i - y_i|$

Also called the L1 or city-block distance. Less sensitive to outliers than Euclidean distance.

• **Chebyshev Distance (p → infinity):** | $\mathbf{d(x, y) = \|x - y\|_\infty = \max_j |x_j - y_j|}$

The maximum difference along any single coordinate. Used in board game move metrics.

1.7.2 Similarity Measures for Text and Sets

For non-numeric data, dedicated similarity functions are required:

• **Cosine Similarity:** $s(x, y) = \frac{x \cdot y}{\|x\| \, \|y\|}$

Measures the angle between two vectors. Widely used for text documents represented as bag-of-words or TF-IDF vectors.

• **Jaccard Similarity:** $s(A, B) = \frac{|A \cap B|}{|A \cup B|}$

where A and B are sets. Ranges from 0 (disjoint) to 1 (identical). Used for comparing document word sets, user preferences, or binary feature vectors.

• **Edit Distance (Levenshtein):** The minimum number of single-character insertions, deletions, or substitutions required to transform one string into another. Fundamental in spell checking and biological sequence alignment.

Example 1.7.1 Computing Euclidean Distance

Given two data points in two-dimensional feature space:

Point A = (3, 4); Point B = (0, 0).

Euclidean distance:.

$$d(A, B) = \sqrt{(3-0)^2 + (4-0)^2}$$
$$= \sqrt{9 + 16} = \sqrt{25} = 5$$

Manhattan distance: d(A, B) = |3 - 0| + |4 - 0| = 3 + 4 = 7.

Interpretation: A and B are 5 units apart in straight-line terms but 7 units apart in city-block terms.

The choice of metric affects which neighbours are considered closest, and therefore affects k-NN classification outcomes.

1.8 Stages in Machine Learning

Building a machine learning system is not a single-step process; it is a structured pipeline of interdependent stages. Each stage transforms the product of the previous stage into a form suitable for the next. Understanding this pipeline and the possible failure modes at each stage is essential for both research and engineering.

The major stages of a machine learning project are: Data Acquisition, Feature Engineering, Data Representation, Model Selection, Model Learning, Model Evaluation, and Model Prediction. These stages are often iterative rather than

strictly sequential; findings at a later stage frequently require revisiting earlier decisions.

Figure 1.1: The Machine Learning Pipeline

Data Acquisition	Feature Engineering	Data Representation	Model Selection	Model Learning	Model Evaluation	Model Prediction

1.9 Data Acquisition

Data acquisition is the process of gathering raw data from the sources that will serve as the empirical foundation of the learning system. The quality and representativeness of the acquired data impose a hard ceiling on the performance of any downstream model a ceiling that no algorithmic sophistication can transcend. The adage commonly expressed as 'Garbage In, Garbage Out' captures this dependency precisely.

1.9.1. Sources of Data

Source Category	Description	Examples
Manual Collection	Human-labelled or survey-collected data	Medical annotations, crowdsourcing (Amazon MTurk)
Instrumentation	Data collected by physical sensors or devices	IoT sensors, medical devices, GPS trackers
Web Scraping	Automated extraction from web pages	Product reviews, social media posts, news articles
Transactional Systems	Operational databases	E-commerce logs, banking transactions
Open Repositories	Publicly available datasets	UCI ML Repository, Kaggle, ImageNet, Wikipedia
Simulation	Synthetically generated data	Physics simulations, game environments, GAN-generated images

1.9.2 Data Quality Considerations

Raw data is rarely clean. Practitioners must audit acquired data along several quality dimensions before proceeding:

- **Completeness:** Are there missing values? For which features and in what proportion? Missing data may be missing completely at random (MCAR),

missing at random (MAR), or missing not at random (MNAR). The missingness mechanism determines appropriate imputation strategies.

- **Accuracy:** Do the recorded values reflect the true underlying measurements? Inaccuracies arise from sensor noise, human error in data entry, or systematic miscalibration.
- **Consistency:** Are the same entities described consistently across sources? Inconsistent naming conventions, unit mismatches, and conflicting records create downstream problems.
- **Relevance:** Does the acquired data actually reflect the population and distribution on which the model will be deployed? Training on data from one distribution and deploying on another leads to distribution shift one of the most common causes of real-world model failure.
- **Timeliness:** Is the data sufficiently recent? Concepts can drift over time; a model trained on historical data may not reflect current patterns.

1.9.3 Data Labeling

For supervised learning, acquired data must be annotated with correct output labels. Labelling is often the most time-consuming and expensive part of a project. Strategies for obtaining labels include expert annotation, crowd-sourcing, programmatic labelling (Snorkel), and active learning (selectively querying the most informative unlabelled examples for human annotation).

1.10 Feature Engineering

Feature engineering is the art and science of transforming raw data attributes into a form that is maximally informative for the learning algorithm. A well-engineered feature set can dramatically improve model performance even when the underlying algorithm is simple; conversely, even the most powerful algorithm will struggle with poorly constructed features.

Definition 1.10.1 Feature

A feature (also called an attribute, predictor, or covariate) is a measurable property of the phenomenon under study. A feature vector x = (x1, x2, ..., xd) is a tuple of d feature values representing a single data instance. The feature space X = R^d is the d-dimensional space in which all instances live.

1.10.1 Feature Selection

Feature selection is the process of identifying and retaining only those features that are genuinely informative for the prediction task, discarding irrelevant and redundant features. It serves two purposes: improving model generalisation by reducing overfitting, and reducing computational cost by lowering dimensionality.

- **Filter Methods:** Evaluate feature relevance using a statistical measure (e.g., mutual information, chi-squared test, correlation coefficient) independently of any learning algorithm. Fast but ignores feature interactions.
- **Wrapper Methods:** Evaluate subsets of features by training a model on each subset and measuring validation performance. Computationally expensive but accounts for interactions between features and the learning algorithm.
- **Embedded Methods:** Incorporate feature selection into the model training process itself. Lasso regression (L1 regularisation) drives irrelevant feature weights to exactly zero; tree-based feature importances rank features during tree construction.

1.10.2 Feature Extraction and Transformation

Feature extraction creates new features from existing ones, often by projecting data into a lower- dimensional space that retains essential information. Principal Component Analysis (PCA) finds the directions of maximum variance; Independent Component Analysis (ICA) finds statistically independent components; autoencoders learn non-linear compressed representations.

Feature transformation encompasses operations that change the scale or distribution of individual features without changing their number:

- **Min-Max Normalisation:** x' = (x - x_min) / (x_max - x_min). Scales values to [0, 1]. Sensitive to outliers.
- **Z-Score Standardisation:** x' = (x - mean(x)) / std(x). Produces zero mean and unit variance. Preferred for algorithms that assume Gaussian distributions or use distance metrics.
- **Log Transformation:** x' = log(x). Compresses right-skewed distributions, useful for count data or income.
- **One-Hot Encoding:** Converts a categorical feature with K categories into K binary indicator features, one for each category.

1.10.3 Feature Creation

Domain knowledge often enables the creation of new features from

combinations of existing ones. For example, in a house price dataset, the ratio of living area to total lot area may be more predictive than either feature individually. In time-series applications, lag features (previous time steps' values), rolling averages, and rate-of-change derivatives are commonly engineered.

Example 1.10.1 Feature Engineering for Loan Default Prediction

Raw features: Annual Income, Total Debt, Number of Credit Accounts, Months Since Last Payment.

Engineered features:

1. Debt-to-Income Ratio = Total Debt / Annual Income. This single derived feature is more predictive than either component because lenders explicitly use it as a risk criterion.

2. Payment Recency Flag = 1 if Months Since Last Payment > 6 else 0. Converts a continuous variable into a binary risk indicator.

3. Log Annual Income = log(Annual Income). Compresses the right-skewed income distribution, making it more amenable to linear models.

4. Account Density = Number of Credit Accounts / (Annual Income / 10000). Normalises account count by income level.

The engineered feature set typically yields substantially lower default prediction error than the raw features alone.

1.11 Data Representation

Data representation concerns the formal mathematical encoding of raw data objects into structures that learning algorithms can process. The choice of representation is not neutral it determines which patterns are accessible to the algorithm, how computationally tractable training will be, and what kinds of inductive biases are implicitly built into the system.

1.11.1 Vector Space Representation

The most prevalent representation in classical machine learning is the feature vector: each data instance is encoded as a fixed-length vector x in R^d. The i-th component x_i represents the value of the i-th feature for that instance. This representation assumes that all relevant information about an instance can be expressed as a finite tuple of numbers.

1.11.2 Representation for Structured Data Types

Data Type	Representation	Notes
Numeric features	Raw values in R^d	Normalisation often required
Categorical features	One-hot encoding; binary vectors	k categories → k-dimensional binary vector
Text	Bag-of-words; TF-IDF; word embeddings (Word2Vec, GloVe)	Embeddings capture semantic similarity
Images	Pixel intensity matrices; CNN feature maps	Spatial structure preserved in grid representation
Graphs	Adjacency matrix; node feature matrix	Graph Neural Networks exploit relational structure
Time series	Fixed-length windows; lag feature vectors	Stationarity preprocessing often required

1.11.3 Dimensionality and the Curse of Dimensionality

As the dimensionality d of the feature space grows, the volume of the space grows exponentially. This has a devastating consequence for distance-based methods: in high-dimensional spaces, data points become approximately equidistant from one another, so the contrast between nearest and farthest neighbours vanishes. A training set of size n that densely covers a one-dimensional space requires n^d points to achieve the same density in d dimensions an exponentially larger dataset.

Practically, this means that algorithms relying on local distance comparisons (such as k-Nearest Neighbours) degrade rapidly with dimensionality. Dimensionality reduction through feature selection, PCA, or learned embeddings is therefore not merely a convenience but a necessity when feature spaces are high-dimensional.

1.12 Model Selection

Model selection is the process of choosing the algorithm, hypothesis class, and hyperparameter configuration that is most appropriate for a given task. It is a decision taken before seeing the test data; it must therefore be guided by principled methodologies to avoid inadvertently 'peeking' at test performance.

1.12.1 The Bias-Variance Trade-off

Every supervised learning algorithm makes commitments about the form of the

hypothesis it can represent. These commitments introduce bias systematic errors that arise when the true function lies outside the hypothesis class. At the same time, a model that is too flexible will fit the specific training data too closely, exhibiting high variance sensitivity to random fluctuations in the training sample.

The expected prediction error on unseen data decomposes as:

Expected Error = (Bias)^2 + Variance + Irreducible Noise

Bias^2 is the squared error of the best hypothesis in the class relative to the true function. Variance is the variation in the learned hypothesis across different training sets. Irreducible Noise is the inherent randomness in the data-generating process that no model can eliminate.

Simple models (e.g., linear models) have high bias and low variance. Complex models (e.g., deep neural networks with many parameters) have low bias and high variance. The optimal model complexity balances these two sources of error.

1.12.2 Cross-Validation

To estimate the generalisation error of a model without using the test set, practitioners use cross- validation. In k-fold cross-validation, the training data is partitioned into k equally sized subsets (folds). The model is trained on k-1 folds and evaluated on the remaining fold. This is repeated k times, each fold serving as the validation set exactly once. The k validation scores are averaged to produce an estimate of generalisation performance.

When data is scarce, Leave-One-Out Cross-Validation (LOOCV) sets k = n each training example serves as the sole validation point in one of n training runs. While maximally data-efficient, LOOCV is computationally expensive for large n.

1.12.3 Hyperparameter Tuning

Most learning algorithms have hyperparameters configuration settings that are not learned from data but must be set before training. Examples include the regularisation coefficient lambda in ridge regression, the depth limit in decision trees, and the number of neurons in each layer of a neural network.

Strategies for tuning hyperparameters include:

- **Grid Search:** Exhaustively evaluate all combinations of a discrete set of candidate values for each hyperparameter. Guaranteed to find the best combination within the grid but exponentially expensive in the number of hyperparameters.

- **Random Search:** Sample hyperparameter combinations randomly from a specified distribution. More efficient than grid search when not all hyperparameters are equally important.
- **Bayesian Optimisation:** Model the hyperparameter-performance relationship with a probabilistic surrogate model (e.g., Gaussian Process) and use an acquisition function to select the next configuration to evaluate. Most sample-efficient approach.

Method	Strengths	Weaknesses	Recommended When
Grid Search	Thorough; reproducible	Exponential cost;impractical for many hyperparameters	Few hyperparameters; small search space
Random Search	Efficient; scales well	No guarantee of finding best	Many hyperparameters; budget-constrained
Bayesian Optimisation	Sample efficient; principled	Implementation complexity	Expensive evaluations (e.g., deep learning)

1.13 Model Learning

Model learning also called training or fitting is the computational process of optimising the model's parameters with respect to a loss function defined over the training data. The outcome of model learning is a set of parameter values that, among all values in the hypothesis class, best explains the training observations according to the chosen criterion.

Definition 1.13.1 Empirical Risk Minimisation

Given a training dataset D = {(x(i), y(i)) | i = 1, ..., n}, a hypothesis class H, and a loss function L(y, y_hat), the empirical risk minimiser is: h* = argmin over h in H of (1/n) * sum over i from 1 to n of L(y(i), h(x(i))). The quantity (1/n) * sum of L(y(i), h(x(i))) is called the empirical risk or training loss.

1.13.1 Common Loss Functions

Task	Loss Function	Formula	Intuition
Regression	Mean Squared Error (MSE)	(1/n) sum (y_i - y_hat_i)^2	Penalises large errors quadratically
Regression	Mean Absolute Error (MAE)	(1/n) sum |y_i - y_hat_i|	Robust to outliers; non-differentiable at 0
Classification	Cross-Entropy (Log Loss)	-(1/n) sum [y_i log(p_i) + (1-y_i) log(1-p_i)]	Penalises confident wrong predictions most
Classification	Hinge Loss (SVM)	(1/n) sum max (0, 1 - y_i * f(x_i))	Penalises margin violations; zero for correct, wide-margin predictions

1.13.2 Gradient Descent

Gradient descent is the workhorse optimisation algorithm for training parameterised models. Given a differentiable loss function L(theta) where theta represents the model parameters, gradient descent iteratively moves the parameters in the direction of steepest descent (negative gradient):

theta(t+1) = theta(t) - eta * gradient_theta L(theta(t))

Here eta > 0 is the learning rate, which controls the step size. If eta is too large, the update may overshoot the minimum and diverge; if too small, convergence is unnecessarily slow.

Three common variants differ in how much data is used to compute each gradient estimate:

- **Batch Gradient Descent:** Computes the gradient using all n training examples. Exact but computationally expensive for large datasets; updates are stable.
- **Stochastic Gradient Descent (SGD):** Computes the gradient using a single randomly selected training example. Very fast per update; noisy gradient estimates provide implicit regularisation but lead to erratic convergence.
- **Mini-Batch Gradient Descent:** Computes the gradient using a small batch of B examples (typically B in {32, 64, 128, 256}). Balances computational efficiency with gradient stability. The standard choice in deep learning.

1.13.3 Regularization during learning

Regularization modifies the training objective to penalise model complexity, thereby discouraging overfitting. The regularised objective is:

Regularized Objective = Empirical Risk + lambda * Complexity Penalty

L2 (Ridge) regularization adds lambda * sum_j (w_j)^2, encouraging all weights to be small. L1 (Lasso) regularisation adds lambda * sum_j |w_j|, encouraging sparse weight vectors where many weights are exactly zero. The strength of regularisation is controlled by the hyperparameter lambda.

1.14 Model Evaluation

Model evaluation is the systematic assessment of a trained model's predictive performance on data it has not seen during training. Rigorous evaluation is essential: training accuracy is not a reliable proxy for generalisation; a model may achieve near-perfect training accuracy while performing poorly on new data.

1.14.1 Train, Validation and Test Sets

The standard practice is to partition the available data into three non-overlapping subsets:

- **Training Set (typically 60–70%):** Used to train model parameters. The model sees these examples during optimisation.
- **Validation Set (typically 10–20%):** Used to tune hyperparameters and perform model selection. The validation set is not seen during parameter optimisation but is implicitly used to guide model choices.
- **Test Set (typically 10–20%):** Used only once, at the very end, to produce the final unbiased estimate of generalisation performance. Any use of the test set during development contaminates the evaluation.

1.14.2 Evaluation Metrics for Classification

For binary classification with outcomes Positive (1) and Negative (0), predictions are characterised by a 2x2 Confusion Matrix:

	Predicted Positive	Predicted Negative
Actual Positive	True Positive (TP)	False Negative (FN)
Actual Negative	False Positive (FP)	True Negative (TN)

From the confusion matrix, key metrics are derived:

- **Accuracy:** (TP + TN) / (TP + TN + FP + FN). Proportion of correct predictions. Misleading when class frequencies are imbalanced.
- **Precision:** TP / (TP + FP). Of all instances predicted positive, what fraction are actually positive? Measures the quality of positive predictions.
- **Recall (Sensitivity):** TP / (TP + FN). Of all actual positive instances, what fraction did the model correctly identify? Measures completeness of positive prediction.
- **F1 Score:** 2 * (Precision * Recall) / (Precision + Recall). Harmonic mean of precision and recall. A single balanced measure when both matter.
- **ROC-AUC:** Area Under the Receiver Operating Characteristic Curve. Measures how well the model ranks positive examples above negative ones across all decision thresholds. 0.5 = random; 1.0 = perfect.

1.14.3 Evaluation Metrics for Regression

- **Mean Squared Error (MSE):** (1/n) * sum over i of (y_i - y_hat_i)^2. Differentiable; penalises large errors heavily.
- **Root Mean Squared Error (RMSE):** sqrt(MSE). Same units as the target variable; more interpretable than MSE.
- **Mean Absolute Error (MAE):** (1/n) * sum over i of |y_i - y_hat_i|. Robust to outliers; directly interpretable.
- **Coefficient of Determination (R-squared):** 1 - (sum (y_i - y_hat_i)^2) / (sum (y_i - mean_y)^2). Proportion of variance explained by the model. Ranges from negative infinity to 1; a value of 1 indicates perfect prediction.

Example 1.14.1 Evaluating a Spam Classifier

A spam classifier is evaluated on a test set of 1000 emails: 900 non-spam and 100 spam. Results:

True Positives (spam correctly identified): 80. False Negatives (spam missed): 20. False Positives (non-spam misclassified as spam): 45. True Negatives (non-spam correctly classified): 855.
Metrics: Accuracy = (80 + 855) / 1000 = 935 / 1000 = 0.935 (93.5%).

Precision = 80 / (80 + 45) = 80 / 125 = 0.640 (64.0%).

Recall = 80 / (80 + 20) = 80 / 100 = 0.800 (80.0%).

F1 Score = 2 * (0.640 * 0.800) / (0.640 + 0.800) = 1.024 / 1.440 = 0.711 (71.1%).

Interpretation: Accuracy appears high (93.5%) but is partly inflated by the large majority class. Precision (64%) is moderate many false alarms. Recall (80%) is acceptable most spam is caught. The F1 score of 71.1% provides a balanced summary.

1.15 Model prediction

Model prediction is the operational phase in which the trained and evaluated model is deployed to produce outputs for new, previously unseen inputs. Prediction is the ultimate purpose of machine learning; all preceding stages exist in service of building a model that performs well in this phase.

1.15.1. Inference Procedure

The prediction process for a trained model h with parameters theta* is conceptually simple: given a new input x*, compute h(x*; theta*) to obtain the predicted output y_hat*. In practice, the inference pipeline involves the same preprocessing and feature engineering transformations applied to the training data, computed now on the new input:

1. Apply the same normalisation or standardisation used during training (using the statistics computed from training data, not from x* alone).
2. Apply the same feature extraction and encoding transformations.
3. Compute h(x*; theta*) using the trained model parameters.
4. Apply any output postprocessing: convert probability scores to class labels using a threshold; inverse-transform scaled regression outputs to the original units.

1.15.2 Batch Vs Online Prediction

- **Batch Prediction:** The model processes a large collection of inputs simultaneously, generating predictions for all of them before any result is delivered. Common in scheduled reporting pipelines and scientific applications where latency is not critical.
- **Online (Real-Time) Prediction:** The model produces a prediction for each input immediately as it arrives, with strict latency requirements. Common in fraud detection, recommendation systems, autonomous driving, and search ranking.

1.15.3 Confidence and Uncertainty Quantification

Raw predictions alone are often insufficient for high-stakes decisions. Practitioners increasingly require models to express their uncertainty to communicate not only what the prediction is but how confident the model is in that prediction. Probabilistic models (logistic regression, Gaussian Processes, Bayesian neural networks) provide calibrated probability outputs. Conformal prediction provides distribution-free prediction intervals with guaranteed coverage.

1.16 Search and Learning

Machine learning can be viewed from a computational perspective as a search problem: the learning algorithm searches through the space of possible hypotheses to find one that fits the training data well and generalises to new data. This perspective unifies many seemingly disparate learning methods and connects machine learning to classical artificial intelligence.

Definition 1.16.1 Hypothesis Space Search

Let H be the hypothesis class and L : H × D → Real be a loss functional mapping hypotheses and training data to a real-valued performance measure. Model learning is the search: h* = argmin over h in H of L(h, D_train). The search space is H, the objective function is L, and the search algorithm (gradient descent, evolutionary algorithm, constraint satisfaction) determines how the space is traversed.

1.16.1 The Size of the Hypothesis space

The hypothesis space can be combinatorially or continuously infinite. For a Boolean function over d binary features, the space of all possible functions has

size 2^(2^d) doubly exponential. For a linear model over d real-valued features, the hypothesis space is all of R^(d+1), which is continuous and uncountably infinite. Tractable learning requires some mechanism regularisation, parameterization, or structural constraints to make the search feasible.

1.16.2 Search strategies

Strategy	Description	Used In
Gradient Descent	Follow the negative gradient of the loss surface	Neural networks; logistic regression; SVMs
Greedy Search	At each step, take the locally best action	Decision tree construction (ID3, C4.5, CART)
Beam Search	Maintain top-k partial solutions at each step	Sequence-to-sequence models; NLP
Genetic / Evolutionary	Evolve population of candidates via selection, crossover, mutation	Neural architecture search; AutoML
Constraint Satisfaction	Express as constrained optimisation; use Lagrangian methods	SVM (quadratic programming)
Dynamic Programming	Decompose into subproblems; memoize solutions	HMMs; Viterbi algorithm

1.16.3. Over fitting as a Search Failure

Overfitting can be reframed as a search failure: the algorithm has found a hypothesis that is an excellent solution to the empirical search problem (minimising training loss) but a poor solution to the true problem (minimising expected loss on the underlying distribution). This occurs when the hypothesis class is too expressive relative to the available training data, allowing the model to exploit idiosyncratic noise in the training sample.

Regularisation, early stopping, and data augmentation are all techniques that modify the search to prefer simpler or more robust hypotheses even if they do not fully minimise training loss.

1.17 Data Sets

In machine learning practice, the term dataset refers to a collection of data instances used to train, validate, or test a model. The properties of a dataset its size, dimensionality, class balance, noise level, and the nature of its features have profound effects on which learning algorithms are applicable and how well they can be expected to perform.

1.17.1 Dataset partitioning

A fundamental principle of model development is that the data used to train a model must be kept separate from the data used to evaluate it. This separation prevents optimistic bias the tendency for a model to appear better than it actually is when evaluated on data it has already seen.

The training set is used to fit model parameters. The validation set (also called the development set) is used to tune hyperparameters and compare models. The test set is the final, unbiased evaluation dataset, used only once after all modelling decisions have been finalised.

It is critical that the partitioning respect any temporal or structural dependencies in the data. For time- series data, the test set must consist of time points strictly later than the training set; random shuffling before splitting would introduce data leakage.

1.17.2 Class Imbalance

Many real-world classification datasets exhibit class imbalance: one class (the majority class) constitutes a much larger fraction of the dataset than the other (the minority class). For example, in fraud detection, fewer than 0.1% of transactions may be fraudulent. A naive classifier that always predicts the majority class achieves high accuracy but is practically useless.

Strategies for handling class imbalance include:

- **Oversampling the Minority Class:** Duplicate minority class examples or generate synthetic ones (SMOTE: Synthetic Minority Over-sampling Technique).
- **Undersampling the Majority Class:** Remove examples from the majority class to balance the training set. Risk of discarding useful information.
- **Cost-Sensitive Learning:** Assign higher misclassification costs to minority class errors, directly encoding the real-world asymmetry in consequences.
- **Evaluation Metrics:** Use metrics that are not confounded by imbalance F1 score, precision- recall curves, and ROC-AUC rather than raw accuracy.

1.17.3 Benchmark Datasets

Benchmark datasets play a central role in machine learning research, providing standardised tasks and evaluation protocols that allow fair comparison of algorithms across publications. The following are among the most widely used:

Dataset	Task	Instances	Features	Notable Use
Iris (Fisher, 1936)	Multi-class classification	150	4 numeric	Introductory ML tutorial; linear separability illustration
MNIST	Digit recognition (0–9)	70,000	784-pixel intensities	Benchmark for image classification; deep learning baseline
CIFAR-10	Object recognition (10 classes)	60,000	3072 (32x32 RGB)	CNN architecture benchmarking
ImageNet (ILSVRC)	1000-classimage classification	1.2 million	Variable resolution images	Deep learning revolution; ResNet, VGG, Inception
UCI Adult Income	Binary classification (income > 50K)	48,842	14 mixed	Fairness and bias research
Boston Housing	Regression (house price)	506	13 numeric	Regression methods illustration
20 News groups	Text classification (20 topics)	18,846	Word frequencies	NLP; document categorisation

1.17.4 Data Augmentation

When the available training data is limited, data augmentation artificially expands the training set by applying label-preserving transformations to existing examples. For image data, common augmentations include random horizontal flipping, random cropping, rotation, colour jitter, and Gaussian noise injection. For text, augmentation techniques include synonym replacement, random insertion, and back-translation (translating to another language and back). Data augmentation is both a regularisation technique and a means of improving model robustness to input variations.

Chapter Summary

This chapter provided a comprehensive foundation for the study of machine learning. We traced the discipline's evolution from the Perceptron and Turing's imitation game through the deep learning revolution, situating modern practice in its historical context.

We distinguished the four principal learning paradigms: supervised learning (labelled input-output pairs), unsupervised learning (structure discovery from unlabelled data), semi-supervised learning (combining few labels with abundant unlabelled data), and reinforcement learning (sequential decision-making via reward feedback). Within supervised learning, we contrasted learning by rote pure memorisation without generalisation with learning by induction, which extracts general rules from specific observations.

We characterised the types of data encountered in practice: nominal, ordinal, interval, and ratio by measurement scale; structured, semi-structured, and unstructured by format; cross-sectional and time- series by temporal character. We examined matching and similarity, introducing the Euclidean, Manhattan, cosine, and Jaccard metrics as tools for measuring correspondence between data objects.

The machine learning pipeline was presented stage by stage: data acquisition (gathering representative, quality data); feature engineering (transforming raw attributes into informative representations); data representation (encoding instances as mathematical objects); model selection (choosing algorithm and hyperparameters using cross-validation and the bias-variance trade-off); model learning (optimising parameters via empirical risk minimisation and gradient descent); model evaluation (measuring generalisation performance using appropriate metrics for the task); and model prediction (deploying the trained model for real-time or batch inference).

Finally, we examined the search perspective on learning viewing training as a search through hypothesis space and the properties of datasets, including benchmark datasets, class imbalance, and data augmentation.

Review Question

Section 1.1 Evolution of Machine Learning

1. Describe three milestone events in the history of machine learning and explain how each contributed to the development of the field as it exists today.
2. What was the significance of Minsky and Papert's Perceptrons (1969) for machine learning research? Did it mark an end or a turning point?
3. Explain why the availability of large datasets and GPU computing in the 2010s proved decisive for the success of deep learning.

Section 1.2–1.5 Learning Paradigms

4. Distinguish supervised, unsupervised, semi-supervised, and reinforcement learning. For each, give a practical application domain where it is the most natural paradigm.
5. A hospital has 10,000 patient records, of which only 200 have been labelled by physicians with diagnoses. Which learning paradigm is most appropriate? Justify your answer.
6. Explain the credit assignment problem in reinforcement learning. How do temporal difference methods address it?
7. Define the Markov Decision Process (MDP) formally and explain the role of the discount factor gamma in the agent's objective.

Section 1.6–1.7 Data Types and Matching

8. A dataset contains the following features: ZIP code, star rating (1–5), temperature in Kelvin, and customer gender. Classify each feature by measurement scale (nominal, ordinal, interval, ratio) and justify your classification.
9. Given points A = (1, 2, 3) and B = (4, 6, 3) in three-dimensional space, compute: (a) Euclidean distance; (b) Manhattan distance; (c) Chebyshev distance.
10. Two documents are represented as word sets: Doc1 = {machine, learning, data, model} and Doc2 = {data, model, neural, network}. Compute their Jaccard similarity. Interpret the result.

Section 1.9–1.10 Data Acquisition and Feature Engineering

11. List and explain five dimensions of data quality that must be assessed

during data acquisition. What are the consequences of neglecting each?

12. A feature 'Annual Income' ranges from 20,000 to 10,000,000 with a strong right skew. What transformation would you apply before training a linear model? Why?
13. A raw dataset contains a categorical feature 'Country' with 195 possible values. Discuss the trade-offs between one-hot encoding and ordinal encoding for this feature.

Section 1.12–1.13 Model Selection and Learning

14. Explain the bias-variance trade-off using the decomposition: Expected Error = Bias^2 + Variance + Irreducible Noise. Give an example of a high-bias and a high-variance model.
15. Why is it incorrect to select a model based on its training accuracy and then evaluate it on the same training data?
16. Compare batch gradient descent, stochastic gradient descent, and mini-batch gradient descent with respect to computational cost, gradient noise, and convergence behaviour.
17. A ridge regression model with lambda = 0.001 achieves training MSE of 2.3 and validation MSE of 8.7. A ridge model with lambda = 10 achieves training MSE of 4.1 and validation MSE of 4.8. Which model would you prefer and why? What does this observation tell you about the first model?

Section 1.14 Model Evaluation

18. A classifier for detecting a rare disease achieves 99.5% accuracy on a test set where 0.4% of cases are positive. Is this a good classifier? Compute precision, recall, and F1 score if TP = 20, FP = 30, FN = 20, TN = 4930.
19. Explain the difference between precision and recall in the context of web search (precision: are returned documents relevant?) versus disease screening (recall: are all diseased patients identified?).
20. What is the purpose of maintaining a held-out test set that is never used during model development? What constitutes data leakage and how can it arise inadvertently?

Section 1.16–1.17 Search and Datasets

21. Frame the training of a decision tree as a search problem: define the hypothesis space, the objective function, and the search strategy.
22. A dataset for fraud detection contains 100,000 transactions, of which 200

are fraudulent. Describe three strategies for handling this class imbalance and the trade-offs of each.

23. What properties make a dataset suitable for use as a benchmark? Why is it important for the machine learning community to have shared benchmarks?
24. Explain why random train-test splitting is inappropriate for time-series data. How should the split be performed instead.

CHAPTER II
Nearest Neighbor-Based Models

Chapter Overview

This unit introduces the family of instance-based learning algorithms that rely on the notion of proximity to make predictions. Unlike parametric models that condense training data into a fixed set of parameters, nearest-neighbor methods retain the entire training set and make decisions by comparing new observations against stored examples. The unit begins with the mathematical foundations of measuring closeness between data points, progresses through classical and advanced nearest-neighbor classifiers, and concludes with regression variants and a thorough treatment of classifier performance evaluation.

By the end of this unit, the reader will be equipped to: (1) select an appropriate distance or similarity measure for a given data type, (2) implement and tune K-Nearest Neighbor and radius-based classifiers, (3) apply KNN regression with kernel weighting, and (4) rigorously evaluate any classifier or regress or using industry-standard metrics.

2.1 Introduction to Proximity Measures

2.1.1 Why Proximity?

At the heart of nearest-neighbor learning lies a deceptively simple assumption: objects that are close to one another in some measurable sense are likely to belong to the same class or to share similar output values. This principle is known as the continuity assumption or the smoothness assumption of the input-output mapping. Although it does not hold universally, it is remarkably effective across a wide range of practical domains, from medical diagnosis to recommendation systems.

Before any nearest-neighbor algorithm can operate, we must answer a prior question: how do we define "closeness"? The answer depends critically on the nature of the data. Consider two contrasting scenarios. In the first, a hospital wishes to classify patients as diabetic or non-diabetic based on blood glucose level (a real-valued measurement in mg/dL) and body mass index (another real number). Here, ordinary arithmetic differences make intuitive sense: a patient with glucose = 110 mg/dL is closer to a patient with glucose = 115 mg/dL than to one with glucose = 200 mg/dL. In the second scenario, an e-commerce site wishes to measure how similar two customers are based on which product categories they have purchased from (a binary yes/no for each category). Arithmetic differences are now meaningless; we need a different notion of proximity based on the overlap of purchased categories.

A proximity measure is a formal mathematical object that quantifies the degree of similarity or dissimilarity between two data objects. The two fundamental types are:

- A distance measure (also called a dissimilarity measure) assigns a non-negative real number $d(x, y)$ to every pair of objects x and y, with smaller values indicating greater similarity.
- A similarity measure assigns a value $s(x, y)$, typically scaled to $[0, 1]$, with larger values indicating greater similarity.

The two types are interchangeable via monotone transformations. For instance, given a similarity $s(x, y)$ in $[0, 1]$, the quantity $1 - s(x, y)$ is a valid dissimilarity. The choice of which form to use depends on algorithmic convenience and on the properties required.

2.1.2 Formal Properties of a Metric

When a distance function satisfies a set of axiomatic properties, it is called a

metric and the space it defines is called a metric space. These properties are not merely mathematical formalities; they have direct algorithmic consequences.

Definition 2.1 — Metric

A function d : X x X -> R is called a metric on a set X if, for all x, y, z in X, it satisfies: (M1) Non-negativity: d(x, y) >= 0, with equality if and only if x = y. (M2) Symmetry: d(x, y) = d(y, x). (M3) Triangle inequality: d(x, z) <= d(x, y) + d(y, z).

The non-negativity property ensures that distances are well-defined as quantities on the real line. The identity of indiscernibles (equality holds if and only if the points are identical) ensures that the metric separates distinct points. Symmetry means that the distance from city A to city B equals the distance from city B to city A — this holds for driving distances on symmetric roads, but not necessarily for one-way routes, which would define a quasi-metric. The triangle inequality is geometrically interpreted as the statement that the direct route between two points is never longer than any detour through a third point.
The triangle inequality has an important algorithmic consequence: it enables branch-and-bound pruning in data structures such as KD-trees and Ball-trees, which dramatically accelerate nearest-neighbor search in high-dimensional spaces. When a distance function does not satisfy the triangle inequality, these acceleration structures cannot be used reliably.

2.1.3 The Interplay between Data Type and Proximity

Data in real-world applications appear in many forms: numeric (continuous or ordinal), categorical (nominal), binary (a special case of categorical), text, graph-structured, and more. The choice of proximity measure must be matched to the data type. Using an inappropriate measure such as computing Euclidean distance between one-hot encoded categorical variables can produce misleading results because the numeric encoding of categories is arbitrary and does not reflect any inherent ordering or magnitude.

Data Type	Example	Recommended Proximity
Continuous numeric	Height, temperature, salary	Euclidean, Minkowski, Mahalanobis
Ordinal	Education level, star rating	Rank-based distances, Spearman
Nominal / Categorical	Blood type, country	Hamming, Overlap (value difference)

Binary	Symptom present/absent, yes/no	Jaccard, Simple Matching, Dice
Text / Document	News articles, reviews	Cosine similarity on TF-IDF vectors
Mixed types	Medical records	Gower's distance

Table 2.1: Recommended proximity measures by data type.

2.2 Distance Measures

This section provides a systematic treatment of the most important distance measures used in nearest-neighbor algorithms. Each measure is defined formally, its properties are discussed, and a concrete worked example is provided.

2.2.1 Euclidean Distance

Euclidean distance is the most familiar notion of distance, corresponding to the length of the straight-line path between two points in ordinary geometric space. For two d-dimensional vectors x = (x1, x2, ..., xd) and y = (y1, y2, ..., yd), the Euclidean distance is:

$$d_E(x, y) = \sqrt{(x_1 - y_1)^2 + (x_2 - y_2)^2 + \cdots + (x_d - y_d)^2}$$

Euclidean distance satisfies all four metric axioms, including the triangle inequality (which follows from the Cauchy-Schwarz inequality in linear algebra). It is the L2 norm of the difference vector x - y, written ||x - y||_2.

Worked Example 2.1 — Patient Similarity: Two hospital patients are characterized by two features: fasting blood glucose (mg/dL) and systolic blood pressure (mmHg). Patient A = (95, 120) and Patient B = (110, 135). The Euclidean distance is:

$$\begin{aligned} d_E(A, B) &= \sqrt{(95 - 110)^2 + (120 - 135)^2} \\ &= \sqrt{(-15)^2 + (-15)^2} \\ &= \sqrt{225 + 225} \\ &= \sqrt{450} \\ &\approx 21.21 \end{aligned}$$

This means that in the two-dimensional feature space, the two patients are approximately 21.21 units apart. If a third patient C = (95, 135), then d_E(A, C) = sqrt(0 + 225) = 15, indicating that Patient C is closer to Patient A than Patient B is.

Important Caveat — Scale Sensitivity: Euclidean distance is highly sensitive to the scale of features. If glucose were measured in micromol/L (roughly 5.56 times the mg/dL value), the glucose term would dominate the distance entirely. Therefore, features must be standardized (z-score normalization) or normalized (min-max scaling) before computing Euclidean distances.

2.2.2 Manhattan Distance

Manhattan distance (also called the L1 norm, city-block distance, or taxicab distance) measures the total absolute coordinate-wise difference between two points:

$$d_M(x,y) = |x_1 - y_1| + |x_2 - y_2| + \cdots + |x_d - y_d|$$

$$= \sum_{i=1}^{d} |x_i - y_i|$$

The name derives from the analogy of navigating a city grid, where one can only travel along horizontal and vertical streets. Manhattan distance is less sensitive to outliers than Euclidean

Distance because squaring amplifies large differences. It is preferred in applications where the features represent independent quantities on a grid or where robustness to outliers is desirable.

Worked Example 2.2 — Retail Store Navigation:

A customer at location (2, 3) in a grocery store grid wishes to reach the cheese counter at (7, 9). Assuming movement along aisles, the Manhattan distance is:

d_M = |2 - 7| + |3 - 9| = 5 + 6 = 11 aisle units

The Euclidean distance would be sqrt(25 + 36) = sqrt(61) approximately 7.81, but this represents a path through shelves and is physically impossible without passing through aisles.

2.2.3 Minkowski Distance - A Unified Framework

Euclidean and Manhattan distances are both special cases of the more general Minkowski distance of order p (also called the Lp norm):

d_p(x, y) = (|x1 - y1|^p + |x2 - y2|^p + ... + |xd – yd|^p)^(1/p)

= (sum over i from 1 to d of |xi - yi|^p)^(1/p)

Setting p = 1 yields Manhattan distance. Setting p = 2 yields Euclidean distance. As p approaches infinity, the Minkowski distance converges to the Chebyshev distance:

d_inf(x, y) = max over i of |xi - yi|

The Chebyshev distance gives the greatest single-coordinate difference and is relevant in chess (where a king can move diagonally), warehouse robotics (where a gantry moves simultaneously in X and Y), and scheduling problems.

Order p	Distance Name	Formula	Sensitive to Outliers?	Use Case
p= 1	Manhattan (L1)	sum |xi - yi|	Less sensitive	Sparse data, grid movement
p= 2	Euclidean (L2)	sqrt(sum (xi-yi)^2)	More sensitive	eometric distance, continuous features
p= 3	Cubic (L3)	(sum |xi-yi|^3)^(1/3)	High sensitivity	arely used directly
p-> inf	Chebyshev (Linf)	max |xi - yi|	Highest sensitivity	Chess moves, machine tool motion

Table 2.2: Minkowski distance family and their properties.

Worked Example 2.3 — Comparing Minkowski Orders:

Let x = (0, 0) and y = (3, 4). Then:

d_1(x, y) = |3 - 0| + |4 - 0| = 3 + 4 = 7 d_2(x, y) = sqrt(9 + 16) = sqrt(25) = 5

d_inf(x, y) = max(3, 4) = 4

Notice that L1 >= L2 >= Linf for any pair of points in any dimension. This can be proven algebraically using Jensen's inequality applied to the convex function f(t) = t^p.

2.2.4 Mahalanobis Distance

A fundamental limitation of Euclidean distance is that it treats all features as equally scaled and independent. In practice, features are often correlated. For

example, height and weight in human anthropometry data are positively correlated, and simply applying Euclidean distance in this space gives misleading proximities because the correlated space is stretched relative to the directions of variation.

Mahalanobis distance corrects for both scale differences and feature correlations by incorporating the covariance matrix S of the data:

d_Mah(x, y) = sqrt((x - y)^T * S^(-1) * (x - y))

Here, S is the d-by-d covariance matrix of the training features, S^(-1) is its inverse, and the superscript T denotes vector transposition. When S is the identity matrix (all features are independent and unit-variance), Mahalanobis distance reduces to Euclidean distance.

Geometric Interpretation: Mahalanobis distance measures how many standard deviations a point y lies from x, accounting for correlations. Points at equal Mahalanobis distance from a center form an ellipsoid whose axes are aligned with the principal components of the data. This makes Mahalanobis distance invariant to linear transformations of the feature space.

Worked Example 2.4 — Mahalanobis in Credit Scoring: Suppose the covariance matrix for two features (credit utilization %, annual income in thousands) is S = [[4, 2], [2, 9]]. The inverse S^(-1) = (1/32) * [[9, -2], [-2, 4]]. For customers A = (60, 50) and B = (65, 55), the difference vector is d = (5, 5). Then:

d_Mah^2 = d^T * S^(-1) * d

= (1/32) * [5, 5] * [[9, -2], [-2, 4]] * [5, 5]^T

= (1/32) * [5*9 + 5*(-2), 5*(-2) + 5*4] * [5, 5]^T

= (1/32) * [35, 10] * [5, 5]^T

= (1/32) * (175 + 50) = 225/32 = 7.03

d_Mah = sqrt(7.03) approximately 2.65

By contrast, the Euclidean distance would be sqrt(50) approximately 7.07. The Mahalanobis distance is smaller here because the two features are positively correlated — the direction of difference (5, 5) is the direction of greatest variance and is therefore not an unusual direction in this data.

2.2.5 Hamming Distance

Hamming distance is defined for strings or binary vectors of equal length. It counts the number of positions at which the corresponding symbols differ:

d_H(x, y) = Number of positions i where xi is not equal to yi

Hamming distance was originally developed in coding theory to measure the number of bit errors in a transmitted message. In machine learning, it applies to:

- Binary feature vectors (e.g., symptom present/absent records)
- Categorical feature vectors after encoding each feature as a single value
- Gene sequences in bioinformatics

Worked Example 2.5 — Error Detection: In a digital communication system, the transmitted codeword is x = '10110101' and the received word is y = '10010111'. Comparing bit by bit:

Position:	1	2	3	4	5	6	7	8
x:	1	0	1	1	0	1	0	1
y:	1	0	0	1	0	1	1	1
Differ?	N	N	Y	N	N	N	Y	N

The Hamming distance d_H(x, y) = 2, meaning two bits were corrupted in transmission.

Real-World Application: In a patient symptom survey with 8 binary symptoms (fever, cough, fatigue, shortness of breath, headache, sore throat, loss of smell, nausea), Patient A = (1, 1, 0, 0, 1, 1, 0, 0) and Patient B = (1, 0, 0, 1, 1, 1, 0, 0) differ in positions 2 and 4, so d_H = 2. They are classified as having similar symptom profiles.

2.3 Non-Metric Similarity Functions

Not all proximity measures satisfy the triangle inequality or even the other metric axioms. Those that do not are called non-metric similarity functions. While their lack of metric properties can complicate certain algorithmic optimizations, they are often far more appropriate and practically effective than metric distances for specific data typ

2.3.1 Cosine Similarity

Cosine similarity measures the cosine of the angle between two non-zero vectors. It is defined as:

cos_sim(x, y) = (x . y) / (||x||_2 * ||y||_2)

= (sum over i from 1 to d of xi * yi) /

(sqrt(sum xi^2) * sqrt(sum yi^2))

Cosine similarity ranges from -1 (perfectly opposite directions) to +1 (perfectly identical directions), with 0 indicating orthogonality. For document frequency vectors, which are always non-negative, the range is [0, 1].

Crucially, cosine similarity ignores the magnitude of the vectors. Two documents — one of 100 words and another of 1000 words — that use the same vocabulary in the same proportions will have cosine similarity = 1, even though their word-count vectors are very different in scale. This magnitude-invariance makes cosine similarity ideal for text retrieval.

Cosine similarity is NOT a metric. In particular, it does not satisfy the triangle inequality. Consider three unit vectors equally spaced at 60 degrees from each other: any two have cosine similarity 0.5, but one cannot travel from the first to the third through the second without violating the triangle inequality in the angular metric.

Worked Example 2.6 — Document Similarity in a Search Engine: Consider three documents, each represented as a TF-IDF vector over five terms: {machine, learning, neural, network, tree}.

Document	A	(ML textbook page): (3, 2,0,0,1)
Document	B	(Deep learning paper): (1,4,3,0)
Document	C	(Short ML summary): (6, 4,0,0,2)

Cosine similarity between A and C:

A . C = 3*6 + 2*4 + 0 + 0 + 1*2 = 18 + 8 + 2 = 28

||A|| = sqrt(9 + 4 + 0 + 0 + 1) = sqrt(14) approximately 3.742

||C|| = sqrt(36 + 16 + 0 + 0 + 4) = sqrt(56) approximately 7.483

cos_sim(A, C) = 28 / (3.742 * 7.483) = 28 / 27.99 approximately 1.000

The cosine similarity between A and C is essentially 1.0, correctly identifying that C is just a scaled- up version of A (same proportions, larger document). By contrast:

A . B = 3*1 + 2*1 + 0*4 + 0*3 + 1*0 = 5

||B|| = sqrt(1 + 1 + 16 + 9 + 0) = sqrt(27) approximately 5.196

cos_sim(A, B) = 5 / (3.742 * 5.196) = 5 / 19.44 approximately 0.257

The low similarity (0.257) correctly identifies that Document B (deep learning) is topically different from Document A (general ML).

2.3.2 Pearson Correlation Coefficient

Pearson correlation is a measure of the linear relationship between two variables. When used as a similarity measure between two feature vectors x and y (each treated as a sequence of observations), it is:

r(x, y) = (sum over i of (xi - x_bar)(yi - y_bar)) / sqrt(sum(xi - x_bar)^2 * sum(yi - y_bar)^2)

where x_bar and y_bar are the means of x and y respectively. Pearson correlation ranges from -1 (perfect negative linear relationship) to +1 (perfect positive linear relationship).

In recommender systems, Pearson correlation between two user rating profiles captures whether the users rate items similarly (accounting for individual rating biases). If User A consistently rates everything 1 point above the average and User B does the same, their cosine similarity may be low but their Pearson correlation will be high.

Worked Example 2.7 — Recommender System: Two users have rated four movies (out of 5 stars): User A = (5, 3, 4, 2) and User B = (4, 2, 3, 1). The mean ratings are: x_bar = 3.5, y_bar = 2.5. Centered vectors: A - mean = (1.5, -0.5, 0.5, -1.5) and B - mean = (1.5, -0.5, 0.5, -1.5). These are identical, so Pearson r = 1.0, confirming that User B consistently rates every film exactly one point lower than User A. They have the same taste profile, just different rating scales.

2.3.3 Tanimoto Coefficient

The Tanimoto coefficient (also called the Tanimoto similarity or extended Jaccard coefficient) generalizes the Jaccard index to real-valued vectors:

T(x, y) = (x . y) / (||x||^2 + ||y||^2 - x . y)

For binary vectors, this reduces to the Jaccard index (ratio of intersection to

union). The Tanimoto coefficient is widely used in cheminformatics to compare molecular fingerprints represented as binary or real-valued vectors, where values near 1 indicate structurally similar molecules.

2.4 Proximity between Binary Patterns

Binary data is ubiquitous in machine learning: medical symptom records, market basket analysis, user-item interactions, and genetic marker data are all naturally binary. Each data point is a vector of 0s and 1s, where 1 typically means "present" or "true" and 0 means "absent" or "false".

When computing proximity between two binary vectors x and y of length d, we construct a 2x2 contingency table that counts the co-occurrence of 0s and 1s across all features:

	y = 1	**y = 0**	**Row Total**
x = 1	a (both 1)	b (x=1, y=0)	a + b
x = 0	c (x=0, y=1)	d (both 0)	c + d
Column Total	a + c	b + d	d (total)

Table 2.3: 2x2 contingency table for binary proximity computation.

Here, a = number of features where both x and y are 1 (11 matches), b = features where x = 1 but y = 0, c = features where x = 0 but y = 1, and d = number of features where both are 0 (00 matches).

The total number of features is d = a + b + c + d. Different binary similarity measures weight these four quantities differently, depending on whether 00 matches (both features absent) should be considered evidence of similarity.

2.4.1 Simple Matching Coefficient (SMC)

The Simple Matching Coefficient treats 11 and 00 matches symmetrically, counting both as evidence of similarity:

$$\mathbf{SMC(x, y) = (a + d) / (a + b + c + d)}$$

SMC is appropriate when both the presence and absence of a feature are equally meaningful. For example, in a survey of political opinions with yes/no answers, agreeing that a policy is bad (both answer "no") is just as informative as agreeing it is good.

2.4.2 Jaccard Coefficient

The Jaccard coefficient ignores 00 matches entirely, focusing only on the features where at least one object has a 1:

$$J(x, y) = a / (a + b + c)$$

Jaccard similarity is appropriate when the absence of a feature (0) carries little meaning. In market basket analysis, if two customers both did not buy a luxury yacht (out of thousands of products), this negative match provides no evidence of customer similarity. Only co-purchases are meaningful. Similarly, in medical diagnosis, the absence of a symptom may occur for many unrelated reasons, whereas the co-presence of symptoms is diagnostically significant.

2.4.3 Dice Coefficient

The Dice coefficient (also called the Sorensen-Dice index or F1 similarity) gives double weight to co-occurrences:

$$Dice(x, y) = 2a / (2a + b + c)$$

The Dice coefficient can be understood as the harmonic mean of precision and recall (which is exactly the F1 score), making it the appropriate similarity measure when the focus is on jointly present features. The Dice coefficient is always greater than or equal to the Jaccard coefficient for the same data.

2.4.2 Russell-Rao Coefficient

The Russell-Rao coefficient counts only 11 co-occurrences and divides by the total number of features:

$$RR(x, y) = a / (a + b + c + d)$$

This coefficient penalizes patterns that have many features set to 1 in one vector but not the other, and also penalizes patterns with many 0s overall.

Measure	Formula	00 Matches Counted?	Best Used When...
Simple Matching	(a + d) / (a+b+c+d)	Yes	Presence and absence equally meaningful
Jaccard	a / (a+b+c)	No	Absence uninformative (market baskets, sparse data)
Dice	2a / (2a+b+c)	No	Co-occurrence doubly important (biological data)
Russell-Rao	a / (a+b+c+d)	No	Only strong co-presence matters

Hamming Similarity	1 - d_H/d	Yes	Coding theory, equal treatment of mismatches

Table 2.4: Binary similarity measures with formulas and applicability.

Worked Example 2.8 — Medical Symptom Records: Two patients are each characterized by 8 binary symptoms: {fever, cough, fatigue, dyspnea, headache, sore throat, anosmia, nausea}.

Patient A: (1, 1, 1, 0, 1, 0, 1, 0)

Patient B: (1, 0, 1, 1, 1, 0, 0, 0)

Computing the contingency table values:

- a (both 1): Fever, Fatigue, Headache => a = 3
- b (A=1, B=0): Cough, Anosmia => b = 2
- c (A=0, B=1): Dyspnea => c = 1
- d (both 0): Sore throat, Nausea => d = 2

Computing similarity measures:

SMC = (3 + 2) / 8 = 5/8 = 0.625

Jaccard = 3 / (3 + 2 + 1) = 3/6 = 0.500

Dice = 2*3 / (2*3 + 2 + 1) = 6/9 = 0.667

Russell-Rao = 3 / 8 = 0.375

The Dice coefficient gives the highest similarity (0.667) because it emphasizes the three shared symptoms. The Jaccard coefficient (0.500) is lower because it penalizes the disagreements. The Russell-Rao coefficient (0.375) is lowest because it also counts the total features. In a clinical setting, the Jaccard or Dice coefficient would be preferred because shared-absent symptoms (both patients lack nausea and sore throat) may not indicate similar disease profiles.

2.5 Different Classification Algorithms Based on Distance Measures

The notion of proximity can be harnessed for classification in multiple ways. In this section, we survey the spectrum of distance-based classifiers, providing the conceptual foundation before presenting detailed algorithms in subsequent sections.

2.5.1 Instance-Based Learning: Foundational Concepts

Instance-based learning (IBL), also called memory-based learning or lazy learning, is a paradigm where the model stores the training instances directly and defers generalization until a query is presented. This stands in sharp contrast to eager learners such as decision trees and neural networks, which generalize during training and discard the training data afterward.

Lazy vs. Eager Learning: The distinction between lazy and eager learning has important practical consequences. Eager learners pay a high computational cost during training but are fast at prediction time. Lazy learners pay negligible cost during training (essentially just storing data) but may require significant computation at prediction time (scanning all stored examples). Lazy learners also adapt naturally to local variations in the target function, since they do not commit to a single global model.

Property	Lazy Learning (KNN)	Eager Learning (Decision Tree, SVM)
Training cost	Very low (just store data)	High (model fitting)
Prediction cost	High (search all examples)	Low (traverse model)
Model storage	All training data	Compact model parameters
Adaptation to local structure	Excellent	Limited by model assumptions
Handling concept drift	Natural (add new examples)	Requires retraining
Interpretability	Individual-case level	Global model level

Table 2.5: Comparison of lazy and eager learning paradigms.

2.5.2 The 1-Nearest Neighbor Classifier

A classifier is a function or algorithm that maps an input observation (represented as a feature vector) to one of a finite set of discrete class labels. Unlike a decision rule, which specifies a procedure, a classifier is an instantiated computational model that performs the mapping from feature space to label space.

The 1-Nearest Neighbor (1-NN) classifier is the simplest instance-based classifier. It defines a decision boundary implicitly through the training data stored in memory. Given a query point x*, the classifier finds the single training example x_nn closest to x* under a chosen distance function d(.,.), and assigns x* the same class label as x_nn.

Feature space concept: The feature space is a d-dimensional vector space where each axis represents one feature. Each training example is a point in this space. The 1-NN classifier partitions the space into Voronoi cells such that all points in a cell are closer to a particular training example than to any other. The classification boundary is the union of the cell boundaries.

Distance Computation

The most commonly used distance function is Euclidean distance, which measures the straightline distance between two points in the feature space. For two *d*-dimensional vectors $\mathbf{x} = (x_1,x_2,...,x_d)$ and $\mathbf{y} = (y_1,y_2,...,y_d)$, the Euclidean distance is:

$$d_E(\mathbf{x}, \mathbf{y}) = \sqrt{\sum_{i=1}^{d} (x_i - y_i)^2}$$

Classification Decision

The 1-NN classifier assigns the label as follows:

$$\text{class}(\mathbf{x}*) = \text{class}(\mathbf{x} \in \text{TrainingSet}^{d(\mathbf{x}^*, \mathbf{x}))}$$

In words: find the training point **x** that minimizes the distance to the query $\mathbf{x}^*$, and assign $\mathbf{x}^*$ the class label of that nearest neighbor.

Properties of the 1-NN Classifier

The 1-NN classifier has remarkable theoretical guarantees. Cover and Hart (1967) proved that as the training set size *n* grows to infinity, the error rate of the 1-NN classifier is at most twice the Bayes error rate (the error of the theoretically optimal classifier with complete knowledge of the data

distribution). Formally, if e^* is the Bayes error rate, the asymptotic 1-NN error rate satisfies:

$$e_{1NN} \leq 2 \cdot e^* \cdot (1 - e^*)$$

Since $2 \cdot e^* \cdot (1 - e^*) \leq 2 \cdot e^*$, the 1-NN error rate is at most twice the optimal. This result is astonishing given the simplicity of the classifier: with enough training data, simply finding the nearest neighbor and copying its label gives nearly optimal performance.

2.5.3 The 1-Nearest Neighbor Rule

While the 1-NN classifier is a computational model that embodies the classification function, the 1-NN rule refers to the algorithmic procedure and decision-making protocol that underlies the classifier. The rule specifies exactly how a classification decision is made given a query point and a labeled training set.

Definition

The 1-Nearest Neighbor rule is a decision procedure that states: *Given a query instance* $\mathbf{x}^*$*, compute its distance to all training instances, identify the single nearest instance, and assign* $\mathbf{x}^*$ *the label of that instance.*

Step-by-Step Working of the 1-NN Rule

Step 1: Compute distance to all training samples

For each training example $(\mathbf{x}_i, y_i)$ in the training set (where $\mathbf{x}_i$ is the feature vector and y_i is the class label), compute the distance $d(\mathbf{x}^*, \mathbf{x}_i)$ using a chosen distance metric.

Step 2: Find the nearest neighbor

Identify the training example $\mathbf{x}_{nn}$ such that:

$$\mathbf{x}nn = \mathbf{x}_i \in \text{TrainingSet}\ d(\mathbf{x}^*, \mathbf{x}_i)$$

This is the training point that has the smallest distance to the query.

Step 3: Assign the class label

Set the predicted class of $\mathbf{x}^*$ to be the label of $\mathbf{x}_{nn}$:

$$\hat{y}^* = \text{class}(\mathbf{x}_{nn})$$

Distance Formula (Euclidean)

When using Euclidean distance, the formula for the distance between query

$\mathbf{x}^*$ and training point $\mathbf{x}_i$ is:

$$d(\mathbf{x}^*, \mathbf{x}_i) = \sqrt{\sum_{j=1}^{d} (x_j^* - x_{ij})^2}$$

where d is the number of features, x_j^* is the j-th feature of the query, and x_{ij} is the j-th feature of the i-th training sample.

Small Worked Example

Consider a binary classification problem with 4 training points in 2 dimensions:

Training Point	x_1	x_2	Class
A	1.0	2.0	Red
B	2.0	3.0	Red
C	5.0	5.0	Blue
D	6.0	4.0	Blue

Table 1: Training data for 1-NN example

Query: $\mathbf{x}^*$ = (3.0,3.5)
Step 1 — Compute distances:

Step 2 — Find nearest neighbor: The smallest distance is 1.12, corresponding to training point B.

$$d(\mathbf{x}^*, A) = \sqrt{(3.0-1.0)^2 + (3.5-2.0)^2} = \sqrt{4 + 2.25} = \sqrt{6.25} \approx 2.50$$
$$d(\mathbf{x}^*, B) = \sqrt{(3.0-2.0)^2 + (3.5-3.0)^2} = \sqrt{1 + 0.25} = \sqrt{1.25} \approx 1.12$$
$$d(\mathbf{x}^*, C) = \sqrt{(3.0-5.0)^2 + (3.5-5.0)^2} = \sqrt{4 + 2.25} = \sqrt{6.25} \approx 2.50$$
$$d(\mathbf{x}^*, D) = \sqrt{(3.0-6.0)^2 + (3.5-4.0)^2} = \sqrt{9 + 0.25} = \sqrt{9.25} \approx 3.04$$

Step 3 — Assign label: Since B has class label Red, the 1-NN rule assigns $\mathbf{x}^*$ the label Red.
Prediction: The query point $\mathbf{x}^*$ = (3.0,3.5) is classified as Red.

Important Distinction — Classifier vs. Rule

- The 1-NN classifier is the instantiated model (function) that performs the mapping from feature space to labels.
- The 1-NN rule is the decision-making protocol (algorithm) that defines how distances are computed, how the nearest neighbor is identified, and how the label is assigned.

In practice, "1-NN classifier" and "1-NN rule" are often used interchangeably, but conceptually the classifier is the *what* (the model) and the rule is the *how* (the procedure).

2.5.4 Distance-Weighted Nearest Neighbors

A natural extension of the basic KNN rule weights each neighbor's vote by its proximity to the query point. Closer neighbors receive greater weight. The most common weighting scheme is the inverse distance weighting:

$$w_i = 1 / d(x^*, x_i) \quad \text{for each neighbor } x_i$$

If the query point coincides exactly with a training example (d = 0), a special case arises. One solution is to assign w = infinity to the coincident neighbor and 0 to all others, effectively making the coincident neighbor the only voter — which is equivalent to the 1-NN rule.

Kernel weighting provides a more sophisticated way to assign influence to neighbors by using a kernel function K(d) that decreases with distance. Instead of simple inverse-distance weights, each neighbor contributes according to K(d(x*, xi)). Popular choices include the Gaussian kernel $K(d) = \exp(-d^2 / 2\sigma^2)$, where σ controls the spread of the weights, and the Epanechnikov kernel $K(d) = 1 - (d / h)^2$ for d ≤ h and K(d) = 0 otherwise, where h is a bandwidth parameter. These kernels assign higher weight to closer neighbors and progressively reduce the influence of points farther away, with the Epanechnikov kernel enforcing a hard cutoff at distance h.

2.6 K-Nearest Neighbor Classifier

2.6.1 Algorithm Description

The K-Nearest Neighbor (KNN) classifier is the workhorse of instance-based learning. Given a training set T = {(x_1, y_1), (x_2, y_2), ..., (x_n, y_n)} where x_i is a d-dimensional feature vector and y_i is a class label, and given a positive integer K and a distance function d, the KNN classifier operates as follows:

Algorithm 2.1 — KNN Classification

Input: Training set T, distance function d(.,.), number of neighbors K, query point x*. Output: Predicted class label y* for x*.

Step 1: For each training point x_i in T, compute the distance d(x*, x_i).

Step 2: Sort all training points in ascending order of their distance to x*.

Step 3: Select the K training points with the smallest distances. Call this set N_K(x*).

Step 4: For each class label c, count the votes: count(c) = number of points in N_K(x*) with label c.

Step 5: Assign x* the class with the maximum vote count:y* = argmax over c of count(c).

Step 6: If there is a tie (two or more classes have the same vote count), break ties using one of:

(a) Choose the class of the nearest neighbor among the tied classes.

(b) Use distance-weighted voting to break the tie.(c) Choose the class with the highest prior probability.

2.6.2 The Role of K

The value of K is the most critical hyper parameter of the KNN classifier. Its choice involves a fundamental bias-variance trade-off:

Small K (e.g., K = 1): The decision boundary closely follows the training data, resulting in a complex, highly irregular boundary. This leads to low training error but potentially high generalization error — a manifestation of variance. The classifier is sensitive to noisy or mislabeled training examples.

Large K (e.g., K = n, all training points): Every query point receives the same label — the majority class in the entire training set. The decision boundary is trivially simple. This is high bias: the classifier under fits the data, ignoring all local structure.

The optimal K lies between these extremes and is chosen via cross-validation. A practical rule of thumb is to start with K = sqrt(n) as an initial estimate, then tune using 5-fold or 10-fold cross-validation.

K should always be chosen as an odd number when there are two classes, to avoid tie-breaking ambiguity. For multiclass problems, K should not be a multiple of the number of classes for the same reason.

Worked Example 2.9 — Effect of K on Decision Boundaries: Consider a binary classification problem with 15 training points in two dimensions: 8 labeled Class A (open circles) and 7 labeled Class B (filled circles). A new query point x* = (3, 4) is to be classified. The five nearest training points, in increasing order of distance, are:

Rank	Training Point	Distance to x*	Class
1 (nearest)	(3.1, 4.2)	0 .224	Class B
2	(2.8, 3.7)	0.361	Class A
3	(3.3, 4.5)	0. 583	Class A
4	(2.5, 4.1)	0.510	Class A
5	(3.6, 3.8)	0.632	Class B

Table 2.6: Five nearest neighbors of the query point x = (3, 4).*votes

K = 1: Nearest neighbor is Class B => Prediction: Class B

K = 3: Neighbors at ranks 1, 2, 3 => B, A, A => Class A wins 2-1 => Prediction: Class A

K = 5: All 5 neighbors => B, A, A, A, B => Class A wins 3-2
=> Prediction: Class A

Notice that K = 1 gives a different (and arguably noisier) prediction because the single nearest neighbor happens to be a Class B point in what is predominantly a Class A region. K = 3 or K = 5 produces a more robust prediction by aggregating multiple neighbors'

2.6.3 Feature Scaling and Weighting

KNN is extremely sensitive to the scale of features. A feature with a large numeric range (e.g., annual income from 0 to 500,000 dollars) will dominate the distance computation compared to a feature with a small range (e.g., age from 0 to 100). This scaling problem must be addressed before applying KNN.

The two standard approaches are:

- **Z-score standardization**: Replace each feature value xi with (xi - mu_i) / sigma_i, where mu_i and sigma_i are the training set mean and standard deviation of feature i. After transformation, all features have mean 0 and standard deviation 1.
- **Min-max normalization**: Replace each feature value with (xi - min_i) /

(max_i - min_i), mapping all values to the range [0, 1]. This is sensitive to outliers that define extreme min or max values.

Beyond equal weighting, it may be beneficial to weight features differently based on their relevance to the classification task. Feature weighting in KNN is equivalent to scaling the axes of the feature space. Methods for learning feature weights include filter methods (weight features by their mutual information with the class label), wrapper methods (select K and feature weights Simultaneously to minimize cross-validation error), and embedded methods (use the training process itself to determine weights).

2.6.4 KNN Computational Complexity

The naive implementation of KNN requires computing the distance from the query to every training point. With n training examples and d features, each distance computation takes O(d) operations, so predicting the label for a single query takes O(n * d) time. For large datasets (e.g., n = 10^6, d = 10^3), this is computationally prohibitive.

Acceleration Structures: Two data structures are commonly used to accelerate KNN search: KD-trees and Ball-trees. A KD-tree is a binary space-partitioning tree where each node splits the data along a coordinate axis at the median value of that axis. Ball-trees partition the data into nested hyperspheres (balls). Both structures enable branch-and-bound pruning: when searching for the K nearest neighbors, entire subtrees can be excluded from consideration if the distance to the bounding region exceeds the current K-th nearest neighbor distance. In low dimensions (d < 20), KD-trees reduce average search time to O (log n). However, as dimension increases, the benefit diminishes — this is one manifestation of the curse of dimensionality, where all points become approximately equidistant from a query point in high-dimensional spaces, making pruning ineffective.

2.6.5 The Curse of Dimensionality

As the number of features d increases, the behavior of distance measures changes fundamentally. In high-dimensional spaces, several problematic phenomena emerge:

- **Distance concentration:** The ratio of the maximum to minimum distance from a query point to its neighbor's approaches 1 as d grows. This means all points are approximately equidistant, making the concept of "nearest neighbor" essentially meaningless.
- **Sparsity:** The volume of the data space grows exponentially with d. A fixed number of training examples becomes exponentially sparse, leaving most of

the feature space empty. The nearest neighbor of a query point may be very far away.

- **Boundary dominance:** In d dimensions, most of the volume of a hypercube is near the boundary. A sphere inscribed in a unit hypercube occupies only (pi/4)^(d/2) / Gamma(d/2+ 1) of the hypercube's volume, which goes to 0 as d increases.

These phenomena collectively degrade KNN performance in high dimensions. Practical remedies include dimensionality reduction (PCA, t-SNE, autoencoders), feature selection, and using distance measures specifically designed for high-dimensional spaces.

Worked Example 2.10 — Sparsity in High Dimensions: Suppose we have n = 1000 training points uniformly distributed in a d-dimensional unit hypercube, and we want the K = 10 nearest neighbors of a query point to lie within a neighborhood of radius r. The fraction of the hypercube's volume within a ball of radius r is approximately proportional to r^d. For K/n = 10/1000 = 0.01, we need r^d = 0.01, so r = 0.01^(1/d). For d = 2, r = 0.1; for d = 10, r = 0.631; for d = 100, r = 0.955. In 100 dimensions, we need a neighborhood that covers 95.5% of each dimension's range just to find 10 nearest neighbors — the neighborhood is no longer local at all.

2.7 Radius Distance nearest Neighbor Algorithm

The K-Nearest Neighbor algorithm fixes the number of neighbors K and allows the neighborhood radius to vary with the density of training points. An alternative approach fixes the neighborhood radius r and allows the number of neighbors to vary. This is called the Radius Nearest Neighbor (RNN) classifier, also known as the ball classifier or fixed-radius classifier.

2.7.1 Algorithm Description

Algorithm 2.2 — Radius Nearest Neighbor Classification

Input: Training set T, distance function d(.,.), radius r, minimum neighbor count K_min (optional), query point x*. **Output:** Predicted class label y* for x*, or 'UNKNOWN' if the neighborhood is empty.
Step 1. Define the neighborhood ball: N_r(x*) = { x_i in T : d(x*, x_i) <= r }.
Step 2. Compute d(x*, x_i) for all i and collect those within N_r(x*).
Step 3. If N_r(x*) is empty: Option (a): Return UNKNOWN (refuse to classify).Option (b): Fall back to KNN with a fixed K. Option (c): Expand r until K_min neighbors are found (adaptive radius).

Step 4. If N_r(x*) is non-empty: For each class c, compute count(c) = number of points in N_r(x*) with label c. Assign y* = argmax over c of count(c).
Step5. (Optional) Apply distance weighting: weight each neighbor by w_i = 1/d(x*, x_i).

2.7.2 Advantages and Disadvantages

The radius nearest neighbor approach has several distinct advantages over fixed-K nearest neighbor:

- In dense regions of the feature space, more training examples contribute to the classification decision, providing a reliable majority vote. In sparse regions, fewer examples contribute, and the algorithm may appropriately abstain.
- The algorithm is more interpretable in terms of the distance metric: a point is classified using all training examples within a meaningful physical or semantic distance r.
- When the query falls in a region of the feature space with no training data, the algorithm can identify this and return an "unknown" response rather than forcing a potentially unreliable prediction.

The disadvantages include:

- Choosing r requires domain knowledge or cross-validation. Unlike K, which is a dimensionless count, r depends on the scale of the feature space.
- The number of neighbors can vary greatly across the feature space, leading to inconsistent confidence in predictions.
- If the data is highly imbalanced in density across regions, a fixed r may give very large neighborhoods in some regions and empty neighborhoods in others.

2.7.3 Adaptive Radius and Density-Sensitive Neighborhoods

A more sophisticated variant combines the radius and count approaches: the adaptive radius nearest neighbor algorithm expands the neighborhood of x* until it contains exactly K training points, but restricts the expansion to a maximum radius r_max. If the K-th nearest neighbor lies beyond r_max, only those within r_max are used. This provides a smooth interpolation between pure KNN and pure RNN behavior.

Worked Example 2.11 — Intrusion Detection System: Consider a network intrusion detection system where each network connection is described by two features: packet_size (in bytes, range 64-65535) and connection duration (in seconds, range 0.001-3600). After normalization, the training set contains 500 labeled connections: 450 "normal" and 50 "attack".

A new connection arrives with normalized features x* = (0.3, 0.4). Setting r = 0.15 in the normalized space:

Neighbors within r = 0.15: 12 normal connections, 2 attack connections.

Count (normal) = 12, Count (attack) = 2.

Prediction: normal (12 > 2).

Now suppose x* = (0.9, 0.1) — a very large packet with very short duration, which is unusual:

Neighbors within r = 0.15: 0 connections found.

Decision: Return UNKNOWN. Flag connection for manual inspection.

The ability to say "I don't know" is a critical safety property in high-stakes applications like cybersecurity. The RNN algorithm naturally provides this capability, whereas KNN always forces a prediction.

2.8 KNN Regression

The nearest-neighbor paradigm extends naturally from classification to regression. In KNN regression, the target variable y is continuous (real-valued) rather than categorical. The prediction for a query point x* is based on the output values of its K nearest neighbors.

2.8.1 Unweighted KNN Regression

The simplest form of KNN regression predicts the output of a query point as the arithmetic mean of the outputs of its K nearest neighbors:

$$\hat{y}(x^*) = \frac{1}{K}\sum_{i=1}^{K} y_i$$

where y_1, y_2, ..., y_K are the target values of the K nearest training points (ordered by increasing distance from x*).

Algorithm 2.3 — KNN Regression
Input: Training set {(x1, y1), …, (xn, yn)}, distance function d, K, query point x*.
Output: Predicted real value ŷ for x*.
Step 1. For each training point xi, compute d(x*, xi).
Step 2. Sort training points in ascending order of distance to x*.
Step 3. Select the K nearest neighbors: NK(x*) = {x1, x2, …, xK} (by sorted order).
Step 4. Compute the unweighted mean: ŷ(x*) = (1 / K) Σ yi
i ∈ NK(x*)
Step 5. Return ŷ.

2.8.2 Distance- Weighted KNN Regression

A more accurate form of KNN regression weights each neighbor's contribution inversely proportional to its distance from the query point. Closer neighbors have more influence on the prediction:

$$\hat{y}(x^*) = \frac{\sum_{i=1}^{K} w_i \, y_i}{\sum_{i=1}^{K} w_i}$$

Where w_i = 1 / d(x*, x_i)^2 (inverse squared distance weighting) or more generally w_i = K(d(x*, x_i)) for some kernel function K.

When the query point coincides exactly with a training example (d = 0), the weight would be infinite. This degenerate case is handled by setting the prediction equal to the training label of the coincident point.

2.8.3 Kernel Regression Interpretation

Distance-weighted KNN regression with a smooth kernel function is a form of Nadaraya-Watson kernel regression. The general formula is:

$$\hat{y}(x^*) = \frac{\sum_{i=1}^{n} K_h(x^*, x_i) \, y_i}{\sum_{i=1}^{n} K_h(x^*, x_i)}$$

where K_h(x*, x_i) is a kernel function with bandwidth h. When K_h is an indicator function (1 if distance < h, else 0), this reduces to unweighted radius-based regression. When K_h is a Gaussian, the prediction smoothly interpolates among all training points, with influence decaying exponentially with distance.

Worked Example 2.12 — House Price Prediction: A real estate company wishes to predict the sale price of a house at a new location using KNN regression. The training set contains 6 recently sold houses with the following features (after normalization): house size (m^2) and distance to city center (km), along with sale prices (in 100,000 USD).

House ID	Size (normalized)	Dist. to Center (normalized)	Sale Price (100K USD)
H1	0.50	0.30	3.2
H2	0.60	0.25	3.8
H3	0.45	0.60	2.5
H4	0.70	0.20	4.5
H5	0.40	0 .70	2.2
H6	0.65	0.40	3.5

Table 2.7: Training data for house price prediction using KNN regression.

Query house x* = (0.55, 0.28). Computing Euclidean distances from x* to each training house:

$$d(x^*, H_1) = \sqrt{(0.55-0.50)^2 + (0.28-0.30)^2} = \sqrt{0.0025+0.0004} = \sqrt{0.0029} = 0.0539$$

$$d(x^*, H_2) = \sqrt{(0.55-0.60)^2 + (0.28-0.25)^2} = \sqrt{0.0025+0.0009} = \sqrt{0.0034} = 0.0583$$

$$d(x^*, H_3) = \sqrt{(0.55-0.45)^2 + (0.28-0.60)^2} = \sqrt{0.01+0.1024} = \sqrt{0.1124} = 0.3353$$

$$d(x^*, H_4) = \sqrt{(0.55-0.70)^2 + (0.28-0.20)^2} = \sqrt{0.0225+0.0064} = \sqrt{0.0289} = 0.1700$$

$$d(x^*, H_5) = \sqrt{(0.55-0.40)^2 + (0.28-0.70)^2} = \sqrt{0.0225+0.1764} = \sqrt{0.1989} = 0.4460$$

$$d(x^*, H_6) = \sqrt{(0.55-0.65)^2 + (0.28-0.40)^2} = \sqrt{0.01+0.0144} = \sqrt{0.0244} = 0.1562$$

Ranking by distance: H1 (0.0539), H2 (0.0583), H6 (0.1562), H4 (0.1700), H3 (0.3353), H5 (0.4460).

For K = 3 (nearest neighbors: H1, H2, H6):

Un weighted prediction = (3.2 + 3.8 + 3.5) / 3 = 10.5 / 3 = 3.5 (i.e., 350,000 USD)

For distance-weighted prediction (weights = 1/d^2):

w_H1 = 1/0.0539^2 = 1/0.00291 = 344.0 w_H2 = 1/0.0583^2 = 1/0.00340 = 294.1 w_H6 = 1/0.1562^2 = 1/0.02440 = 41.0

Sum of weights = 344.0 + 294.1 + 41.0 = 679.1

Weighted prediction = (344.0*3.2 + 294.1*3.8 + 41.0*3.5) / 679.1

= (1100.8 +1117.6+143.5)/679.1

= 2361.9 / 679.1

= 3.478 (i.e., approximately 347,800 USD)

The distance-weighted prediction (347,800 USD) places slightly more emphasis on the two closest houses (H1 and H2), giving a subtly different and arguably more accurate estimate than the simple mean.

2.8.4 Choosing K for Regression

The bias-variance trade-off in regression is analogous to that in classification. Small K produces a regression function that follows the training data closely (high variance, low bias). Large K averages over many neighbors, producing a smooth function (low variance, high bias). The optimal K is chosen to minimize the mean squared error on a validation set, typically via cross-validation.

For regression, an additional metric useful for model selection is the root mean squared error (RMSE): sqrt((1/n_val) * sum over validation examples of (y_true - y_hat)^2). Plotting RMSE against K and choosing the elbow of the curve provides a principled method for K selection.

2.9 Performance of Classifiers

Evaluating the performance of a classifier is as important as building it. A classifier is of little practical `value if we cannot assess its reliability, correctness, and generalization ability. This section provides a comprehensive treatment of classifier evaluation metrics, starting from the confusion matrix and progressing to ROC analysis.

2.9.1 The Confusion Matrix

For a binary classifier (two classes: Positive and Negative), all possible outcomes of classification can be organized into a 2x2 matrix called the confusion matrix. Each cell represents a count of instances:

	Predicted Positive	**Predicted Negative**
Actual Positive	True Positive (TP)	False Negative (FN)
Actual Negative	False Positive (FP)	True Negative (TN)

Table 2.8: Binary confusion matrix structure.

The four cells are defined as follows:

- True Positive (TP): The classifier correctly predicts Positive for an actually Positive instance. (Hit)
- False Negative (FN): The classifier incorrectly predicts Negative for an actually Positive instance. (Miss, Type II Error)
- False Positive (FP): The classifier incorrectly predicts Positive for an actually Negative instance. (False Alarm, Type I Error)
- True Negative (TN): The classifier correctly predicts Negative for an actually Negative instance. (Correct Rejection)

2.9.2 Derived Performance Metrics

From the four quantities TP, FP, TN, FN, a rich set of derived metrics can be computed. Each metric emphasizes a different aspect of classifier performance and is appropriate for different application contexts.

Metric	Formula	Interpretation	Range
Accuracy	(TP + TN) / (TP + FP + TN + FN)	Overall fraction correct	[0, 1]
Error Rate	(FP + FN) / (TP + FP + TN + FN)	Overall fraction incorrect	[0, 1]
Precision (PPV)	TP / (TP + FP)	Of predicted positives, fraction truly positive	[0, 1]
Recall(Sensitivity, TPR)	TP / (TP + FN)	Of actual positives, fraction detected	[0, 1]
Specificity (TNR)	TN / (TN + FP)	Of actual negatives, fraction correctly rejected	[0, 1]
F1 Score	2 * Precision * Recall / (Precision + Recall)	Harmonic mean of precision and recall	[0, 1]

F-beta Score	(1+b^2) * P * R / (b^2*P + R)	Weighted F measure	[0, 1]
MCC	(TP*TN - FP*FN) / sqrt(...)	Matthews Correlation Coefficient; robust to imbalance	[-1, 1]
Balanced Accuracy	(TPR + TNR) / 2	Mean of sensitivity and specificity	[0, 1]

Table 2.9: Classifier performance metrics derived from the confusion matrix.

Worked Example 2.13 — COVID-19 Screening Test: A KNN-based COVID-19 screening classifier is applied to 200 patients. 80 patients are truly COVID-positive, and 120 are truly COVID-negative. The confusion matrix is:

TP = 72 (correctly identified COVID-positive)

FN = 8 (COVID-positive patients missed)

FP = 10 (COVID-negative patients falsely alarmed)

TN = 110 (correctly identified COVID-negative)

Computing all metrics:

Accuracy = (72 + 110) / 200 = 182/200 = 0.910 (91.0%)

Error Rate = (10 + 8) / 200 = 18/200 = 0.090 (9.0%)

Precision = 72 / (72 + 10) = 72/82 = 0.878 (87.8%)

Recall = 72 / (72 + 8) = 72/80 = 0.900 (90.0%)

Specificity = 110 / (110 + 10) = 110/120 = 0.917 (91.7%)

F1 Score = 2 * 0.878 * 0.900 / (0.878 + 0.900)

= 1.580 / 1.778 = 0.889

In a COVID screening context, recall (sensitivity) is more critical than precision: missing a positive patient (FN) is more dangerous than falsely alarming a negative one (FP). The classifier's recall of 90% means it detects 90 out of every 100 COVID-positive patients a clinically significant rate. Specificity of 91.7% means it correctly clears 91.7% of truly negative patients. This balance between sensitivity and specificity must be chosen based on the clinical consequence of each type of error.

In a COVID screening context, recall (sensitivity) is more critical than precision:

missing a positive patient (FN) is more dangerous than falsely alarming a negative one (FP). The classifier's recall of 90% means it detects 90 out of every 100 COVID-positive patients a clinically significant rate. Specificity of 91.7% means it correctly clears 91.7% of truly negative patients. This balance between sensitivity and specificity must be chosen based on the clinical consequence of each type of error.

2.9.3 Multiclass Confusion Matrix

When there are more than two classes (C1, C2, ..., Ck), the confusion matrix generalizes to a k x k matrix M where entry M[i][j] is the number of instances of class i that were predicted as class j. The diagonal entries M[i][i] count correct classifications for class i.

Worked Example 2.14 — Handwritten Digit Recognition: A KNN classifier distinguishes three types of skin conditions: Eczema (E), Psoriasis (P), and Rosacea (R). The 3x3 confusion matrix is:

Actual \ Predicted	red. Eczema	red. Psoriasis	red. Rosacea
Actual Eczema	45	3	2
Actual Psoriasis	4	38	8
Actual Rosacea	1	5	44

Table 2.10: Multiclass confusion matrix for skin condition classification.

From this matrix: Total instances = 150. Correct = 45 + 38 + 44 = 127. Overall accuracy = 127/150 = 84.7%. The most common misclassification is Psoriasis being predicted as Rosacea (8 cases) likely because both conditions involve facial redness, suggesting these two classes overlap in the feature space.

Per-class metrics can be computed by treating each class as a one-vs-rest binary problem. For Eczema: TP = 45, FN = 5, FP = 5, TN = 95. Precision = 45/50 = 0.90, Recall = 45/50 = 0.90, F1 = 0.90.

2.9.4 ROC Curve and AUC

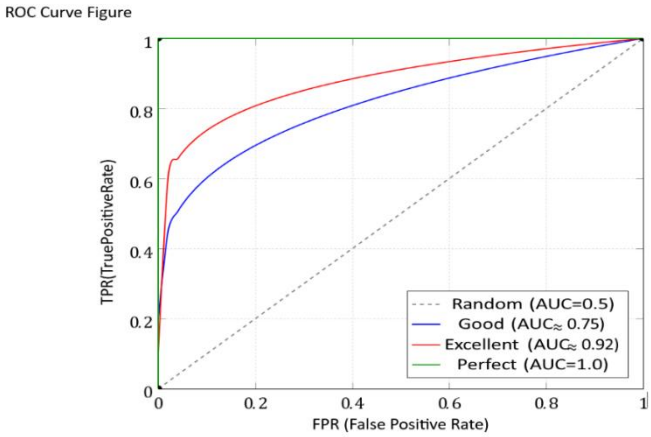
The Receiver Operating Characteristic (ROC) curve is a graphical tool that evaluates a binary classifier's performance across all possible classification thresholds. For a probabilistic classifier that outputs a score s(x) in [0, 1], we classify x as Positive if s(x) >= tau for some threshold tau. By varying tau from 0 to 1, we sweep through all possible (FPR, TPR) pairs, where:

False Positive Rate (FPR) = FP / (FP + TN) = 1 - Specificity

True Positive Rate (TPR) = TP / (TP + FN) = Recall = Sensitivity

The ROC curve plots TPR (y-axis) against FPR (x-axis). Key reference points:

- (0, 0): threshold tau = 1 — classify everything as Negative
- (1, 1): threshold tau = 0 — classify everything as Positive.
- (0, 1): perfect classifier — detects all positives with no false alarms.
- The diagonal line from (0,0) to (1,1) represents a random classifier.



The Area Under the ROC Curve (AUC) summarizes the ROC curve in a single scalar. AUC has a probabilistic interpretation: it equals the probability that the classifier assigns a higher score to a randomly selected Positive instance than to a randomly selected Negative instance. AUC = 1.0 means perfect discrimination; AUC = 0.5 means random chance.

AUC Range	Interpretation	Example Application Quality
0.5 - 0.6	Fail near random	Classifier provides no useful information
0.6 - 0.7	Poor	Some discriminative power but unreliable
0.7 - 0.8	Fair	Acceptable for low-stakes decisions
0.8 - 0.9	Good	Appropriate for many real-world applications
0.9 - 1.0	Excellent	High-quality classifier; suitable for high-stakes use

Table 2.11: AUC interpretation guidelines.

Worked Example 2.15 — Loan Default Prediction ROC: A KNN-based loan default classifier produces the following scores for 10 test applicants (1 = actual default, 0 = no default): Applicant scores sorted descending and their true labels:

Threshold tau	Predicted Pos.	TP	FP	FN	FNTN	TPR	FPR
0.95	1	1	0	4	5	0.20	0.00
0.80	2	2	0	3	5	0.40	0.00
0.70	4	3	1	2	4	0.60	0.20
0.60	6	4	2	1	3	0.80	0.40
0.50	8	5	3	0	2	1.00	0.60
0.30	10	5	5	0	0	1.00	1.00

Table 2.12: ROC operating points for the loan default classifier. The ROC curve passes through the points (0.00, 0.20), (0.00, 0.40), (0.20, 0.60), (0.40, 0.80), (0.60, 1.00), (1.00, 1.00).

The AUC can be estimated using the trapezoidal rule: AUC approximately 0.84, indicating a good classifier. In the loan context, the institution would typically choose a threshold that minimizes the expected cost of defaults versus unnecessary rejections.

2.9.5 Precision-Recall Curve and Average Precision

Interpretation of the PR curve:

- Top-left region: high precision, low recall (very selective).
- Top-right region: high precision, high recall (ideal).
- Bottom-right region: low precision, high recall (overly permissive).
- Baseline: for p% positives, a random classifier achieves precision ~= p%.

Worked example - Fraud detection:

Average Precision (AP) summarizes the trade-off between precision and recall; higher AP indicates better ranking of positive instances.

Threshold	TP	FP	FN	TN	Precision	Recall
0.9	20	5	30	9945	0.800	0.400
0.7	35	50	15	9900	0.412	0.700
0.5	45	200	5	9750	0.184	0.900

When the positive class is rare (class imbalance), the ROC curve can be overly optimistic because the large number of True Negatives inflates the TN value and makes the FPR appear small. In such cases, the Precision-Recall (PR) curve is more informative. The PR curve plots Precision (y-axis) against Recall (x-axis) as the threshold varies. The area under the PR curve, called Average Precision (AP), is the summary metric. A higher AP indicates better performance under class imbalance.

Comparison: ROC-AUC is appropriate when the dataset is balanced. PR-AUC (Average Precision) is appropriate when positives are rare (e.g., fraud detection where 0.1% of transactions are fraudulent, cancer detection where incidence is low).

2.9.6 Cross-Validation for Reliable Evaluation

Complete pseudocode (condensed):

1. function cross_validate_knn(data, labels, K_neighbors=7, K_folds=5):
2. folds = stratified_split(data, labels, K_folds)
3. scores = []
4. for i in 1..K_folds:
5. validation_data = folds[i]
6. training_data = concatenate(folds excluding folds[i])
7. scaler = StandardScaler()
8. scaler.fit(training_data.features)
9. X_train = scaler.transform(training_data.features)

10. X_val = scaler.transform(validation_data.features)
11. predictions = []
12. for each x in X_val:
13. distances = compute_euclidean(x, X_train)
14. nearest_K = find_K_smallest(distances, K_neighbors)
15. majority_vote = mode(training_data.labels[nearest_K])
16. predictions.append(majority_vote)
17. accuracy = mean(predictions == validation_data.labels)
18. scores.append(accuracy)
19. return mean(scores), std(scores)

K Value	**Pros**	**Cons**	**Best Use Case**
K=5	Fast, reasonable estimate	Higher variance	Large datasets (n>1000)
K=10	Good bias – variance balance	Moderate cost	Most applications
K=20	Low variance	High cost	Medium datasets
K=n (LOOCV)	Minimal bias, uses all data	Very expensive	Small dataseta (n<200)

Fold	Accuracy	Precision	Recall	F1	AUC
1	82.5%	0.71	0.68	0.695	0.86
2	84.0%	0.74	0.70	0.720	0.88
3	81.0%	0.68	0.65	0.665	0.84
4	83.5%	0.72	0.69	0.705	0.87
5	82.0%	0.70	0.67	0.685	0.85
6	82.6%	0.71	0.68	0.694	0.86

The performance metrics computed above should be estimated on data not used for training. Evaluating on the training set gives an optimistically biased estimate (training error is always lower than test error for flexible models like KNN with small K). The standard approach is K-fold cross-validation:

- Randomly partition the dataset into K equal-sized folds (K = 5 or K = 10 is typical).
- For each fold i from 1 to K:
- Use fold i as the validation set and all remaining folds as the training set.
- Train the KNN classifier on the training folds.
- Evaluate on the validation fold and record the performance metric.
- Average the K performance estimates to obtain the cross-validated performance.

Leave-One-Out Cross-Validation (LOOCV) is the special case where K = n (the total number of examples). Each example is used as a single-point validation set once. LOOCV is computationally expensive for large n but provides an approximately unbiased estimate of the true generalization error.

2.10 Performance of Regression Algorithms

Regression performance cannot be measured using classification metrics such as accuracy, since the predicted output is a continuous value. Instead, regression performance is assessed by measuring the deviation of predictions from true values. This section presents the standard regression evaluation metrics and illustrates their use with concrete examples.

2.10.1 Mean Absolute Error (MAE)

The Mean Absolute Error measures the average magnitude of prediction errors, treating all errors equally regardless of direction:

MAE = (1/n) * sum over i from 1 to n of |y_i - y_hat_i|

MAE is robust to outliers (a single large error does not dominate) and has the same units as the target variable, making it directly interpretable. If MAE = 25,000 USD in a house price prediction problem, the average prediction error is $25,000.

2.10.2 Mean Squared Error (MSE) and Root Mean Squared Error (RMSE)

The Mean Squared Error squares the errors before averaging:

MSE = (1/n) * sum over i from 1 to n of (y_i - y_hat_i)^2

Squaring penalizes large errors more heavily than small ones, making MSE sensitive to outliers. The Root Mean Squared Error (RMSE) restores the units of the target variable:

RMSE = sqrt((1/n) * sum over i from 1 to n of (y_i - y_hat_i)^2)

RMSE is the most widely used regression metric in machine learning literature. It is differentiable and therefore useful as a loss function for gradient-based optimization, unlike MAE.

2.10.3 Mean Absolute Percentage Error (MAPE)

MAPE expresses errors as a percentage of the true values:

MAPE = (100% / n) * sum over i from 1 to n of |y_i - y_hat_i| / |y_i|

MAPE is scale-independent, making it useful for comparing regression performance across datasets with different magnitudes. However, MAPE is undefined when any true value y_i = 0 and is asymmetric: overprediction and underproduction of the same percentage magnitude give different contributions

2.10.4 Coefficient of Determination (R-squared)

The R-squared (R^2) metric measures the proportion of variance in the target variable that is explained by the model:

R^2 = 1 - SS_res / SS_tot

where SS_res = sum of (y_i - y_hat_i)^2 is the residual sum of squares, and SS_tot = sum of (y_i - y_bar)^2 is the total sum of squares (proportional to the variance of y). y_bar is the mean of the true target values. R^2 = 1 means perfect prediction (all residuals are zero). R^2 = 0 means the model performs no better than predicting the mean of y for every point. R^2 < 0 means the model performs worse than the naive mean predictor — this can occur for highly flexible models (overfitting) evaluated on new data

Metric	Formula	Units	Best Value	Sensitive to Outliers?	Use When...
MAE	(1/n) sum |y - y_hat|	Same as y	0	No (robust)	Outliers should not be over-penalized
MSE	(1/n) sum (y - y_hat)^2	Square of y's units	0	Yes (high sensitivity)	Large errors are very undesirable
RMSE	sqrt(MSE)	Same as y	0	Yes	Interpretable scaled metric
MAPE	(100%/n) sum |y-y_hat|/|y|	Percentage (%)	0	No	Comparing across scales; y never 0
R-squared	1 - SS_res/SS_tot	Dimensionless	1	Moderate	Explaining proportion of variance

Table 2.13: Summary of regression performance metrics.

Worked Example 2.16 — Evaluating KNN Regression for Energy Consumption: A KNN regressor (K = 5) predicts hourly energy consumption (in kWh) for a commercial building. The test set contains 8 examples:

Example	True y	Predicted y_hat	Error (y - y_hat)	|Error|	Error^2	% Error
1	125	130	-5	5	25	4.00%
2	200	188	12	12	144	6.00%
3	150	155	-5	5	25	3.33%
4	80	90	-10	10	100	12.50%
5	310	295	15	15	225	4.84%
6	175	180	-5	5	25	2.86%
7	220	215	5	5	25	2.27%
8	95	100	-5	5	25	5.26%

Table 2.14: KNN regression predictions and error computation.

Summing the columns: Sum |Error| = 62, Sum Error^2 = 594, Sum %Error = 41.06%.

Mean of true values: y_bar = (125 + 200 + 150 + 80 + 310 + 175 + 220 + 95) / 8 = 1355/8 =169.375.

MAE = 62 / 8 = 7.75 kWh

MSE = 594 / 8 = 74.25 kWh^2 RMSE = sqrt(74.25) = 8.62 kWh MAPE = 41.06% / 8 = 5.13%

For R-squared: SS_tot = sum (y_i - 169.375)^2 = (125-169.375)^2 + ... = 1968.25 + 938.27 + ... (computing all terms) approximately 48,953.

SS_res = sum (y_i - y_hat_i)^2 = 594. R^2 = 1 - 594/48953 = 1 - 0.01213 = 0.9879.

An R^2 of 0.9879 indicates that the KNN regressor explains approximately 98.79% of the variance in energy consumption. The RMSE of 8.62 kWh on an average consumption of 169

kWh represents a 5.1% average error, which is excellent performance for an energy management system.

2.10.5 Adjusted R-squared

One limitation of R^2 is that it always increases (or stays the same) when more features are added to the model, even if the additional features are irrelevant. Adjusted R-squared penalizes model complexity:

$$R^2_{adj} = 1 - (1 - R^2) * (n - 1) / (n - d - 1)$$

where n is the number of training examples and d is the number of features. Adjusted R^2 decreases when irrelevant features are added (because the denominator n - d - 1 decreases). For KNN, the notion of "number of features" in adjusted R^2 is less directly applicable, but the concept of penalizing model complexity remains relevant when comparing models of different complexity.

2.11 Comparing and Selecting Classifiers

In practice, we often wish to compare multiple classifiers (or the same classifier with different hyperparameters) to select the best one for deployment. This requires statistical rigor beyond simply comparing point estimates of accuracy.

2.11.1 McNemar's Test

When two classifiers are evaluated on the same test set, their errors are correlated (because both classifiers see the same examples). McNemar's test is a chi-squared test designed for paired binary outcomes. It compares two classifiers by counting the examples that one classifier gets right but the other gets wrong.

Define: n01 = examples that Classifier 1 gets wrong but Classifier 2 gets right. n10 = examples that Classifier 1 gets right but Classifier 2 gets wrong. The McNemar statistic is:

$$chi^2 = (|n01 - n10| - 1)^2 / (n01 + n10)$$

This statistic follows a chi-squared distribution with 1 degree of freedom under the null hypothesis that both classifiers have the same error rate. If chi^2 > 3.841 (the 0.05 critical value), we reject the null hypothesis and conclude the classifiers differ significantly.

2.11.2 Bias-Variance Decomposition Revisited

The expected generalization error of any learning algorithm on a new point x can be decomposed as:

$$\text{Expected Error} = \text{Bias}^2 + \text{Variance} + \text{Irreducible Noise}$$

- Bias measures how much the average prediction (across many training sets of the same size) deviates from the true value. High bias = systematic error = under fitting.

- Variance measures how much the prediction changes across different training sets. High variance = sensitivity to training data = over fitting.
- Irreducible Noise is the inherent randomness in the data that no model can eliminate.

For KNN: as K increases, bias increases and variance decreases. As K decreases toward 1, bias decreases but variance increases. The optimal K minimizes the sum of bias squared and variance, which is achieved through cross-validation.

2.12 Comprehensive Case Study: Iris Species Classification with KNN

This section presents a complete end-to-end application of the KNN classifier to the famous Iris dataset, illustrating all concepts from feature preparation through performance evaluation.

2.12.1 Dataset Description

The Iris dataset, introduced by Ronald Fisher in 1936, contains measurements of 150 iris flowers from three species: Iris setosa (50 samples), Iris versicolor (50 samples), and Iris virginica (50 samples). Each flower is described by four features: sepal length (cm), sepal width (cm), petal length (cm), and petal width (cm).

2.12.2 Preprocessing

Step 1 - Feature Scaling: The four features have different scales and variances. After z-score standardization, the mean of each feature becomes 0 and the standard deviation becomes 1 across the training set. For illustration, a representative subset of 10 training points (pre- and post-scaling):

Species	Sepal L (raw)	Sepal W (raw)	Petal L (raw)	Petal W (raw)
Setosa	5.1	3.5	1.4	0.2
Setosa	4.9	3.0	1.4	0.2
Versicolor	7.0	3.2	4.7	1.4
Versicolor	6.4	3.2	4.5	1.5
Virginica	6.3	3.3	6.0	2.5
Virginica	5.8	2.7	5.1	1.9

Table 2.15: Representative Iris training samples (raw values).

2.12.3 KNN Classification with K = 5

A query flower with raw features (6.1, 3.0, 4.9, 1.8) is to be classified. After z-score

transformation using training set statistics (means: 5.843, 3.057, 3.758, 1.199; standard deviations: 0.828, 0.436, 1.765, 0.762), the standardized query is approximately:

petal_w_z = (1.8 - 1.199) / 0.762 = 0.788

The 5 nearest neighbors in the standardized space, along with their distances and species labels:

Rank	Training Sample (original)	Euclidean Distance (standardized)	Species
1	(6.0, 2.9, 4.5, 1.5)	0.340	Versicolor
2	(6.2, 2.9, 4.3, 1.3)	0.498	Versicolor
3	(6.3, 3.3, 4.7, 1.6)	0.385	Versicolor
4	(6.1, 2.8, 4.7, 1.2)	0.440	Versicolor
5	(6.4, 2.9, 4.3, 1.3)	0.612	Virginica

Table 2.16: Five nearest neighbors of the query iris flower.

Vote count: Versicolor = 4, Virginica = 1. Prediction: Iris versicolor. The query is classified as Iris versicolor with high confidence (4 out of 5 votes).

2.12.4 Performance Evaluation

Using 10-fold cross-validation on all 150 samples, the KNN classifier achieves the following performance for various values of K:

K	Cross-Val Accuracy (%)	Training Accuracy (%)	Overfit Indicator
1	95.3	100.0	High (gap = 4.7%)
3	96.0	97.3	Moderate (gap = 1.3%)
5	96.7	96.7	Minimal (gap = 0.0%)
7	96.0	95.3	None (bias increases)
11	94.7	94.0	None (under fitting)
21	92.0	91.3	Significant under fitting

Table 2.17: KNN cross-validation performance on the Iris dataset for varying K.

The optimal K is 5, achieving 96.7% cross-validated accuracy. At K = 1, the training accuracy is 100% (every training point is its own nearest neighbor), but the cross-validation accuracy drops, indicating overfitting. At K = 21, both training and test accuracy drop, indicating underfitting (the neighborhood is too large, erasing local class boundaries).

Chapter Summary

1. This chapter provided a rigorous and comprehensive treatment of nearest-neighbor-based machine learning models. The following key concepts were developed:

2. Proximity Measures: Distance and similarity are formalized through axioms (metric properties). The choice of proximity measure must match the data type — numeric data calls for Minkowski-family distances, binary data calls for Jaccard or Dice coefficients, and text data calls for cosine similarity.

3. Distance Measures: The Minkowski family (L1/Manhattan, L2/Euclidean, Linf/Chebyshev) provides a unified framework. Mahalanobis distance corrects for feature correlation and scale differences. Hamming distance handles binary and categorical strings.

4. Non-Metric Similarities: Cosine similarity and Pearson correlation are powerful for text and recommendation systems but do not satisfy the triangle inequality. The Tanimoto coefficient extends Jaccard to real-valued vectors.

5. Binary Proximity: SMC, Jaccard, Dice, and Russell-Rao differ in whether they count 00 matches. Jaccard and Dice are preferred when feature absences are uninformative.

6. KNN Classifier: A lazy learner that classifies queries by majority vote among K nearest neighbors. The hyperparameter K controls the bias-variance trade-off. Feature scaling is mandatory before applying KNN with Euclidean distance.

7. Radius Nearest Neighbor: Fixes the neighborhood radius r instead of K, enabling the classifier to abstain when no neighbors lie within r - a valuable property for anomaly and outlier detection.

8. KNN Regression: Predicts continuous output as the (possibly weighted) mean of K neighbors' output values. Distance-weighted regression gives more influence to closer neighbors.

9. Classifier Performance: The confusion matrix provides TP, FP, TN, FN counts from which accuracy, precision, recall, F1 score, and other metrics are derived. ROC-AUC measures discriminability; PR-AUC is preferred under class imbalance.

10. Regression Performance: MAE, MSE, RMSE, MAPE, and R^2 each capture different aspects of predictive accuracy. RMSE is the most commonly reported metric; R^2 provides a relative measure of variance explained.

11. Model Selection: Cross-validation provides an unbiased estimate of generalization performance and is essential for choosing K and comparing

models.

Key Terms

Term	Definitions
Metric	A distance function satisfying non-negativity, identity, symmetry, and triangle inequality
Euclidean Distance	L 2 norm; straight-line distance in d-dimensional space
Manhattan Distance	L1 norm; sum of absolute coordinate differences
Minkowski Distance	Generalized Lp norm; includes Euclidean and Manhattan as special cases
Mahalanobis Distance	Scale-invariant distance accounting for feature covariance
Hamming Distance	Number of differing positions between two equal-length strings or vectors
Cosine Similarity	Cosine of the angle between two vectors; magnitude-invariant
Jaccard Coefficient	Ratio of intersection to union size; ignores 00 matches in binary data
Dice Coefficient	Double-weighted Jaccard; equivalent to F1 score in binary similarity
KNN Classifier	Assigns query the majority class among its K nearest training neighbors
Lazy Learning	Defers generalization to query time; stores all training instances
Bias-Variance Trade-off	Tension between underfitting (high bias) and overfitting (high variance)
Confusion Matrix	Table of TP, FP, FN, TN counts for a binary classifier
ROC Curve	P lot of TPR vs FPR across all thresholds; area (AUC) measures discriminability
MAE	Mean Absolute Error; average absolute deviation of predictions from true values
RMSE	Root Mean Squared Error; square root of MSE; in same units as target
R-squared	Proportion of target variance explained by the model; 1 = perfect prediction

Cross-Validation	Repeated train/test split to obtain reliable generalization estimates
Curse of Dimensionality	Degradation of distance-based methods as number of features grows large
Feature Scaling	Normalization or standardization of features to equal range or variance

Table 2.18: Key terms and definitions for Chapter II

Review Questions

Section A — Conceptual Questions

1. Explain why cosine similarity is preferred over Euclidean distance for measuring the similarity between two text documents. Under what conditions would Euclidean distance give misleading results in this context?

2. Compare and contrast the Simple Matching Coefficient (SMC) and the Jaccard Coefficient for binary data. Provide a real-world scenario where each would be the more appropriate choice, and justify your answer.

3. Explain the curse of dimensionality in the context of KNN classification. Describe two practical strategies to mitigate its effects.

4. Why must features be scaled before applying the KNN algorithm with Euclidean distance? Provide a concrete numerical example to illustrate the problem that arises without scaling.

5. Describe the bias-variance trade-off in KNN classification as a function of K. What happens at K = 1 and K = n? How would you determine the optimal K for a given dataset?

6. What is the difference between lazy and eager learning? Describe two advantages and two disadvantages of lazy learning compared to eager learning.

7. Explain the Mahalanobis distance. When would you prefer Mahalanobis distance over Euclidean distance? Give a real-world example.

Section B — Analytical Problems

8. Three data points in two dimensions are: A = (1, 2), B = (4, 6), C = (3, 1). Compute: (a) d_E(A, B), d_E(A, C), d_E(B, C) using Euclidean distance. (b) d_M(A, B), d_M(A, C) using Manhattan distance. (c) Verify the triangle inequality for Euclidean distance with A, B, C.

9. Two binary patient records of length 10 are: x = (1, 0, 1, 1, 0, 1, 0, 0, 1, 1) and y = (1, 1,0, 1, 0, 1, 0, 1, 0, 1). Compute: (a) The 2x2 contingency table (a, b, c, d). (b) SMC, Jaccard coefficient, and Dice coefficient. (c) Which measure would you use if the records represent which rare symptoms a patient has? Why?

10. Using the training data in Table 2.7 (house price prediction), apply KNN regression with K = 4 to predict the price of a house with normalized features x* = (0.62, 0.32). Compare the unweighted and inverse-distance-weighted

predictions.

11. A binary classifier is evaluated on 500 test examples: TP = 180, FP = 30, TN = 270, FN = 20. Compute: (a) Accuracy, Error Rate, Precision, Recall, Specificity, F1 Score. (b) If this were a cancer screening test, which metric would you prioritize and why?

12. For a multiclass KNN classifier distinguishing three fruit types (apple, orange, banana) from sensor data, the 9-cell confusion matrix is: M[apple][apple] = 40, M[apple][orange]= 3, M[apple][banana] = 2; M[orange][apple] = 2, M[orange][orange] = 35, M[orange][banana] = 8; M[banana][apple] = 1, M[banana][orange] = 4, M[banana][banana] = 45. Compute the overall accuracy and per-class precision and recall.

Section C — Applied and Design Problems

13. You are building a product recommendation system for an e-commerce platform. The item-user interaction data is sparse binary (1 = purchased, 0 = not purchased). Describe your complete approach to computing item-item similarity, choosing K, and evaluating the recommender system performance. Which proximity measure would you choose and why?

14. Design a KNN-based intrusion detection system for a corporate network. Your features include: packet size (bytes), packets per second, connection duration (seconds), number of failed login attempts. Describe: (a) How you would preprocess each feature. (b) Which distance measure you would use. (c) How you would choose K. (d) what performance metric is most appropriate for this security application and why.

15. Compare the Radius Nearest Neighbor classifier and the KNN classifier for a medical imaging application where some tissue regions may not resemble any known category of tissue in your training set. Which approach is more appropriate and how would you set its parameters?

16. A KNN regression model for predicting electricity demand (kWh) achieves RMSE = 45 kWh, MAE = 32 kWh, MAPE = 8.5%, and R^2 = 0.87 on a validation set. A linear regression model on the same data achieves RMSE = 52 kWh, MAE = 38 kWh, MAPE = 10.2%, and R^2 = 0.82. (a) Which model performs better according to each metric? (b) In what scenario might you prefer the linear regression despite its lower performance? (c) How would you decide which to deploy?

CHAPTER III
Models Based on Decision Trees

Chapter Overview

This Chapter introduces **tree-based and probabilistic classification methods** two foundational families of supervised learning algorithms that take fundamentally different approaches to decision-making. Starting from the intuitive concept of splitting data by asking a series of yes/no questions, the chapter builds systematically through increasingly powerful methods: Decision Trees, Random Forests, the Bayes Classifier, and the Naïve Bayes Classifier. Throughout, the emphasis is on mathematical rigor, geometric intuition, and worked numerical examples.

The chapter starts with **Decision Trees** — flowchart-like structures that recursively partition the feature space using binary splits on individual features. Each internal node tests a feature condition, branches represent outcomes, and leaf nodes output class predictions. The greedy tree-building algorithm selects the best feature and threshold at each step using **impurity measures**: Entropy (used in ID3/C4.5) and the Gini Index (used in CART). Information Gain quantifies the entropy reduction achieved by each candidate split, guiding the algorithm toward splits that most cleanly separate classes.

Decision trees are highly interpretable and handle mixed feature types naturally, but they are prone to **over fitting** a fully grown tree may memorize training data, achieving perfect training accuracy but poor test accuracy. Pruning strategies (pre-pruning by limiting depth or minimum samples; post-pruning by trimming non-contributory branches) address this. The same tree structure extends naturally to **regression tasks** by outputting the mean of training samples at each leaf and splitting to minimize variance.

The **bias–variance trade-off** the central tension between model simplicity (high bias, low variance) and complexity (low bias, high variance) — is explained formally: Total Error = Bias^2 + Variance + Irreducible Noise. Decision trees exemplify high variance: small changes in training data produce very different trees. This motivates **Random Forests**, an ensemble method that constructs B

trees on bootstrap samples and, crucially, limits each split to a random subset of m features. By decorrelating trees, Random Forests dramatically reduce variance while maintaining low bias, producing more stable and accurate predictions through majority vote (classification) or averaging (regression).

The second half of the chapter shifts to **probabilistic classification** via Bayes' theorem. The **Bayes Classifier** predicts the class with the highest posterior probability P(Y|X), computed from the likelihood P(X|Y), the prior P(Y), and the evidence P(X). It is the theoretically **optimal classifier** when true distributions are known — no other classifier can achieve lower error. The Bayes error rate ε* quantifies the irreducible floor of prediction error, arising from class overlap in feature space.

The chapter concludes with the **Naïve Bayes Classifier**, a practical approximation that assumes conditional independence among features given the class label. This assumption reduces the number of parameters to estimate from exponential (2^d per class) to linear (d per class), making Naïve Bayes fast, scalable, and effective even with small datasets. **Laplace smoothing** prevents zero-probability catastrophes for unseen feature values. Despite its often-violated independence assumption, Naïve Bayes works well in practice particularly for text classification and spam filtering because correct class ranking (not exact probabilities) is sufficient for accurate predictions

Section-by-Section Roadmap

Section	Topic	Key Concepts
3.1	**Decision Trees for Classification**	Root/internal/leaf nodes, branch splits, numeric & categorical features, greedy algorithm
3.2	**Impurity Measures**	Entropy, Information Gain, Gini Index, comparison of ID3/C4.5/CART
3.3	**Properties of Decision Trees**	Advantages (interpretability, no scaling), disadvantages (overfitting, instability), pruning strategies

3.4	**Regression Based on Decision Trees**	Variance minimization criterion, mean leaf prediction, variance reduction formula
3.5	**Bias–Variance Trade-off**	$Bias^2$, Variance, irreducible noise, model complexity curve, underfitting vs. overfitting
3.6	**Random Forests**	Bootstrap aggregating (bagging), OOB samples, random feature selection, majority vote/averaging
3.7	**Introduction to Bayes Classifier**	Posterior probability, MAP prediction, probabilistic classification intuition
3.8	**Bayes' Rule and Inference**	Likelihood, prior, posterior, evidence, Bayes' theorem, medical diagnosis example
3.9	**Bayes Classifier and Its Optimality**	Bayes error rate ε^*, irreducible error, estimation error, theoretical gold standard
3.10	**Multi-Class Classification**	K-class Bayes rule, decision boundaries, Gaussian class-conditionals, Iris example
3.11	**Class-Conditional Independence & Naïve Bayes**	Dimensionality problem, independence assumption, Laplace smoothing, log-form computation

3.1 Decision Trees and Classification

A decision tree is a flowchart-like structure used for classification. It asks a series of questions about the input features, and based on the answers, arrives at a class prediction.

Definition 3.1.1 Decision Tree
A decision tree has:

- **Root node:** Top node where classification starts
- **Internal nodes:** Test a feature condition
- **Branches:** Represent test outcomes
- **Leaf nodes:** Give final class predictions

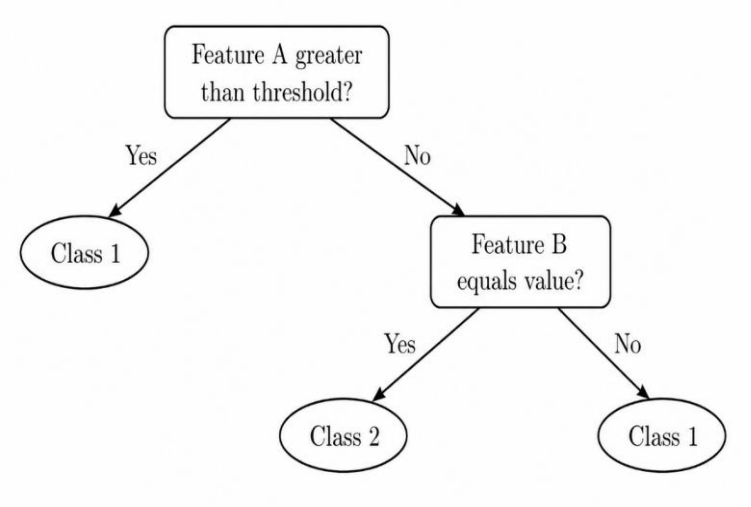


Figure 3.1: Decision Tree Structure for Binary Classification

Classification Process:

1. Start at root node
2. Test the condition on the relevant feature
3. Follow the branch matching the outcome
4. Repeat until reaching a leaf
5. Output the class label at the leaf

Types of Splits:

Numeric features: $X \leq t$ vs $X > t$ for threshold t

Categorical features: One branch per category

Example 3.1.1 Decision Tree for Loan Approval

A bank wants to classify loan applications as **Approve** or **Reject**.

Features: Credit Score, Annual Income, Employment Status

Decision Tree:

1. Is Credit Score > 700?
   - Yes → Approve
   - No → Is Annual Income > 50000?
     - Yes → Is Employment = Permanent?
       - Yes → Approve
       - No → Reject
     - No → Reject

Classifying a new applicant:
Credit Score = 650, Income = 60000, Employment = Permanent

- Credit Score > 700? → No → go right
- Income > 50000? → Yes → go left
- Employment = Permanent? → Yes → **Approve**

Building a Decision Tree (Greedy Algorithm):

1. Start with all training data at root

2. Find the best feature and split point (using impurity measures)

3. Split data into child nodes

4. Repeat recursively for each child

5. Stop when: all samples have same class, max depth reached, or min samples threshold.

3.2 Impurity Measures

Definition 3.2.1 (Impurity Measure): An impurity measure is a function that quantifies how "mixed" or "impure" the class labels are at a node in a decision tree. A node is pure if all samples belong to one class; it is impure if samples are divided among multiple classes. Impurity measures help us evaluate and compare potential splits.

We use impurity measures to decide which feature gives the best split.

3.2.1 Entropy

Entropy is a measure of uncertainty or randomness borrowed from information theory. In the context of decision trees, it tells us how much "disorder" exists in the class labels at a node.

Definition 3.2.2 (Entropy) For a node with K classes and class proportions p1, p2… pK:

$$H = -\sum_{k=1}^{K} p_k \log_2(p_k)$$

Convention: 0 · log2(0) = 0

Properties:
- H = 0: Pure node (all one class)
- H = 1: Maximum impurity for binary classification (50–50 split)

Example 3.2.1 (Entropy Calculation)
Dataset: 14 samples for "Play Tennis" decision — 9 Yes, 5 No

Calculate entropy at root:

$p_(yes) = 9/14 = 0.643$, $p_(no) = 5/14 = 0.357$ (3.2)

$H = -0.643 \log_2(0.643) - 0.357 \log_2(0.357)$ (3.3)

$= -0.643 \times (-0.637) - 0.357 \times (-1.485)$ (3.4)

$= 0.410 + 0.530 = 0.940$ bits (3.5)

This is close to 1, meaning high uncertainty (classes are fairly mixed)

3.2.2 Information Gain

Information Gain measures how much a particular feature "informs" us about the class label essentially, how much uncertainty is reduced after splitting on that feature.

Definition 3.2.3 (Information Gain)

$$Gain(S, A) = H(S) - \sum_{v \in Values(A)} \frac{|S_v|}{|S|} H(S_v)$$

where Sv is the subset where feature A has value v.

Example 3.2.2 (Information Gain Calculation)

Continuing the "Play Tennis" example. Split on *Outlook* (Sunny, Overcast, Rain):

After split:

- Sunny: 5 samples (2 Yes, 3 No) → H = 0.971
- Overcast: 4 samples (4 Yes, 0 No) → H = 0 (pure!)
- Rain: 5 samples (3 Yes, 2 No) → H = 0.971

Weighted average entropy after split:

$$H_{after} = \frac{5}{14}(0.971) + \frac{4}{14}(0) + \frac{5}{14}(0.971) = 0.694$$

Information Gain:

Gain(Outlook) = 0.940 − 0.694 = 0.246 bits (3.8)

Higher gain = better split. We choose the feature with highest gain.

3.2.3 Gini Index

Gini Index (or Gini Impurity) is another measure of node impurity, used by the CART (Classification and Regression Trees) algorithm. It represents the probability of incorrectly classifying a randomly chosen sample if it were labeled according to the class distribution at that node.

Definition 3.2.4 (Gini Index)

$$\text{Gini} = 1 - \sum_{k=1}^{K} p_k^2$$

Interpretation: Probability of misclassifying a randomly chosen sample.

Example 3.2.3 (Gini Index Calculation) Same root node:
9 Yes, 5 No (14 total)
Gini = $1 - (9/14)^2 - (5/14)^2 = 1 - 0.413 - 0.128 = 0.459$ (3.11)
For a pure node (all Yes): Gini = $1 - 1^2 = 0$
For maximum impurity (7 Yes, 7 No): Gini = $1 - 0.25 - 0.25 = 0.5$

3.2.4 Comparison of Impurity Measures

Measure	Formula (Binary)	Range	Used In
Entropy	$-p \log_2 p - (1-p) \log_2(1-p)$	[0, 1]	ID3, C4.5
Gini Index	$2p(1-p)$	[0, 0.5]	CART

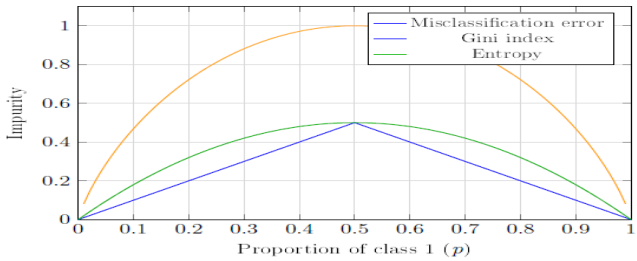


Figure 3.2: *Impurity Measures as Functions of Class Proportion. All three measures are minimized (equal to 0) at pure nodes (p = 0 or p = 1) and maximized at p = 0.5.*

3.3.1 Advantages

- Interpretable: Easy to visualize and explain decisions
- No feature scaling: Works with raw values
- Handles mixed types: Both numeric and categorical
- Non-linear boundaries: Can capture complex patterns
- Feature selection: Important features appear at top

Example 3.3.1 (Interpretability Advantage)

A hospital uses a decision tree to predict heart disease risk:

"If age > 55 AND cholesterol > 240 AND blood pressure > 140, then High Risk". This rule is easily understood by doctors, unlike a neural network's prediction.

3.3.2 Disadvantages

• Overfitting: Can memorize training data
• Instability: Small data changes create very different trees
• Axis-parallel splits: Cannot model diagonal boundaries easily
• Greedy: May miss globally optimal structure

Example 3.3.2 (Overfitting Problem)

Training data has 100 samples. A fully grown tree might create 100 leaves (one per sample), achieving 100% training accuracy but poor test accuracy.

Solution: Limit tree depth or require minimum samples per leaf.

3.3.3 Pruning to Prevent Overfitting

Pruning is a technique to reduce the size of a decision tree by removing sections that provide little predictive power. It helps prevent overfitting by simplifying the model.

Pre-pruning (stop early):
• Set maximum depth
• Require minimum samples to split
• Require minimum samples per leaf

Post-pruning (grow then cut):
• Grow full tree, then remove branches that do not improve validation accuracy

3.4 Regression Based on Decision Trees

Regression trees extend the decision tree concept to predict continuous numerical values instead of discrete class labels. While classification trees output a class at each leaf, regression trees output a numerical value (typically the average of training samples that fall into that region).

Definition 3.4.1 (Regression Tree): A regression tree partitions feature space into regions and predicts the mean value in each region:

where $\mathbf{c_m}$ **= mean of y values in region** $\mathbf{R_m}$.

Splitting Criterion: Instead of entropy/Gini, we minimize variance:

$$\mathrm{Var}(t) = \frac{1}{n}\sum_{i \in t}(y_i - \bar{y})^2$$

Choose splits that maximize variance reduction:

$$\Delta \text{Var} = \text{Var}(\text{parent}) - \frac{n_L}{n}\text{Var}(\text{left}) - \frac{n_R}{n}\text{Var}(\text{right})$$

Example 3.4.1 (Regression Tree for House Prices) Data:

House	Sq. Feet	Price ($K)
1	1200	150
2	1400	180
3	1600	200
4	2000	280
5	2400	350
6	2800	400

Tree splits at 1800 sq ft:

• Left child (Houses 1,2,3): Mean price = (150 + 180 + 200) / 3 = $177K

• Right child (Houses 4,5,6): Mean price = (280 + 350 + 400) / 3 = $343K

Prediction for 1500 sq ft house: Falls in left region → predict $177K

3.5 Bias–Variance Trade-off

The bias–variance trade-off is a fundamental concept in machine learning that explains why models either underfit or overfit. It describes the tension between a model's ability to fit training data closely (low bias) versus its ability to generalize to new data (low variance).

Total Prediction Error = Bias² + Variance + Irreducible Noise

Definition 3.5.1 (Bias and Variance)

- **Bias:** Error from oversimplified models that miss patterns
- **Variance:** Error from models too sensitive to training data
- **Irreducible Noise:** Random error in the data itself

Model Complexity	Bias	Variance
Too Simple (underfitting)	High	Low
Just Right	Medium	Medium
Too Complex (overfitting)	Low	High

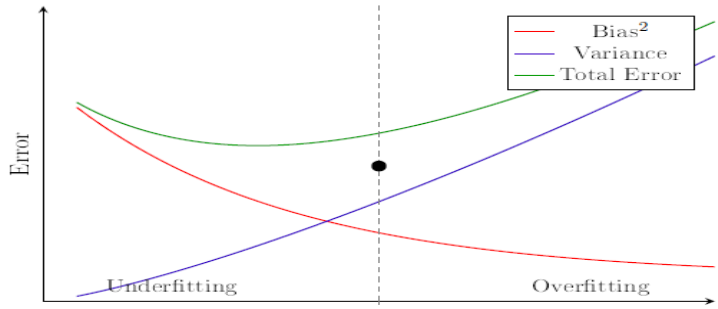


Figure 3.3: *Bias–Variance Trade-off with Model Complexity. As model complexity increases, bias decreases but variance increases. The optimal model minimizes total error.*

Example 3.5.1 (Bias-Variance in Decision Trees)
Task: Predict exam scores based on study hours

Depth-1 tree (decision stump):

- Rule: If hours > 5, predict 75; else predict 45
- High bias: Too simple, misses the gradual relationship
- Low variance: Similar tree on different training sets

Depth-10 tree:

- Creates many small regions, fits training data perfectly
- Low bias: Captures every detail
- High variance: Different training sets give completely different trees

Depth-3 tree:

- Balanced: Captures main pattern without overfitting
- Best test performance

Key Insight: Decision trees have high variance — small changes in data lead to very different trees. This motivates ensemble methods.

3.6 Random Forests for Classification and Regression

A Random Forest is an ensemble learning method that constructs multiple decision trees during training and combines their predictions. It addresses the high variance problem of individual decision trees by averaging predictions from many diverse trees.

Key Idea: The average of many unstable models is stable. By combining many trees that make different errors, the errors tend to cancel out.

3.6.1 Bootstrap Aggregating (Bagging)

Bagging (Bootstrap Aggregating) is a technique for reducing variance by training multiple models on different random subsets of the training data and combining their predictions.

Definition 3.6.1 (Bootstrap Sample) Sample n instances with replacement from a dataset of size n.

Each bootstrap sample contains about 63% of the original data. The remaining 37% are out-of-bag (OOB) samples.

3.6.2 Random Forest Algorithm

1. For each tree b = 1 to B:
(a) Draw a bootstrap sample
(b) Grow a tree, but at each split:
    - Randomly select m features (not all)
    - Find best split among only these m features
(c) Grow fully (no pruning)

2. Predict by:
- Classification: Majority vote
- Regression: Average

Typical m values:
- Classification: $m = \sqrt{d}$
- Regression: $m = d/3$

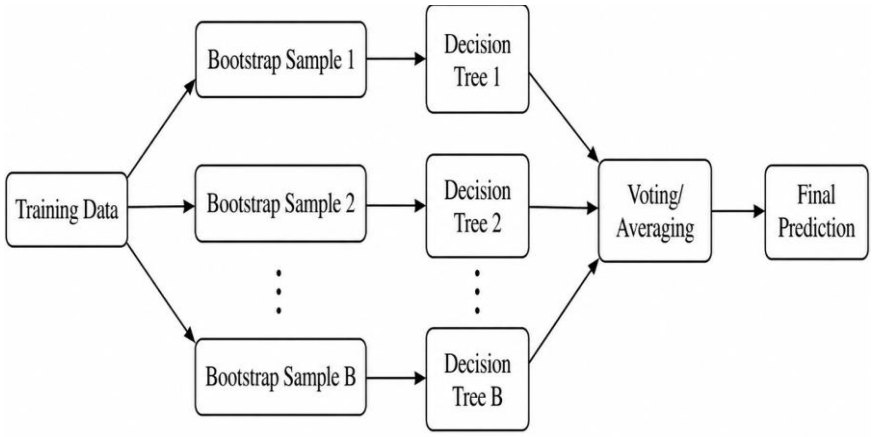


Figure 3.4: *Random Forest Architecture. Training data is used to create multiple bootstrap samples, each used to train a decision tree. Predictions from all trees are combined through voting (classification) or averaging (regression).*

Example 3.6.1 (Random Forest for Email Classification)
Task: Classify emails as Spam or Not Spam

Features (20 total): word frequencies ("free", "money", "meeting", etc.)

Random Forest with 100 trees:

For each tree:
• Draw bootstrap sample of 1000 emails
• At each split, randomly consider $\sqrt{20} \approx 4$ words
• Tree 1 might split on "free" at root
• Tree 2 might split on "money" at root (since "free" wasn't in the random 4)

New email arrives:
• 73 trees vote "Spam"
• 27 trees vote "Not Spam"

Final prediction: Spam (majority)

3.6.3 Why Random Feature Selection Works

If one feature dominates (e.g., "free" is very predictive of spam), all trees would split on it first — trees become similar (correlated).
Random feature selection forces trees to explore different structures, making them decorrelated. The ensemble variance reduces.

Example 3.6.2 (Variance Reduction)
Single decision tree on email data: 85% accuracy (varies 80–90% on different test sets)

Random Forest (100 trees): 92% accuracy (varies 91–93% on different test sets) The forest is more stable and accurate.

3.6.4 Properties of Random Forests

Advantages:
- Much lower variance than single trees
- Resistant to overfitting
- Built-in OOB error estimate
- Provides feature importance

Disadvantages:
- Less interpretable (cannot visualize 100 trees)
- Slower to train and predict
- Higher memory usage

3.7 Introduction to the Bayes Classifier

The Bayes classifier is a probabilistic approach to classification that uses Bayes' theorem from probability theory. Unlike decision trees which learn explicit rules, Bayes classifiers learn the probability distributions of features for each class and use these to make predictions.

Core Idea: Given features x, compute the probability of each class and predict the class with the highest probability.

Definition 3.7.1 (Bayes Classifier)

Predict the class with the highest posterior probability:

$$\hat{y}(\mathbf{x}) = \arg\max_{k} P(Y = c_k \mid \mathbf{X} = \mathbf{x})$$

Example 3.7.1 (Bayes Classifier for Weather Prediction) Task: Predict if it will rain tomorrow

Features: Today's humidity (High/Low), Wind (Strong/Weak)

Given: Humidity = High, Wind = Weak

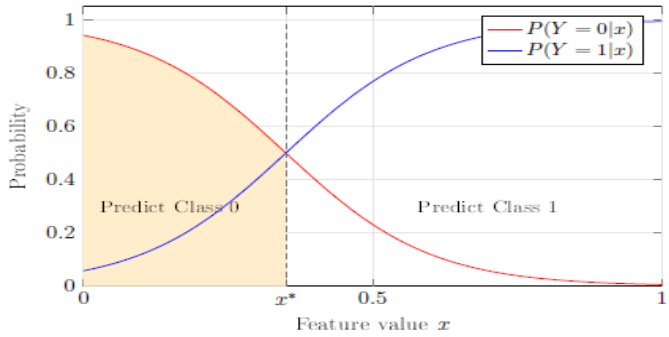


Figure 3.5: Posterior Probability and Decision Boundary. The Bayes Classifier predicts

the class with higher posterior probability. The decision boundary x∗ is where P(Y =

0|x) = P(Y = 1|x).

Suppose we know:

$$P(\text{Rain}|\text{High, Weak}) = 0.7 \quad (3.16)$$

$$P(\text{No Rain}|\text{High, Weak}) = 0.3 \quad (3.17)$$

Bayes classifier predicts: Rain (since 0.7 > 0.3)

The challenge is: How do we compute these posterior probabilities?

3.8 Bayes' Rule and Inference

Bayes' theorem (or Bayes' rule) is a fundamental result in probability theory that describes how to update our beliefs about hypothesis given new evidence. In classification, it allows us to compute the probability of a class given the observed features, using quantities we can estimate from training data.

Definition 3.8.1 (Bayes' Theorem)

Components:

- Posterior P(Y |X): Probability of class given features (what we want)
- Likelihood P(X|Y): How features are distributed in each class
- Prior P(Y): Overall class frequency
- Evidence P(X): Probability of seeing these features

Since evidence is same for all classes, we simplify:

$$P(Y = c_k|\mathbf{X} = \mathbf{x}) = \frac{P(\mathbf{X} = \mathbf{x}|Y = c_k) \cdot P(Y = c_k)}{P(\mathbf{X} = \mathbf{x})}$$

$$\hat{y} = \arg\max_k \underbrace{P(\mathbf{X} \mid Y = c_k)}_{\text{likelihood}} \cdot \underbrace{P(Y = c_k)}_{\text{prior}}$$

Example 3.8.1 (Medical Diagnosis using Bayes' Rule)
Task: Diagnose disease based on test result

Known information:
• Disease prevalence: P(Disease) = 0.01 (1%)
• Test sensitivity: P(Positive | Disease) = 0.95 (95%)
• Test specificity: P(Negative | Healthy) = 0.90 (90%)

So: P(Positive | Healthy) = 0.10 (false positive rate)

Question: Patient tests positive. What is P(Disease | Positive)?
Step 1: Calculate P(Positive)

P(Positive) = P(Positive | Disease) · P(Disease) + P(Positive | Healthy) · P(Healthy) (3.20)

= 0.95 × 0.01 + 0.10 × 0.99 (3.21)

= 0.0095 + 0.099 = 0.1085 (3.22)

Step 2: Apply Bayes' theorem

P(Disease | Positive) = (0.95 × 0.01) / 0.1085

= 0.0095 / 0.1085

= 0.0875 = 8.75% (3.23)

Surprising result: Despite a positive test, there is only an **8.75%** chance of disease.

Why? The disease is rare (low prior). Most positive results come from the large healthy population (false positives).

3.9 The Bayes Classifier and Its Optimality

The optimality of the Bayes Classifier refers to the mathematical proof that when we know the true probability distributions, no other Classifier can achieve a lower error rate. This makes the Bayes Classifier the theoretical gold standard against which all other Classifiers are compared.

Definition 3.9.1 (Optimality of Bayes Classifier) The Bayes Classifier minimizes the probability of misclassification. No other Classifier can achieve lower error when true distributions are known.

Intuition: If you know the true probabilities, always picking the most likely class is the best strategy.

Definition 3.9.2 (Bayes Error Rate) The minimum possible error rate:

$$\text{Bayes Error Rate} = \mathbb{E}_X \left[1 - \max_k P(Y = k \mid X) \right]$$

This is irreducible _ even the perfect classifier cannot beat it.

Example 3.9.1 (Bayes Error Rate) Scenario: Classifying fruits as Apple or

Orange based on weight.

Suppose for a fruit weighing 150g:

- P(Apple|150g) = 0.6
- P(Orange|150g) = 0.4

Bayes classifier predicts: Apple

But 40% of 150g fruits are actually oranges _ we will misclassify them.

Bayes error at this point = 1 − 0.6 = 0.4 = 40%

This error is unavoidable because apples and oranges overlap at 150g.

Practical Note: We don't know true distributions, so real classifiers have higher

error:

Real Error = Bayes Error + Estimation Error

3.10 Multi-Class Classification

Multi-class classification refers to problems where the output variable can take more than two possible values. While binary classification deals with two classes (e.g., spam/not- spam), multi-class classification handles three or more classes (e.g., classifying owers into setosa, versicolor, or virginica).

Bayes classification extends naturally to any number of classes.

Definition 3.10.1 (Multi-Class Bayes Classifier) For K classes:

$$\hat{y} = \arg \max_{k \in \{1,2,\ldots,K\}} P(Y = c_k | X = x)$$

Decision Boundaries: The boundary between classes i and j is where:

$$P(Y = c_i|\mathbf{X}) = P(Y = c_j|\mathbf{X})$$

Example 3.10.1 (3-Class Classification: Iris Flowers) Task: Classify iris flowers into Setosa, Versicolor, or Virginica

Features: Petal length, Petal width

For a flower with petal length = 4.5cm, petal width = 1.5cm:

P(Setosa|features) = 0.02 (3.27)

P(Versicolor|features) = 0.73 (3.28)

P(Virginica|features) = 0.25 (3.29)

Prediction: Versicolor (highest probability)

Confidence: 73% _ we can report this probability to the user.

Common Model: Gaussian class-conditional distributions

If features follow Gaussian in each class:

$$P(\mathrm{X}|Y = c_k) = \frac{1}{(2\pi)^{d/2}|\Sigma_k|^{1/2}} \exp\left(-\frac{1}{2}(\mathrm{x} - \mu_k)^T \Sigma_k^{-1}(\mathrm{x} - \mu_k)\right)$$

3.11 Class-Conditional Independence and Naive Bayes Classifier

The **Naive Bayes classifier** is a simplified Bayesian classifier that makes a strong (_naive_) assumption: all features are conditionally independent given the class label. Despite this often unrealistic assumption, Naive Bayes works remarkably well in practice and is widely used for text classification, spam filtering, and other applications.

3.11.1 The Dimensionality Problem

Estimating P(X|Y) is hard in high dimensions.

With d binary features, we need 2d probability values per class!

- d = 10: 1,024 values per class
- d = 20: 1,048,576 values per class
- d = 30: Over 1 billion values per class

We don't have enough data to estimate all these.

3.11.2 The Naive Bayes Assumption

Conditional independence means that once we know the class, knowing the value of one feature tells us nothing about another feature. For example, if we know an email is spam, the presence of _free_ tells us nothing about the presence of _money_ _ they are assumed to be independent.

Definition 3.11.1 (Conditional Independence) Naive Bayes assumes features are independent given the class:

$$P(X_1, X_2, \ldots, X_d | Y) = \prod_{j=1}^{d} P(X_j | Y)$$

This reduces parameters from 2d to just d per class!

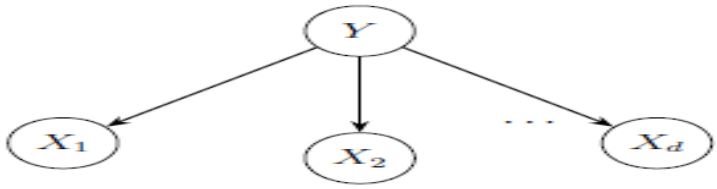


Figure 3.6: *Naive Bayes Independence Assumption. The class variable Y is the parent of all feature variables X1,X2, . . ,Xd. Given the class, features are conditionally independent of each other (no arrows between features).*

Definition 3.11.2 (Naive Bayes Classifier)

$$\hat{y} = \arg\max_k \left[P(Y = c_k) \prod_{j=1}^{d} P(X_j = x_j | Y = c_k) \right]$$

Log form (numerically stable):

$$\hat{y} = \arg\max_k \left[\log P(Y = c_k) + \sum_{j=1}^{d} \log P(X_j = x_j | Y = c_k) \right]$$

3.11.3 Training Naive Bayes

Simply count occurrences in training data:

Prior:

$$P(Y = c_k) = \frac{\text{number of samples in class } k}{\text{total samples}}$$

Likelihood (categorical features):

$$P(X_j = v | Y = c_k) = \frac{\text{count of } X_j = v \text{ in class } k}{\text{total samples in class } k}$$

Example 3.11.1 (Naive Bayes for Spam Detection) Training data: 100 emails

(60 spam, 40 ham)

Features: Contains word “free”?, Contains word “meeting”?

Feature	Spam (60)	Ham (40)
“free” = Yes	48	4
“free” = No	12	36
“meeting” = Yes	6	32
“meeting” = No	54	8

Counts:

Probabilities:

P(Spam) = 60/100 = 0.6 (3.36)
P(Ham) = 40/100 = 0.4 (3.37)

P(free = Y | Spam) = 48/60 = 0.8 (3.38)
P(free = Y | Ham) = 4/40 = 0.1 (3.39)

P(meeting = Y | Spam) = 6/60 = 0.1 (3.40)
P(meeting = Y | Ham) = 32/40 = 0.8 (3.41)

Classify new email: Contains “free”, no “meeting”

P(Spam) · P(free = Y |Spam) · P(meeting = N|Spam) = 0.6 × 0.8 × 0.9 = 0.432 (3.42)

Prediction: Spam (0.432 >> 0.008)

3.11.4 Laplace Smoothing

Laplace smoothing (also called additive smoothing) is a technique to handle the zero-frequency problem in Naive Bayes. It adds a small count to all feature-class combinations so that no probability is ever exactly zero.

Problem: If a word never appears in spam training emails, P(word|Spam) = 0, making entire product zero.

Solution: Add small count to all:

$$P(X_j = v|Y = c_k) = \frac{\text{count}(X_j = v, Y = c_k) + 1}{\text{count}(Y = c_k) + V} \quad (3.44)$$

where V = number of possible values.

Example 3.11.2 (Laplace Smoothing) Word “lottery” appears 0 times in 40 ham

training emails.

Without smoothing: P(lottery|Ham) = 0/40 = 0

Any email with “lottery” gets P(Ham) = 0 _ always classified as Spam!

With Laplace smoothing (binary feature, V = 2):

$$P(lottery = Y|Ham) = \frac{0+1}{40+2} = \frac{1}{42} = 0.024$$

Now “lottery” in ham is unlikely but not impossible.

3.11.5 Why Does Naive Bayes Work?

The independence assumption is clearly wrong "free" and "money" often appear together in spam.

But Naive Bayes still works because:

- We only need correct ranking of classes, not exact probabilities
- Errors from dependencies may cancel out
- Few parameters = less overfitting

3.11.6 Properties of Naive Bayes

Advantages:

- Very fast: O(nd) training, O(Kd) prediction
- Works well with small data
- Handles high dimensions
- Handles missing values naturally

Disadvantages:

- Independence assumption often violated
- Probability estimates are poorly calibrated
- Cannot capture feature interactions

Chapter Summary

Decision Trees:

- Flowchart-like classifiers using if-then rules
- Built using impurity measures (Entropy, Gini) to find best splits
- Easy to interpret but prone to overfitting

Impurity Measures:

$$\text{Entropy: } H = -\sum p_k \log_2 p_k$$

$$\text{Gini: } G = 1 - \sum p_k^2$$

- Choose splits that maximize information gain / Gini reduction

Regression Trees:

- Predict continuous values using mean of leaf region
- Split to minimize variance

Bias-Variance Trade-off:

- Simple models: high bias, low variance
- Complex models: low bias, high variance
- Find the right balance

Random Forests:

- Ensemble of trees with bootstrap samples and random feature selection
- Reduces variance while keeping low bias

Bayes Classifier:

- Predicts class with highest posterior probability
- Uses Bayes' theorem: $P(Y|X) \propto P(X|Y) \cdot P(Y)$
- Optimal when true distributions known

Naive Bayes:

- Assumes feature independence given class
- Reduces parameters from exponential to linear
- Fast, simple, and surprisingly effective.

Review Questions

Section A — Conceptual Questions

1. Define a decision tree and explain the role of each of its components: root node, internal nodes, branches, and leaf nodes. Using the loan approval example from the chapter, trace the classification path for an applicant with Credit Score = 720, Income = 40,000, and Employment = Contract.

2. Compare and contrast Entropy and Gini Index as impurity measures for decision tree splitting. Provide the formula for each, state the algorithm each is associated with, and explain in which situations one might be preferred over the other.

3. Explain the concept of Information Gain in decision trees. Why do we choose the feature with the highest information gain at each node? Using a concrete example, describe what it means for a feature to have an information gain of zero.

4. What is overfitting in the context of decision trees, and why are fully grown trees particularly prone to it? Describe two pruning strategies (pre-pruning and post-pruning) and explain how each addresses the overfitting problem.

5. Explain the bias–variance trade-off as it applies to decision trees. What happens to bias and variance as tree depth increases? How does the concept of an optimal tree depth relate to minimising total prediction error?

6. What is a regression tree, and how does it differ from a classification tree in terms of splitting criterion and leaf node output? Using the house price example from the chapter, explain why minimising variance is an appropriate splitting criterion for regression tasks.

7. Describe the Random Forest algorithm in detail. Explain the roles of (a) bootstrap aggregating and (b) random feature selection, and explain why combining these two techniques reduces variance compared to a single decision tree.

8. State Bayes' Theorem and identify each of its four components: posterior, likelihood, prior, and evidence. Explain why the evidence term can be ignored when comparing posterior probabilities across classes, and reformulate the Bayes classifier decision rule accordingly.

9. Define the Bayes Error Rate and explain why it is considered irreducible. Using the fruit classification example from the chapter (apples and oranges at

150g), illustrate how unavoidable class overlap gives rise to a non-zero Bayes error rate.

10. Explain the naive assumption that underlies the Naive Bayes classifier. Why is this assumption considered unrealistic in practice, yet why does Naive Bayes still achieve good performance in many applications such as spam filtering and text classification?

11. What is the zero-frequency problem in Naive Bayes, and why does it cause the entire posterior probability to collapse to zero? Explain how Laplace smoothing (additive smoothing) resolves this problem and write the smoothed likelihood formula, defining all terms.

12. Compare decision trees and Naive Bayes classifiers across the following dimensions: interpretability, computational cost of training and prediction, handling of feature dependencies, sensitivity to irrelevant features, and performance with small training data. For each dimension, state which classifier is generally preferred and justify your answer.

Section B — Analytical Problems

13. A dataset for predicting customer churn contains 20 samples at a node: 12 labelled "Churn" and 8 labelled "Stay".

(a) (a) Calculate the Entropy at this node. Show all working.

(b) (b) Calculate the Gini Index at this node. Show all working.

(c) (c) If a split on feature "Contract Type" produces two child nodes — Child 1: 10 samples (9 Churn, 1 Stay) and Child 2: 10 samples (3 Churn, 7 Stay) — calculate the Information Gain for this split using Entropy. Is this a good split? Justify your answer.

14. A regression tree is being built to predict student exam scores (out of 100) based on hours of study. A candidate split on "hours > 4" produces the following:

Region	Samples	Scores
Left (hours ≤ 4)	5	40, 50, 45, 55, 48
Right (hours > 4)	5	70, 75, 80, 68, 77

(d) (a) Compute the mean prediction for each child node.

(e) (b) Calculate the variance at the parent node (all 10 samples combined) and at each child node.

(f) (c) Calculate the variance reduction (ΔVar) achieved by this split. Would you select this split? Why?

15. Apply Bayes' Theorem to the following medical testing scenario. A rare genetic disorder affects 0.5% of the population. A diagnostic test has a sensitivity of 98% (true positive rate) and a specificity of 95% (true negative rate).

(g) (a) What is the false positive rate of the test?

(h) (b) A patient tests positive. Calculate P(Disorder | Positive) using Bayes' Theorem. Show all steps.

(i) (c) Comment on your result. Why might a patient or clinician find the posterior probability surprising? What role does the prior probability play?

16. A Naive Bayes classifier is trained on 80 emails: 50 spam and 30 ham. The table below shows word counts for three features:

Feature	Spam (50)	Ham (30)
"win" = Yes	40	3
"win" = No	10	27
"invoice" = Yes	5	22
"invoice" = No	45	8
"click" = Yes	38	6
"click" = No	12	24

A new email contains the words "win" and "click" but not "invoice". Classify this email as Spam or Ham using Naive Bayes. Show all probability calculations.

17. Using the Random Forest framework, suppose you have a dataset with d = 16 features and are building a forest of B = 5 trees for a classification problem.

(j) (a) How many features should be randomly considered at each split? State the standard rule.

(k) (b) Suppose the 5 trees vote as follows for a new test instance: Tree 1: Class A, Tree 2: Class B, Tree 3: Class A, Tree 4: Class A, Tree 5: Class B. What is the final Random Forest prediction? What is the confidence of this prediction?

(l) (c) Explain why decorrelating trees through random feature selection leads to a lower ensemble variance. Use the formula Var(average of B trees) = σ^2/B for independent trees as a starting point, and explain what happens when trees are correlated.

18. Consider a 3-class Bayes classifier for flower species (Setosa, Versicolor, Virginica). For a query flower with petal length = 5.0 cm and petal width = 1.8 cm, the posteriors are computed as:

Class	Posterior Probability
Setosa	0.03
Versicolor	0.31
Virginica	0.66

(m) (a) What class does the Bayes classifier predict? State the decision rule used.

(n) (b) What is the Bayes error rate at this point in the feature space?

(o) (c) If the true class of this flower is Virginica, does the Bayes classifier make the correct prediction? Is it possible for the Bayes classifier to make an error here, and if so, why?

Section C — Applied and Design Problems

19. A hospital wants to build a classifier to predict whether a patient is at High Risk or Low Risk for a heart attack, based on features including age, cholesterol level, blood pressure, BMI, and smoking status (binary). Design a complete decision tree pipeline for this application:

(p) (a) Which impurity measure would you choose Entropy or Gini Index and why? Does the choice significantly affect the resulting tree?

(q) (b) The fully grown tree achieves 99% training accuracy but only 74% test accuracy. Diagnose the problem and propose two specific remedies, explaining the mechanism by which each remedy improves generalisation.

(r) (c) A hospital administrator requires that the model's reasoning be explainable to non-technical doctors. Explain how a decision tree satisfies this requirement. Compare this to a Random Forest with 200 trees from the perspective of interpretability.

(s) (d) Describe how you would use cross-validation to select the optimal maximum tree depth from the candidate values {3, 5, 7, 10, unlimited}.

20. Design a Naive Bayes spam filter for an email system with a vocabulary of 10,000 unique words. The training corpus contains 5,000 spam emails and 3,000 ham emails.

(t) (a) Describe how you would estimate the prior probabilities P(Spam) and P(Ham) from the training data.

(u) (b) Explain what the independence assumption means in this context. Give a realistic example of two words in an email whose co-occurrence violates this assumption. Does this violation necessarily prevent the classifier from working well? Why or why not?

(v)(c) A new word "cryptocurrency" appears in a test email but never occurred in the training corpus. Without Laplace smoothing, what happens to the posterior probability of Spam? Apply Laplace smoothing ($\alpha = 1$) to compute a non-zero probability for this word given the Spam class. Assume $V = 2$ (binary presence/absence feature).

(w) (d) The spam filter achieves 96% accuracy on a balanced test set. However, 4% of legitimate emails are incorrectly flagged as spam. From a user-experience perspective, is accuracy the right metric to optimise? Which metric would you use instead and why?

21. A financial services company wishes to build an ensemble model to predict loan default (Yes/No) based on 25 features including income, credit score, debt-to-income ratio, employment length, and loan amount. Compare using a single deep decision tree versus a Random Forest of 500 trees for this application.

(x) (a) Explain the out-of-bag (OOB) error estimate in Random Forests. How does it provide a validation error estimate without requiring a separate validation set? What fraction of training samples are excluded from each bootstrap sample on average?

(y) (b) Random Forests provide a feature importance measure based on how much each feature reduces impurity across all trees. How would you use this feature importance to identify the top three most predictive features for loan default? Why might this be useful for regulatory compliance in lending?

(z) (c) The bank's regulator requires that each individual loan decision must be fully explainable. Discuss whether a Random Forest with 500 trees satisfies this requirement. Propose a strategy that uses Random Forest insights while maintaining decision-level interpretability.

(aa) (d) Using the bias–variance framework, explain why a Random Forest with 500 fully grown trees is unlikely to overfit, even though each individual tree is fully grown and has low bias and high variance.

22. A medical research team is building a multi-class Naive Bayes classifier to diagnose one of four conditions (Flu, COVID-19, Allergy, Bacterial Infection) based on 12 binary symptoms (fever, cough, fatigue, etc.).

(bb) (a) Without the naive independence assumption, how many probability values would need to be estimated per class for 12 binary features? With the assumption, how many values are required per class? Show your calculations and comment on the practical significance of this reduction.

(cc) (b) Describe how you would train the multi-class Naive Bayes classifier. Specifically, write out the general formula for computing the posterior probability for each of the four classes and explain how the final diagnosis is determined.

(dd) (c) A patient presents with fever = Yes, cough = Yes, fatigue = Yes, and all other symptoms = No. Suppose the computed (un-normalised) posteriors are: Flu: 0.042, COVID-19: 0.038, Allergy: 0.003, Bacterial: 0.017. What is the diagnosis? Compute the normalised posterior probability for each class.

(ee) (d) Discuss two limitations of Naive Bayes that are particularly relevant in a clinical diagnosis setting. For each limitation, suggest a modification or alternative approach that addresses it.

CHAPTER IV

Linear Discriminants for Machine Learning

Chapter Overview

Chapter Introduces the family of linear discriminant models the foundational supervised learning methods that use a linear function of input features to make classification and regression decisions. Starting from the simplest geometric concept of a separating hyperplane, the chapter builds systematically toward increasingly powerful and practically important algorithms: the Perceptron, Support Vector Machines, Logistic Regression, Linear Regression, Multi-Layer Perceptron, and Backpropagation. Throughout, the emphasis is on mathematical rigor, geometric intuition, and worked numerical examples that make abstract ideas concrete.

The chapter opens by defining a linear discriminant as a weighted combination of features $f(x) = w^T x + b$, where the weight vector w determines the orientation of the decision boundary and the bias b controls its offset from the origin. The geometric interpretation that w is perpendicular to the hyperplane, that the signed distance from any point to the boundary equals $f(x)/\|w\|$, and that the sign of f(x) determines class membership is developed carefully. The binary classification rule is then extended to multi-class settings via One-vs-All and One-vs-One strategies.

The Perceptron, invented by Rosenblatt in 1958, is introduced as the oldest and simplest learning algorithm for finding a linear separator. Its update rule adding the misclassified point's feature vector (scaled by its label and the learning rate) to the weight vector has an elegant geometric interpretation as rotating the decision boundary toward correct classification. The Perceptron Convergence Theorem provides a finite bound on the number of updates needed when the data is linearly separable, but the algorithm's critical limitation inability to solve non-linearly separable problems such as XOR motivates the algorithms that follow.

Support Vector Machines (SVMs) advance beyond the Perceptron by finding not just any separating hyperplane but the unique maximum-margin hyperplane. The hard-margin formulation and its convex quadratic optimization dual are presented alongside the key concepts of support vectors, margin width ($2/\|w\|$), and margin boundaries. The soft-margin extension introduces slack variables ξ_i

and the regularization parameter C to handle non-separable data, trading off margin width against training error. Non-linear SVMs resolve the limitation of linear boundaries by mapping input data to a high-dimensional feature space via a feature map φ, and the kernel trick computing inner products $K(x, z) = \varphi(x)^T\varphi(z)$ directly without forming φ explicitly — makes this computationally feasible. The four standard kernels (Linear, Polynomial, RBF/Gaussian, Sigmoid) are presented with formulas and use-case guidance.

Logistic Regression brings a probabilistic perspective: rather than a hard binary output, it outputs the probability of class membership via the sigmoid (logistic) function $\sigma(z) = 1/(1+e^{-z})$. Training by minimizing cross-entropy loss (equivalent to maximum likelihood estimation) yields well-calibrated probability estimates and smooth gradients suitable for gradient-based optimization. Linear Regression then extends the discriminant framework to continuous target prediction, finding the best-fit hyperplane under the Ordinary Least Squares (OLS) criterion, with a closed-form normal equation solution and regularization variants (Ridge, Lasso, Elastic Net).

The chapter concludes with Multi-Layer Perceptron (MLPs) and Backpropagation, bridging classical linear discriminants to modern deep learning. MLPs overcome the linear separability limitation by stacking layers of neurons with non-linear activations (Sigmoid, Tanh, ReLU, SoftMax), creating universal function approximators. The XOR example demonstrates concretely how two neurons computing OR and AND can be combined to produce XOR. Backpropagation — a systematic application of the chain rule to compute gradients layer by layer enables efficient training. The chapter ends with practical training techniques: mini-batch gradient descent, learning rate scheduling, momentum, the Adam optimizer, AdaGrad, Nesterov Accelerated Gradient, RMSprop, weight initialization (Xavier/He), and batch normalization.

Learning Objectives	By the end of this chapter, the reader will be able to: (1) define and geometrically interpret a linear discriminant function; (2) implement and trace the Perceptron Learning Algorithm and prove its convergence; (3) formulate and solve the SVM optimization problem in both hard-margin and soft-margin settings; (4) apply the kernel trick with standard kernels to handle non-linear classification;

	(5) train a logistic regression model using cross-entropy loss and interpret its probability outputs; (6) fit a linear regression model using OLS and apply Ridge/Lasso regularization; (7) design and perform forward propagation through a multi-layer perceptron; and (8) compute gradients via backpropagation and apply modern optimization algorithms.

Section-by-Section Roadmap

Section	Topic	Key Concepts
4.1	Introduction to Linear Discriminants	Decision function $f(x)=w^Tx+b$, hyperplane geometry, weight vector, bias, signed distance, binary classification rule
4.2	Linear Discriminants for Classification	Binary classification constraint, linear separability, One-vs-All, One-vs-One multi-class strategies
4.3	Perceptron Classifier	Perceptron definition, sign activation, AND/OR gate examples, linear separability limitation, XOR failure
4.4	Perceptron Learning Algorithm	Update rule, learning rate η, convergence theorem, step-by-step training trace, epoch iterations
4.5	Support Vector Machines	Margin maximization, support vectors, hard-margin formulation, convex QP, margin width $2/\|\|w\|\|$
4.6	Linearly Non-Separable Case	Soft-margin SVM, slack variables ξ_i, regularization parameter C, overfitting vs. underfitting trade-off

4.7	Non-linear SVM	Feature mapping φ, lifting to high-dimensional space, XOR solved in 3D, implicit feature spaces
4.8	Kernel Trick	Kernel function $K(x,z)=\varphi(x)^T\varphi(z)$, Linear/Polynomial/RBF/Sigmoid kernels, RBF bandwidth σ
4.9	Logistic Regression	Sigmoid function, probability output, cross-entropy loss, MLE training, threshold-based prediction
4.10	Linear Regression	OLS objective, normal equation, Ridge/L2, Lasso/L1, Elastic Net, house price example
4.11	Multi-Layer Perceptrons	Architecture (input/hidden/output), non-linear activations, forward propagation, XOR solution, Universal Approximation Theorem
4.12	Backpropagation	Chain rule, forward/backward pass, error signal δ, weight/bias gradients, Adam, momentum, batch normalization

4.1 Introduction to Linear Discriminants

A linear discriminant is one of the fundamental techniques in machine learning for classification tasks. It works by using a linear combination of input features to separate different classes of data. The core idea is remarkably simple yet powerful: we construct a linear function that takes the input features and produces a single value, and then we use this value to determine which class the input belongs to.

In two dimensions, the decision boundary created by a linear discriminant is a straight line that divides the feature space into two regions, one for each class. When we move to three dimensions, this boundary becomes a flat plane that separates the 3D space. In general, for d-dimensional data, the decision boundary is a (d − 1)-dimensional hyper plane. This **hyperplane** acts as the separator between different classes, and any new data point is classified based on which side of the hyperplane it falls.

Definition 4.1.1 (Linear Discriminant)

A linear discriminant is a function of the form:

$$f(\mathrm{x}) = \mathrm{w}^T \mathrm{x} + b = w_1 x_1 + w_2 x_2 + \cdots + w_d x_d + b \quad (4.1)$$

where $\mathbf{w} = (w_1, w_2, \ldots, w_d)^T$ is called the **weight vector** which determines the orientation of the decision boundary, b is the **bias** (also called the intercept or threshold) which determines the position of the boundary, and $\mathrm{x} = (x_1, x_2, \ldots, x_d)^T$ is the input feature vector that we want to classify.

The weight vector w plays a crucial role in determining how important each feature is for the classification decision. A larger weight magnitude for a particular feature means that feature has more influence on the final classification. The sign of each weight determines whether higher values of that feature push the classification toward one class or the other.

Classification Rule: Once we compute the value of the discriminant function $f(\mathrm{x})$, we apply a simple decision rule:

- If $f(\mathrm{x}) > 0$: The point lies on the positive side of the hyperplane, so we predict **Class 1** (or Class +1)
- If $f(\mathrm{x}) < 0$: The point lies on the negative side of the hyperplane, so we predict **Class 2** (or Class -1)

- If $f(x) = 0$: The point lies exactly on the decision boundary, which is the equation $w^T x + b = 0$

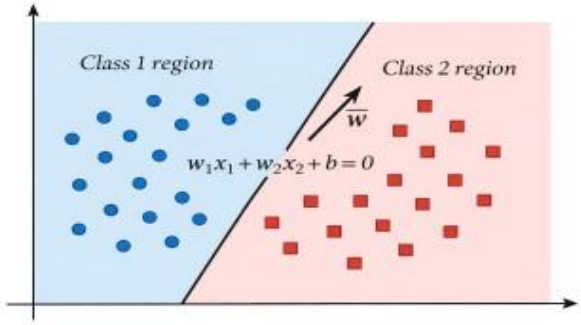


Figure 4.1: Linear Decision Boundary for Binary Classification

Figure 4.1: Linear Decision Boundary for Binary Classi cation. The weight vector w is perpendicular to the decision boundary and points toward the positive class region.

Geometric Interpretation:

Understanding the geometry behind linear discriminants is essential for building intuition about how they work:

- The weight vector w is always perpendicular (orthogonal) to the decision boundary hyperplane. This means w acts like an arrow pointing from the Class 2 region toward the Class 1 region.
- The bias term b determines the o set of the decision boundary from the origin. If $b = 0$, the hyperplane passes through the origin. A positive b shifts the boundary in the negative w direction, and a negative b shifts it in the positive w direction.
- The signed distance from any point x to the decision boundary can be calculated as $\frac{f(\mathbf{x})}{\|\mathbf{w}\|}$, where ‖w‖ is the Euclidean length of the weight vector. Points further from the boundary have larger absolute distance values.
- The magnitude of $f(x)$ indicates how confident we are about the classification.

A large positive or negative value suggests the point is far from the decision boundary, making it a more certain classification.

Example 4.1.1 (2D Linear Discriminant) Task: Given a linear discriminant function, classify points as belonging to Class A or Class B.

Linear discriminant function: $f(x_1, x_2) = 2x_1 + 3x_2 - 6$

In this function, the weight vector is $w = (2,3)^T$, which tells us that both features contribute positively to the discriminant. The feature x_2 has a slightly higher weight (3 vs 2), meaning it has more influence on the classification. The bias is $b = -6$, which shifts the decision boundary away from the origin.

Decision boundary (the line where $f = 0$): Setting $f(x_1, x_2) = 0$ gives us $2x_1 + 3x_2 = 6$, which can be rewritten as $x_2 = -(2/3)x_1 + 2$. This is a line with slope $-2/3$ and y-intercept

Of 2.

Classifying point (1,2):

$f(1,2) = 2(1) + 3(2) - 6 = 2 + 6 - 6 = 2 > 0 \Rightarrow$ Class A

Since the discriminant value is positive (+2), this point lies on the positive side of the boundary and is classified as Class A. The distance from the boundary is

$$\frac{2}{\sqrt{2^2+3^2}} = \frac{2}{\sqrt{13}} \approx 0.55 \text{ units.}$$

Classifying point (0,1):

$f(0,1) = 2(0) + 3(1) - 6 = 0 + 3 - 6 = -3 < 0 \Rightarrow$ Class B

Since the discriminant value is negative (−3), this point lies on the negative side of the boundary and is classified as Class B. This point is $3/\sqrt{13} \approx 0.83$ units away from the boundary.

Example 4.1.2 One-Dimensional Linear Discriminant

Problem: Classify data points in one dimension using a linear decision boundary (thresh old).

Training Data:

Feature (x)	Class (y)
2	0 (Negative)
3	0 (Negative)
7	1 (Positive)
8	1 (Positive)

Finding the Decision Boundary:

For a one-dimensional problem, the decision boundary is simply a threshold value θ.

We classify a point x as:

$$\hat{y} = \begin{cases} 1 & \text{if } x \geq \theta \\ 0 & \text{if } x < \theta \end{cases} \qquad (4.1)$$

From the training data, we observe that examples with x ∈ [2, 3] belong to class 0, and examples with x ∈ [7, 8] belong to class 1. A reasonable threshold would be $\theta = 5$.

Classification: For the threshold $\theta = 5$:

Class(x = 4) = 0 (below threshold) Class(x = 7.5) = 1 (above threshold) (4.2)

Decision Rule: The linear discriminant function is:

$$f(x) = w \cdot x + b = 1 \cdot x - 5 \qquad (4.3)$$

where we assign class 1 if $f(x) \geq 0$, otherwise class 0.

Example 4.1.3 Two-Dimensional Linear Discriminant

Problem: Classify 2D data points using a linear decision boundary (line in 2D space).

Training Data:

Feature x_1	Feature x_2	Class (y)
1	1	0 (Negative)
2	1	0 (Negative)
6	5	1 (Positive)
7	5	1 (Positive)

Linear Decision Boundary:

In 2D, the decision boundary is a line defined by:

$$w_1x_1 + w_2x_2 + b = 0 \qquad (4.4)$$

The linear discriminant function is:

$$f(x) = w_1x_1 + w_2x_2 + b = 1 \cdot x_1 + 1 \cdot x_2 - 4 \quad (4.5)$$

Classification Rule:

$$\hat{y} = \begin{cases} 1 & \text{if } f(\mathbf{x}) = x_1 + x_2 - 4 \geq 0 \\ 0 & \text{if } f(\mathbf{x}) < 0 \end{cases} \quad (4.6)$$

Example Classifications

$$:f(1,1) = 1 + 1 - 4 = -2 < 0 \Rightarrow \text{Class } 0 \quad (4.7)$$

$$f(6, 5) = 6 + 5 - 4 = 7 \geq 0 \Rightarrow \text{Class } 1 \quad (4.8)$$

The decision boundary line passes through points where $x_1 + x_2 = 4$. Points above this line are classified as positive (class 1), and points below are classified as negative (class 0).

Why Use Linear Discriminants?

Linear discriminants are widely used in practice for several compelling reasons:

- **Simplicity and Interpretability**: The classification decision can be easily understood by examining the weights. A large positive weight for a feature means that feature strongly supports one class, making it easy to explain predictions to stakeholders.
- **Computational Efficiency**: Both training and prediction are extremely fast since they involve only basic linear algebra operations. This makes linear discriminants suitable for real-time applications and large datasets.
- **Effectiveness for Linearly Separable Data:** When classes can be separated by a hyperplane (even approximately), linear discriminants often achieve excellent performance with minimal computational cost.
- **Foundation for Advanced Methods:** Linear discriminants form the building blocks for more sophisticated algorithms like Support Vector Machines, Logistic Regression, and even Neural Networks. Understanding them well provides a solid foundation for learning these more complex methods.
- **Low Risk of Overfitting:** Due to their simplicity (limited number of parameters), linear discriminants are less prone to overfitting compared to more complex models, especially when training data is limited.

4.2 Linear Discriminants for Classification

Linear discriminant analysis is the systematic approach to using linear functions to separate multiple classes in a dataset. The fundamental challenge in classification is finding the optimal values for the weight vector w and bias b that will correctly separate the training data and generalize well to new, unseen data points.

When we have labeled training data, our goal is to find a decision boundary that places each training point on the correct side. For a binary classification problem, we want points from one class to have positive discriminant values and points from the other class to have negative values. The magnitude of these values represents how far each point is from the decision boundary.

Definition 4.2.1 (Binary Linear Classification)

Given a training dataset consisting of n examples $\{(x_i, y_i)\}^n_{i=1}$ where each x_i is a feature vector and $y_i \in \{+1, -1\}$ is the corresponding class label, the goal is to nd weights w and bias b such that:

$$y_i \cdot (w^T x_i + b) > 0 \text{ for all training examples } i = 1, 2, \ldots, n \qquad (4.4)$$

This constraint means that for each point, the product of its true label and its discriminant value should be positive. When $y_i = +1$, we need $f(x_i) > 0$, and when $y_i = -1$, we need $f(x_i) < 0$. This ensures every training point is correctly classified.

Different Approaches to Finding the Optimal Discriminant: Over the years, researchers have developed several methods for finding good values of w and b, each with its own advantages:

1. **Perceptron Algorithm:** This is one of the oldest and simplest approaches. It iteratively examines training points and updates the weights whenever a point is misclassified. The perceptron is guaranteed to converge if the data is linearly separable, but it may find any valid separator, not necessarily the best one.
2. **Support Vector Machines (SVM):** This approach finds the separator that maximizes the margin, the distance between the decision boundary and the nearest training points. This often leads to better generalization because the boundary is as far as possible from all training points.

3. **Logistic Regression:** Instead of finding a hard boundary, logistic regression models the probability that a point belongs to each class. It uses maximum likelihood estimation to find weights that make the observed class labels most probable.

4. **Fisher's Linear Discriminant Analysis (LDA):** This approach projects the data onto a lower-dimensional space in a way that maximizes the separation between class means while minimizing the spread within each class. It has connections to Bayesian classification under certain assumptions about the data distribution.

Example 4.2.1 (Checking if Data is Linearly Separable) Given Training Data:

x_1	x_2	Class
1	2	+1
2	3	+1
3	1	-1
4	2	-1

Question: Can we find a straight line that perfectly separates the two classes?

Analysis: Let's examine the data to understand the class distribution. The Class +1 points are (1, 2) and (2, 3), which have smaller x_1 values (1 and 2). The Class -1 points are (3, 1) and (4, 2), which have larger x_1 values (3 and 4). This suggests that the x_1 feature is a good discriminator; points with smaller x_1 tend to be Class +1, while points with larger x_1 tend to be Class -1.

Finding a Separator: Based on our analysis, a simple separator using only x_1 might work. Consider the discriminant: $f(x_1, x_2) = -x_1 + 2.5$. This creates a vertical decision boundary at $x_1 = 2.5$.

Verification:

- For (1, 2) with Class +1: $f(1, 2) = -1 + 2.5 = 1.5 > 0$ Correctly classified as +1
- For (2, 3) with Class +1: $f(2, 3) = -2 + 2.5 = 0.5 > 0$ Correctly classified as +1
- For (3, 1) with Class -1: $f(3, 1) = -3 + 2.5 = -0.5 < 0$ Correctly classified as -1
- For (4, 2) with Class -1: $f(4, 2) = -4 + 2.5 = -1.5 < 0$ Correctly classified as -1

Conclusion: Yes, this data is linearly separable! The line $x_1 = 2.5$ successfully separates all Class +1 points from all Class -1 points. Note that this is not the only valid separator; there are infinitely many lines that could work, and different algorithms will and different ones.

Extending to Multi-Class Problems: When we have more than two classes, we need to extend our binary classification approach. There are two common strategies:

- One-vs-All (OvA) Strategy: For K classes, we train K separate binary classifiers. Each classifier learns to distinguish one class from all other classes combined. During prediction, we run all K classifiers and assign the class whose classifier gives the highest confidence score. This approach is efficient and works well when classes are well-separated.
- **One-vs-One (OvO) Strategy:** For K classes, we train $\binom{K}{2} = \frac{K(K-1)}{2}$ binary classifiers, one for each possible pair of classes. During prediction, each classifier votes for one of its two classes, and we assign the class that receives the most votes. This approach can work better when classes have complex relationships but requires training more classifiers.

4.3 Perceptron Classifier

The perceptron is historically significant as the first algorithm that could automatically learn from data to perform classification. Invented by Frank Rosenblatt in 1958, it represents the simplest form of an artificial neural network, a single artificial neuron that performs binary classification. Despite its simplicity, the perceptron laid the groundwork for the entire field of neural networks and deep learning that we see today.

The perceptron is inspired by biological neurons in the brain. Just as a biological neuron receives signals from other neurons, sums them up, and res if the total signal exceeds a threshold, the perceptron receives input features, computes a weighted sum, and produces an output based on whether this sum exceeds a threshold.

Definition 4.3.1: Perceptron

A **perceptron** is a binary classifier that computes the weighted sum of its input features and applies a step function (also called a sign or threshold function) to produce the output

$$\hat{y} = \text{sign}\left(\sum_{j=1}^{d} w_j x_j + b\right) = \text{sign}(\mathbf{w}^T\mathbf{x} + b)$$

.

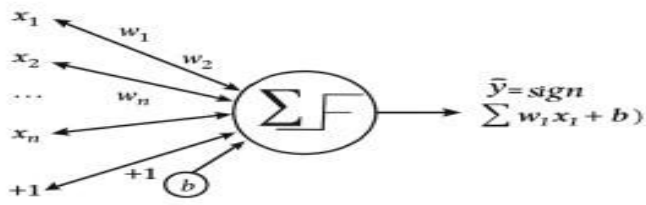


Figure 4.2: Perceptron Model Structure

Figure 4.2: Perceptron Model Structure showing inputs $x_1, x_2, \ldots, x_n$ with corresponding weights, a bias term, and the activation function that produces the output.

Components of a Perceptron: Understanding each component helps us see how the perceptron makes decisions:

- **Inputs** ($x_1, x_2, \ldots, x_d$): These are the feature values that describe the data point we want to classify. For example, if classifying emails as spam or not spam, inputs might include word frequencies, email length, number of links, etc.
- **Weights** ($w_1, w_2, \ldots, w_d$): Each input has an associated weight that determines how much in sequence that feature has on the nal decision. Weights are the learnable parameters they are adjusted during training to improve classification accuracy. A large positive weight means that feature strongly supports Class +1, while a large negative weight supports Class -1.
- **Bias** (b): The bias acts as a threshold for activation. It allows the decision boundary to be shifted away from the origin. Without a bias term, the decision boundary would always have to pass through the origin, which is too restrictive for most real-world problems.
- **Weighted Sum**: The perceptron computes $z = w^T x + b$, which represents how strongly the inputs, weighted by their importance, support the positive class.
- **Activation Function:** The step function (sign function) makes the nal decision. If the weighted sum is non-negative, output +1; otherwise, output -1.

  This is a hard decision the perceptron commits fully to one class or the other.
- **Output** ($\hat{y}$): The predicted class label, either +1 or -1.

Example 4.3.1 (Perceptron for AND Gate) Task: Design a perceptron that computes the logical AND function of two binary inputs.

The AND function returns 1 only when both inputs are 1, and returns 0 otherwise. Let's see if we can find weights that make a perceptron compute this function. Truth table for AND:

x_1	x_2	x_1 *AND* x_2
0	0	0
0	1	0
1	0	0
1	1	1

Perceptron weights: After some analysis (or learning), we find that $w_1 = 1$, $w_2 = 1$, and $b = -1.5$ work. The idea is that both inputs need to contribute positively, and together they need to overcome the threshold of 1.5.

Verification: Let's check each input combination:

$f(0, 0) = 1(0) + 1(0) - 1.5 = -1.5 < 0 \Longrightarrow$ output 0 ✓ (4.6)

$f(0, 1) = 1(0) + 1(1) - 1.5 = -0.5 < 0 \Longrightarrow$ output 0 ✓ (4.7)

$f(1, 0) = 1(1) + 1(0) - 1.5 = -0.5 < 0 \Longrightarrow$ output 0 ✓ (4.8)

$f(1, 1) = 1(1) + 1\,f(1,1) - 1.5 = 0.5 > 0 \Longrightarrow$ output 1 ✓ (4.9)

The perceptron successfully learns the AND function! The decision boundary is the line $x_1 + x_2 = 1.5$, which separates the point (1, 1) from the other three points.

Example 4.3.2 (Perceptron for OR Gate)Task: Design a perceptron that computes the logical OR Gate function of two binary inputs**.**

***x*1**	***x*2**	**OR**
0	0	0
0	1	0
1	0	0
1	1	1

1. The OR function returns **1** if *at least one input is 1*, and returns **0 only when both inputs are 0.**

We want to determine weights and bias such that a **Perceptron** correctly computes this function.

Truth Table: **Perceptron weights**

After analysis (or training), one valid solution is:

- w1=1
- w2=1
- b= -0.5

For OR, *any one input should be enough* to cross the threshold.

So the threshold must be **low enough** that a single "1" activates the neuron.

Verification: We compute: **f(x1,x2)=w1x1+w2x2+b**

Activation rule:

- Output = 1 if f ≥ 0

Output = 0 if f < 0 f(0,0)=1(0)+1(0)−0.5=−0.5<0⇒output 0 ✓ *(4.10)*

f(0,1)=1(0)+1(1)−0.5=0.5≥0⇒output 1 ✓ *(4.11)*

f(1,0)=1(1)+1(0)−0.5=0.5≥0⇒output 1 ✓ *(4.12)*

f(1,1)=1(1)+1(1)−0.5=1.5≥0⇒output 1 ✓ *(4.13)*

The **Perceptron** successfully learns the **OR Gate** function. The decision boundary is: x1+x2=0.5 This boundary separates the point (0,0) from the other three input combinations, correctly classifying the OR logic.

Key Difference from AND Gate: The OR gate requires a lower threshold (−0.5 instead of −1.5), reflecting that it outputs 1 when *at least one* input is 1, whereas AND requires *both* inputs to be 1.

Fundamental Limitation of the Perceptron: Despite its elegance, the perceptron has a critical limitation: it can only learn to classify data that is linearly separable. This means there must exist a hyperplane that perfectly separates the two classes. The most famous example of a function the perceptron cannot learn is the XOR (exclusive OR) function, because the points where XOR equals 1 (namely (0, 1) and (1, 0)) and the points where it equals 0 (namely (0, 0) and (1, 1)) cannot be separated by any straight line. This limitation, pointed out by Minsky and Papert in 1969, temporarily slowed research in neural networks until the development of multi-layer networks that could overcome this constraint.

4.4 Perceptron Learning Algorithm

The Perceptron Learning Algorithm is a simple yet powerful iterative method for finding weights that correctly classify all training points. The beauty of this algorithm is that it learns from its mistakes whenever it misclassifies a training point, it adjusts its weights in a direction that would help classify that point correctly in the future.

The algorithm is remarkably intuitive. Starting with zero weights (no knowledge), it examines each training point one by one. If a point is correctly classified, no action is needed. But if a point is misclassified, the algorithm updates the weights to push the decision boundary in a direction that would x this mistake. By repeatedly cycling through the training data and correcting mistakes, the algorithm gradually finds a separator (if one exists).

Definition 4.4.1 (Perceptron Learning Algorithm) The algorithm proceeds as follows:

1. **Initialize**: Set all weights to zero: w = 0 and bias $b = 0$. At this point, the perceptron has no knowledge and classifies everything the same way.
2. **Iterate through training data:** For each training example (x_i, y_i):

- Compute the prediction: $\hat{y}_i = \text{sign}(w^T x_i + b)$
- If $\hat{y}_i \neq y_i$ (the point is misclassi ed), update the weights:

$$w \leftarrow w + \eta \cdot y_i \cdot x_i \quad (4.14)$$

$$b \leftarrow b + \eta \cdot y_i \quad (4.15)$$

where $\eta > 0$ is the learning rate, a parameter that controls how much we adjust the weights for each mistake.

3. **Repeat:** Continue iterating through the training data until all points are correctly classified, or until a maximum number of iterations is reached. The learning rate η is often set to 1 for simplicity, which is sufficient for finding a separator if one exists.

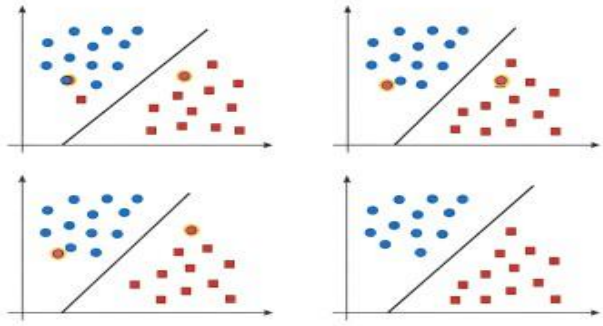


Figure 4.3: Perceptron Learning Algorithm Visualized

Figure 4.3: Perceptron Learning Algorithm Visualized:The decision boundary adjusts iteratively as misclassified points (highlighted) trigger weight updates, eventually converging to a correct separator.

Understanding the Update Rule: The update rule $w \leftarrow w + \eta \cdot y_i \cdot x_i$ has an elegant geometric interpretation. When we misclassify a positive point ($y_i = +1$, but $\hat{y}_i = -1$), adding x_i to w rotates the weight vector toward x_i, making it more likely to be classi ed correctly next time. Conversely, when we misclassify a negative point ($y_i = -1$, but $\hat{y}_i = +1$), subtracting x_i from w rotates the weight vector away from x_i.

Example 4.4.1 (Perceptron Learning Step-by-Step) Training data: We have three points:

(1, 1) with label +1, (2, 0) with label +1, and (0, 1) with label −1.

Initialize: Start with $w_1 = 0$, $w_2 = 0$, $b = 0$, and learning rate $\eta = 1$. With these weights, the perceptron has no knowledge yet it will classify all points based solely on the bias, which is zero.

Epoch 1 (rst pass through all training points):

point (1, 1), true label $y = +1$:

Compute: $f = 0(1) + 0(1) + 0 = 0 \geq 0$, so $\hat{y} = +1$.

Since $\hat{y} = y$, this point is correctly classi ed. No update needed.

- **Point (2, 0), true label** $y = +1$:

Compute: $f = 0(2) + 0(0) + 0 = 0 \geq 0$, so $\hat{y} = +1$.

Correct again! No update needed.

- **Point (0, 1), true label** $y = -1$:

Compute: $f = 0(0) + 0(1) + 0 = 0 \geq 0$, so $\hat{y} = +1$.

Misclassified! The true label is −1 but we predicted +1.

Update weights: $w_1 = 0 + (-1)(0) = 0$, $w_2 = 0 + (-1)(1) = -1$, $b = 0 + (-1) = -1$

Epoch 2 (second pass, using updated weights $w_1 = 0$, $w_2 = -1$, $b = -1$):

- **Point (1, 1), true label** $y = +1$:

  Compute: $f = 0(1) + (-1)(1) + (-1) = -2 < 0$, so $\hat{y} = -1$.

  Misclassified! We predicted −1 but the true label is +1.

  Update: $w_1 = 0 + 1(1) = 1$, $w_2 = -1 + 1(1) = 0$, $b = -1 + 1 = 0$

The algorithm continues in this manner, adjusting weights whenever a mistake is made. Eventually, it will converge to weights that correctly classify all three points.

Perceptron Convergence Theorem: One of the most important theoretical results about the perceptron is that if the training data is linearly separable, the perceptron learning algorithm is **guaranteed to converge** in a nite number of steps. Specifically, if there exists a separator with margin γ (minimum distance from any point to the boundary) and all training points have norm at most R, then the algorithm will converge in at most $(R/\gamma)^2$ updates. However, if the data is not linearly separable, the algorithm will never converge and will oscillate forever, which is why more sophisticated algorithms like SVMs were developed.

4.5 Support Vector Machines

Support Vector Machines (SVMs) represent a significant advancement over the perceptron by introducing the concept of margin maximization. While the perceptron ends any hyperplane that separates the classes, SVM ends the optimal hyperplane, the one that is as far as possible from all training points. This optimal hyperplane often generalizes better to new data because it doesn't get too close to any particular training point.

The key insight behind SVM is that among all possible separating hyperplanes, the one that maximizes the distance to the nearest training points (the margin) is likely to perform best on unseen data. Imagine drawing a street between two classes SVM finds the widest possible street where no training points fall inside it.

Definition 4.5.1 (Support Vector Machine) *SVM finds the hyperplane* $w^T x + b = 0$ *that maximizes the geometric margin between the two classes:*

$$Maximize \ \frac{2}{\|\mathbf{w}\|} \quad (the\ margin\ width) \tag{4.16}$$

subject to the constraint that all training points are correctly classified ed with a
functional margin of at least 1:
$y_i(w^T x_i + b) \geq 1$ for all training points i

The margin $\frac{2}{\|\mathbf{w}\|}$ *represents the total width of the street between the two classes.*

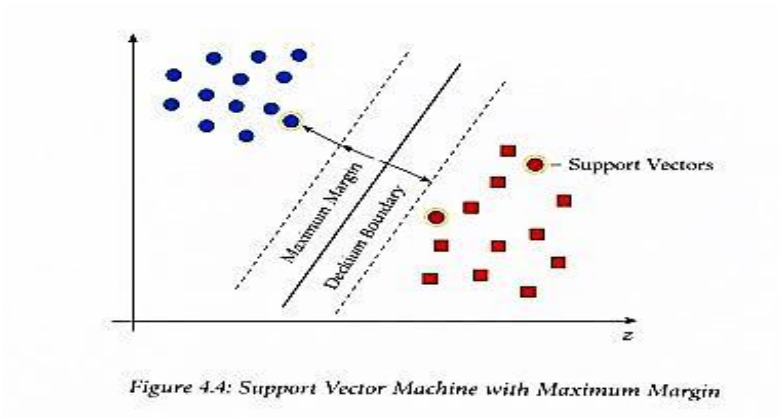


Figure 4.4: Support Vector Machine with Maximum Margin

Figure 4.4: Support Vector Machine with Maximum Margin: The optimal hyperplane (solid line) maximizes the distance to the nearest points from both classes. Support vectors are the points lying exactly on the margin boundaries (dashed lines).

Key Concepts in SVM: Understanding these concepts is essential for working with SVMs:

- Margin: The margin is the perpendicular distance from the decision boundary to the nearest training point. SVM aims to maximize this distance. A larger margin means the classifier is more confident in its decisions and has more room for error when classifying new points.

- Support Vectors: These are the training points that lie exactly on the mar gin boundary; they are at the minimum distance from the hyperplane. They are called support vectors because they support or define the optimal hyperplane. If you removed all other training points, the solution would remain the same. The support vectors are the critical points that determine the decision boundary.

- Maximum Margin Hyperplane: SVM finds the unique hyperplane that creates the widest possible separation between classes. This hyperplane lies exactly in the middle of the street defined by the support vectors from each class.

- Margin Boundaries: The two parallel hyperplanes $w^T x + b = +1$ and $w^T x + b = -1$ de ne the edges of the margin. No training points should fall inside this margin (in the hard margin case).

Why Does Maximum Margin Work Better? Maximizing the margin provides several benefits:

- **Better Generalization:** By staying as far as possible from all training points, the classifier is less sensitive to small perturbations or noise in the data, leading to better performance on new, unseen examples.

- **Robustness:** The large margin provides a safety buffer that makes the classifier more robust to measurement errors and noise in the features.

- **Unique Solution:** Unlike the perceptron which can find any separating hyperplane, SVM has a unique solution (the

  maximum margin hyperplane), which makes it more predictable and reproducible.

Example 4.5.1 (Computing SVM Margin) Given: *An SVM has found the optimal hyperplane with equation* $2x_1 + x_2 - 4 = 0$, so the weight vector is $w = (2, 1)^T$ and bias $b=-4$.

Computing the margin width: *The margin is determined by the norm of the weight vector:*

$$Margin = \frac{2}{\|\mathbf{w}\|} = \frac{2}{\sqrt{2^2 + 1^2}} = \frac{2}{\sqrt{5}} \approx 0.894$$

This means the street between the two classes has a total width of approximately 0.894 units. The decision boundary is in the middle, so each side of the margin extends about 0.447 *units from the boundary.*

Finding the distance from a specific point to the hyperplane: *Consider the point x = (3, 2). Its signed distance to the hyperplane is:*

$$Distance = \frac{|\mathbf{w}^T\mathbf{x} + b|}{\|\mathbf{w}\|} = \frac{|2(3) + 1(2) - 4|}{\sqrt{5}} = \frac{|6 + 2 - 4|}{\sqrt{5}} = \frac{4}{\sqrt{5}} \approx 1.789$$

Since this distance (1.789) is greater than half the margin width (0.447), this point lies outside the margin, which is where we want training points to be.

4.6 Linearly Non-Separable Case

In the real world, data is rarely perfectly linearly separable. There might be noise in the measurements, outliers in the data, or genuine overlap between classes where the same feature values can belong to different classes. If we insist on finding a hyperplane that perfectly separates all training points, we may either fail (if no such hyperplane exists) or end up with an overly complex decision boundary that doesn't generalize well.

To handle this reality, we extend SVM to the **soft margin** case, which allows some training points to be inside the margin or even on the wrong side of the decision boundary. The idea is to trade-off between having a large margin (which promotes generalization) and having few classification errors on the training data.

Definition 4.6.1 (Soft Margin SVM) The soft margin SVM introduces slack variables $\xi_i \geq 0$ for each training point, which measure how much each point violates the margin constraint:

$$\min_{\mathbf{w},b,\boldsymbol{\xi}} \frac{1}{2}\|\mathbf{w}\|^2 + C\sum_{i=1}^{n}\xi_i \tag{4.17}$$

subject to the relaxed constraints: $y_i(\mathbf{w}^T \mathbf{x}_i + b) \geq 1 - \xi_i$ and $\xi_i \geq 0$ for all i. The parameter $C > 0$ is called the **regularization parameter** and controls the trade-off between maximizing the margin and minimizing the total amount of constraint violation.

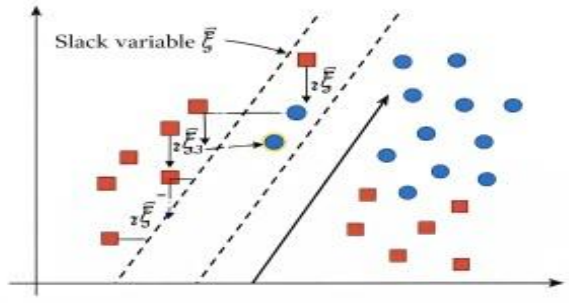


Figure 4.5: Soft Margin SVM (Linearly Non-Separable Case)

Figure 4.5: Soft Margin SVM for the Linearly Non-Separable Case: Slack variables ξ_i allow some points to be inside the margin or misclassified. The optimization balances margin width against the total slack.

Understanding Slack Variables: The slack variable ξ_i for each training point tells us how much that point violates the margin constraint:

- $\xi_i = 0$: The point is correctly classified and lies outside the margin (on the correct side at least 1 unit from the boundary). This is the ideal case.
- $0 < \xi_i < 1$: The point is correctly classified but lies inside the margin. It's on the correct side of the decision boundary, but closer than we'd like. The point is in the "street" between the margin boundaries.
- $\xi_i = 1$: The point lies exactly on the decision boundary.
- $\xi_i > 1$: The point is misclassified it's on the wrong side of the decision boundary. The amount by which ξ_i exceeds 1 indicates how far into the wrong region the point has gone.

The Role of the Regularization Parameter *C*: The parameter *C* is crucial and must be chosen carefully:

- **Large *C* (e.g., *C* = 1000):** A large value of *C* penalizes margin violations heavily. The SVM will try very hard to classify all training points correctly, even if this means having a very narrow margin. This can lead to a complex decision boundary that closely follows the training data, increasing the risk of **overfitting** (fitting the noise in the training data rather than the underlying pattern). **Small *C* (e.g., *C* = 0.01):** A small value of *C* allows more violations without incurring a large penalty. The SVM will prioritize having a wide margin, even if this means misclassifying some training points. This leads to a smoother, more generalizable boundary, but too small a *C* may result in **underfitting** (missing important patterns in the data).
- **Optimal *C*:** The best value of *C* depends on the specific c dataset and should be found using techniques like cross-validation, where we try different values and see which one performs best on held-out validation data.

Example 4.6.1 (Understanding the Effect of the C Parameter) Scenario:

Consider a dataset of 100 points that is mostly separable, but contains 5 outlier's points from one class that are located in the region typically occupied by the other class. With a large C = 1000:

- The SVM tries very hard to correctly classify all 100 points, including the 5 outliers.
- The decision boundary will twist and bend to accommodate the outliers, moving close to many correctly-positioned points.
- The resulting margin will be very narrow because the boundary has to navigate around the outliers.

- Risk: This classifier may perform poorly on new data because it has over t to the peculiarities (including noise) in the training set.

With a small C = 0.01:

- The SVM prioritizes having a wide margin and is willing to misclassify the 5 outliers.
- The decision boundary remains smooth and well-separated from the bulk of both classes.
- The margin is wide, providing good separation between the main clusters of each class.
- Risk: If C is too small, the classifier might misclassify too many points and fail to capture the true pattern in the data.

*Best practice: Use cross-validation to find the optimal C. Split your data into training and validation sets, train SVMs with different C values, and choose the one that performs best on the validation s*et.

4.7 Non-linear SVM

Many real-world classification problems are inherently non-linear; no straight line, plane, or hyperplane can adequately separate the classes. Consider trying to separate points arranged in two concentric circles, or a checkerboard pattern. A linear classifier would fail miserably on such data.

The brilliant insight behind non-linear SVMs is that data which is not linearly separable in its original space may become linearly separable when transformed into a higher dimensional space. By mapping the input features to a new, richer representation, we can apply linear SVM in this transformed space, effectively creating non-linear decision boundaries in the original space.

Definition 4.7.1 (Non-linear SVM via Feature Mapping) *Instead of working directly with input features x, we first apply a* ***feature map*** ϕ *that transforms each input to a Higher-dimensional space*

$$\mathbf{x} \in \mathbf{R}^d \xrightarrow{\phi} \boldsymbol{\phi}(\mathbf{x}) \in \mathbf{R}^D \textbf{ where typically } D >> d \qquad (4.18)$$

We then apply the standard linear SVM in this transformed D-dimensional feature space. The decision boundary is linear in the feature space, but when projected back to the original space, it becomes non-linear

The Core Idea: Data that appears hopelessly intertwined in low dimensions often becomes separable when viewed from a higher-dimensional perspective. As a simple analogy, imagine points on a table (2D) that cannot be separated by

a line. If you lift some points on the table (adding a 3rd dimension), you might be able to separate them with a flat plane.

Example 4.7.1 (Solving the XOR Problem with Feature Transformation) **The XOR Data** *(famously not linearly separable in 2D):*

x_1	x_2	*Class (XOR output)*
0	*0*	*-1 (False)*
0	*1*	*+1 (True)*
1	*0*	*+1 (True)*
1	*1*	*-1 (False)*

If you plot these four points, you'll see that the +1 points are at opposite corners, as are the -1 points. No straight line can separate them.

The Feature Transformation: *We add a new feature that is the product of the two*

inputs: $\phi(x_1, x_2) = (x_1, x_2, x_1 \cdot x_2)$ ***Transformed data in 3D:***

x_1	x_2	$x_1 \cdot x_2$	*Class*
0	*0*	*0*	*-1*
0	*1*	*0*	*+1*
1	*0*	*0*	*+1*
1	*1*	*1*	*-1*

Now look at the third column: the two +1 points both have $x_1 \cdot x_2 = 0$*, while one -1 point has* $x_1 \cdot x_2 = 0$ *and the other has* $x_1 \cdot x_2 = 1$*. Combined with the original features, we can now find a separating hyperplane! For instance,* $x_1 + x_2 - 2(x_1 \cdot x_2) - 0.5 = 0$ works.

This demonstrates how a problem that was impossible in 2D becomes solvable in 3D through clever feature engineering.

The Problem with Explicit Feature Mapping: While the feature transformation idea is powerful, it comes with a significant computational challenge. If we map to a very high-dimensional space (which we often need for complex problems), computing $\phi(x)$ explicitly and then computing inner products $\phi(x)^T \phi(z)$ becomes extremely expensive or even impossible. For some useful mappings, the feature space is infinite-dimensional! This is where the kernel trick comes to the rescue.

4.8 Kernel Trick

The kernel trick is one of the most elegant ideas in machine learning. It allows us to work in very high-dimensional (even infinite-dimensional) feature spaces without ever explicitly computing the feature vectors in those spaces. This makes non-linear SVM practical and computationally efficient.

The key observation is that the SVM algorithm (both training and prediction) only needs to compute inner products (dot products) between feature vectors. We never need the feature vectors themselves, only the inner products $\phi(x)^T \phi(z)$ between pairs of transformed points. The kernel trick provides a shortcut: a kernel function $K(x, z)$ that computes this inner product directly from the original features, without ever computing $\phi(x)$ or $\phi(z)$.

Definition 4.8.1 (Kernel Function) A kernel function $K(x, z)$ computes the inner product between two points in the transformed feature space directly from the original representations:

$$K(x, z) = \phi(x)^T \phi(z) = \langle \phi(x), \phi(z) \rangle \qquad (4.18)$$

The magic is that K can be computed efficiently even when $\phi(x)$ would be impractical or impossible to compute explicitly. This allows us to implicitly work in the high dimensional feature space.

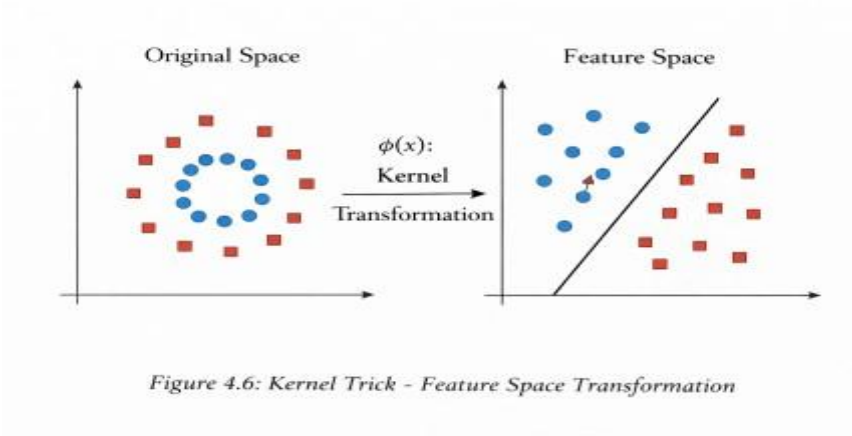


Figure 4.6: Kernel Trick - Feature Space Transformation

Figure 4.6: The Kernel Trick: Data that is not linearly separable in the original space (left, with points arranged in a circle surrounded by another class) becomes linearly separable after kernel transformation to a higher-dimensional feature space (right).

Common Kernel Functions: Several kernel functions have been developed, each corresponding to a different implicit feature space:

Kernel Name	Formula	Use Case
Linear	$K(\mathbf{x}, \mathbf{z}) = \mathbf{x}^T\mathbf{z}$	Already linearly separable data
Polynomial (degree d)	$K(\mathbf{x}, \mathbf{z}) = (\mathbf{x}^T\mathbf{z} + c)^d$	Polynomial decision boundaries
RBF (Gaussian)	$K(\mathbf{x}, \mathbf{z}) = \exp\left(-\frac{\|\mathbf{x}-\mathbf{z}\|^2}{2\sigma^2}\right)$	Complex, local patterns
Sigmoid	$K(\mathbf{x}, \mathbf{z}) = \tanh(\alpha\mathbf{x}^T\mathbf{z} + c)$	Neural network similarity

Example 4.8.1 (Understanding the Polynomial Kernel) Kernel: *Consider the polynomial kernel of degree 2: K(x, z) = $(x^T z)^2$*

Let's see what implicit feature transformation this kernel corresponds to for 2D inputs x = (x_1, x_2) and z = (z_1, z_2).

Computing the kernel directly:

$$K(x, z) = (x^T z)^2 = (x_1 z_1 + x_2 z_2)^2 \tag{4.19}$$

$$= x_1^2 z_1^2 + 2x_1 x_2 z_1 z_2 + x_2^2 z_2^2 \tag{4.20}$$

This looks like an inner product! Indeed, if we de ne the feature map:

$$\phi(x_1, x_2) = (x_1^2, \sqrt{2}x_1 x_2, x_2^2) \tag{4.21}$$

then we can verify:

$$\phi(\mathbf{x})^T\phi(\mathbf{z}) = x_1^2 z_1^2 + (\sqrt{2}x_1 x_2)(\sqrt{2}z_1 z_2) + x_2^2 z_2^2 \tag{4.22}$$

$$= x_1^2 z_1^2 + 2x_1 x_2 z_1 z_2 + x_2^2 z_2^2 = K(\mathbf{x}, \mathbf{z}) \tag{4.23}$$

The power of the kernel: To compute K(x, z), we only need to compute the 2D inner product $(x_1z_1 + x_2z_2)$ and square it just 3 multiplications and 1 addition. But this gives us the same result as if we had transformed both points to 3D and computed the inner product there. For higher-degree polynomials in higher dimensions, the savings become even more dramatic.

The RBF (Gaussian) Kernel: The Radial Basis Function kernel is the most popular choice for non-linear SVM because of its flexibility and strong performance on a wide variety of problems. It is defined as:

$$K(\mathbf{x}, \mathbf{z}) = \exp\left(-\frac{||\mathbf{x} - \mathbf{z}||^2}{2\sigma^2}\right)$$

The RBF kernel implicitly maps data to an **infinite-dimensional** feature space, yet we can compute the kernel value efficiently. The parameter σ controls the "reach" of each point's influence: smaller σ means each training point only affects nearby predictions, while larger σ means each point influences a wider region.

Example 4.8.2 (Computing RBF Kernel Values) Two points: x *= (1, 0) and z = (0, 1), with σ = 1*

First, compute the squared Euclidean distance:

$$\|x - z\|^2 = (1 - 0)^2 + (0 - 1)^2 = 1 + 1 = 2 \quad (4.24)$$

Then apply the RBF formula:

$$K(\mathbf{x}, \mathbf{z}) = \exp\left(-\frac{2}{2 \times 1^2}\right) = \exp(-1) = e^{-1} \approx 0.368 \quad (4.25)$$

Interpretation: The kernel value can be thought of as a similarity measure. A value of 1 means the points are identical (maximum similarity). A value close to 0 means the points are very different. Here, 0.368 indicates moderate similarity. Points that are closer together have kernel values closer to 1, while points farther apart have kernel values approaching 0.

4.9 Logistic Regression

While the perceptron and SVM make hard classification decisions (definitively assigning each point to one class or another), many applications benefit from knowing the **probability** or confidence of a prediction. **Logistic regression** is a linear classifier that outputs the probability that an input belongs to a particular class, rather than just a class label.

Despite its name containing "regression", logistic regression is actually a classification method. The name comes from its use of the logistic (sigmoid) function to transform a linear combination of features into a probability value between 0 and 1. This probabilistic output is valuable in many scenarios: a medical diagnosis system might report "80% probability of disease" rather than just "diseased"; a fraud detection system might flag transactions with probability greater than 0.7 for review.

Definition 4.9.1 (Logistic Regression Model) Logistic regression models the probability of class 1 given input features x using the sigmoid function applied to a linear combination:

$$P(Y = 1/x) = \sigma(w^T x + b) = 1/(1 + e^{-(w^T x + b)}) \quad (4.26)$$

where $\sigma(z) = 1/1+e^z$ is the sigmoid function (also called the logistic function). The

weights w and bias *b* are learned from training data to maximize the likelihood of the observed class labels.

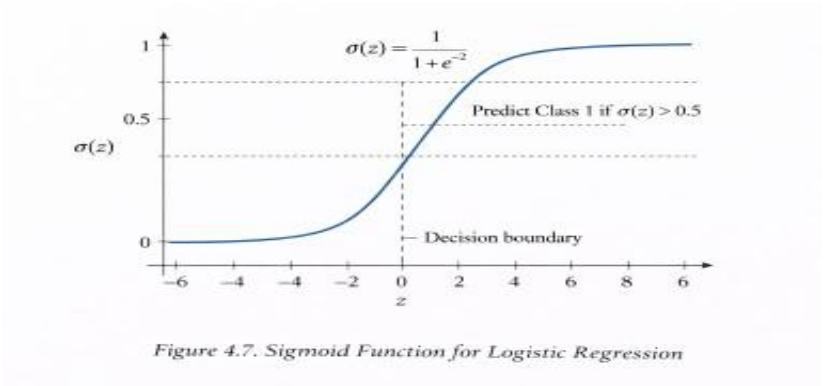


Figure 4.7. Sigmoid Function for Logistic Regression

Figure 4.7: The Sigmoid Function for Logistic Regression: The S-shaped curve smoothly maps any real value to a probability between 0 and 1. The decision boundary is at z = 0 where σ(z) = 0.5.

Properties of the Sigmoid Function: The sigmoid function has several properties that make it ideal for modeling probabilities:

- **Output Range [0, 1]:** The sigmoid always outputs a value between 0 and 1, which can be interpreted as a valid probability. No matter how extreme the input, the output stays within this range.
- **Decision Boundary at $\sigma(0) = 0.5$:** When the linear combination $w^T x+b = 0$, the sigmoid outputs exactly 0.5, indicating maximum uncertainty. This is the natural decision boundary.
- **Asymptotic Behavior:** As the input z becomes very large and positive, $\sigma(z)$ approaches 1 (high confidence in class 1). As z becomes very large and negative, $\sigma(z)$ approaches 0 (high confidence in class 0).
- **Smooth and Differentiable:** Unlike the step function in the perceptron, the sigmoid is smooth everywhere, which allows us to use gradient-based optimization methods to train the model.
- **Convenient Derivative:** The derivative of the sigmoid has the elegant form $\sigma'(z) = \sigma(z)(1 - \sigma(z))$, which makes gradient computations efficient.

Classification Decision Rule: To make a concrete classification decision from the probability, we typically use a threshold of 0.5:

- Predict Class 1 if $P(Y = 1/x) \geq 0.5$ (equivalently, if $w^T x + b \geq 0$)
- Predict Class 0 if $P(Y = 1/x) < 0.5$ (equivalently, if $w^T x + b < 0$)

However, the threshold can be adjusted based on the application's needs (e.g., a lower threshold for disease detection where missing a positive case is more costly than a false alarm).

Definition 4.9.2 (Training Logistic Regression via Maximum Likelihood) The weights are learned by minimizing the cross-entropy loss (which is equivalent to maximizing the log-likelihood of the observed labels):

$$L(w, b) = -\sum_{i=1} [y_i \log(\hat{p}_i) + (1 - y_i) \log(1 - \hat{p}_i)] \quad (4.27)$$

where $\hat{p}_i = \sigma(w^T x_i + b)$ is the predicted probability for training example i, and $y_i \in \{0, 1\}$ is the true label. This loss function penalizes con dent wrong predictions more heavily than uncertain ones.

Example 4.9.1 (Making Predictions with Logistic Regression) Scenario: A logistic regression model has been trained to predict whether a customer will purchase a product based on their age and income. The trained weights are: $w_1 = 2$ (for age, scaled), $w_2 = -1$ (for income, scaled), and bias $b = 0.5$.

New customer: A customer has features x = (1, 3) (representing scaled age and income values).

Step 1 - Compute the linear combinate

$$z = w^T x + b = 2(1) + (-1)(3) + 0.5 = 2 - 3 + 0.5 = -0.5 \quad (4.28)$$

The negative value of z suggests this customer is more likely to be in class 0.

Step 2 - Apply the sigmoid function:

$$P(Y = 1|\mathbf{x}) = \sigma(-0.5) = \frac{1}{1 + e^{0.5}} = \frac{1}{1 + 1.649} = \frac{1}{2.649} \approx 0.378 \quad (4.29)$$

Step 3 - Make the prediction: Since 0.378 < 0.5, we predict Class 0 (customer will not purchase).

Interpreting the result: The model predicts with 62.2% confidence that the customer will not purchase (probability 0.622 of Class 0). Alternatively, there is a 37.8% chance the customer will purchase. This probabilistic output allows the business to make nuanced decisions perhaps customers with purchase probability above 0.3 could receive a targeted promotion.

Logistic Regression vs. Perceptron: Both methods create linear decision boundaries, but they differ significantly:

- **Perceptron:** Makes hard decisions (+1 or -1) with no indication of confidence. A point just barely on one side of the boundary gets the same output as a point far from the boundary.
- **Logistic Regression:** Outputs probabilities (0 to 1) that reject the model's confidence. Points far from the boundary have probabilities

close to 0 or 1, while points near the boundary have probabilities close to 0.5, honestly rejecting uncertainty.

- **Training:** Perceptron uses a simple update rule on misclassified points. Logistic regression uses gradient descent to optimize the cross-entropy loss, which considers all training points and their distances from the boundary.

4.10 Linear Regression

While all the previous methods in this chapter are designed for classification (predicting discrete categories), **linear regression** is used for predicting c**ontinuous numerical values.** It finds the best linear relationship between input features and a continuous target variable.

Linear regression is one of the oldest and most widely used statistical methods. Despite its simplicity, it serves as the foundation for understanding more complex models and remains a go-to method for many practical applications. Examples include predicting house prices from features like size and location, estimating sales based on advertising spend, or forecasting temperatures based on various meteorological measurements.

Definition: 4.10.1 (Linear Regression Model) Linear regression predicts a continuous target value *y* as a linear function of the input features:

$$\hat{y} = w^T x + b = w_1 x_1 + w_2 x_2 + \cdots + w_d x_d + b \qquad (4.30)$$

where *y*ˆ is the predicted value, w is the weight vector determining the contribution of each feature, and *b* is the bias (intercept) representing the baseline prediction when all features are zero.

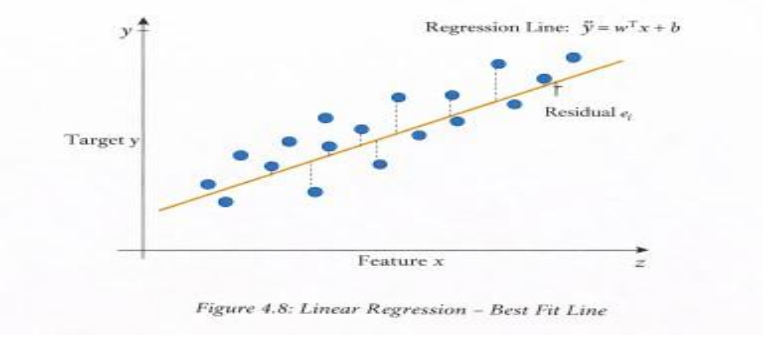


Figure 4.8: Linear Regression: Finding the Best Fit Line that minimizes the sum of squared residuals (vertical distances from data points to the line).

The Goal of Linear Regression: We want to find the weights w and bias *b* that make our predictions $\hat{y}_i$ as close as possible to the actual target values y_i for all training examples. The most common measure of closeness is the sum of squared errors (SSE), which penalizes larger errors more heavily than smaller ones.

Definition: 4.10.2 (Ordinary Least Squares (OLS) Objective) The OLS method finds weights that minimize the sum of squared differences between predicted and actual values:

$$\min_{w,b} \sum_{i=1}^{n} (y_i - \hat{y}_i)^2 = \sum_{i=1}^{n} (y_i - w^T x_i - b)^2 \qquad (4.31)$$

This objective function is convex, meaning it has a unique global minimum that can be found analytically or through optimization algorithms.

Closed-Form Solution: One beautiful property of linear regression is that the optimal weights can be computed directly using a formula (no iterative optimization required). If we stack all training inputs into a matrix X (with a column of ones for the bias) and all targets into a vector y, the optimal weights are:

$$W^* = (X^T X)^{-1} X^T y \qquad (4.32)$$

This is called the normal equation. It gives us the exact optimal solution in one step, as long as X^TX is invertible.

Example 4.10.1 (Predicting House Prices with Linear Regression) Problem: Build a model to predict house prices based on their size.

Training Data:

Size (sq ft)	Price ($K)
1000	200
1500	280
2000	350
2500	420

Fitting the Model: Using the OLS method (or gradient descent), we find the best linear relationship. The fitted model is approximately:

$$\hat{y} = 0.147x + 55 \qquad (4.33)$$

where x is the size in square feet and $\hat{y}$ is the predicted price in thousands of dollars. The weight 0.147 means that each additional square foot adds about $147 to the price. The intercept 55 represents the base price

Making a Prediction: For a house with 1800 square feet:

$$\hat{y} = 0.147 \times 1800 + 55 = 264.6 + 55 = 319.6 \quad (4.34)$$

The model predicts a price of approximately $319,600.

Evaluating the Model: We can compute the predictions for training data and compare with actual prices to see how well the model fits. The residuals (differences between actual and predicted) should be small and randomly distributed if the linear model is appropriate.

Regularization: When we have many features or limited training data, the model might overfit; fitting the training data very well but generalizing poorly to new data. Regularization adds a penalty term to discourage overly complex models:

- **Ridge Regression (L2 Regularization):** Adds penalty term

$$\lambda \|w\|_2^2 = \lambda \sum_{j=1}^{d} w_j^2$$

  to the loss function. This encourages all weights to be small but non-zero. The parameter λ controls the strength of regularization.

- **Lasso Regression (L1 Regularization):** Adds penalty term

$$\lambda \|w\|_1 = \lambda \sum_{j=1}^{d} |w_j|$$

  to the loss. This not only shrinks weights but can drive some weights exactly to zero, effectively performing feature selection. Lasso is useful when we suspect only a few features are truly relevant.

- **Elastic Net:** Combines both L1 and L2 penalties, providing a balance between the properties of Ridge and Lasso regression.

$$\lambda_1 \|w\|_1 + \lambda_2 \|w\|_2^2$$

4.11 Multi-Layer Perceptrons (MLPs)

Before understanding MLPs, it is crucial to review the **single-layer perceptron**, which forms the building block of neural networks.

Definition 4.11.1 (Single Layer Perceptron): A single-layer perceptron is a linear classifier that computes:

$$\hat{y} = \sigma(\mathbf{w}^T\mathbf{x} + b) \quad \textbf{(4.26)}$$

where:

- w is the weight vector,
- x is the input vector,
- b is the bias term,
- σ is an activation function.

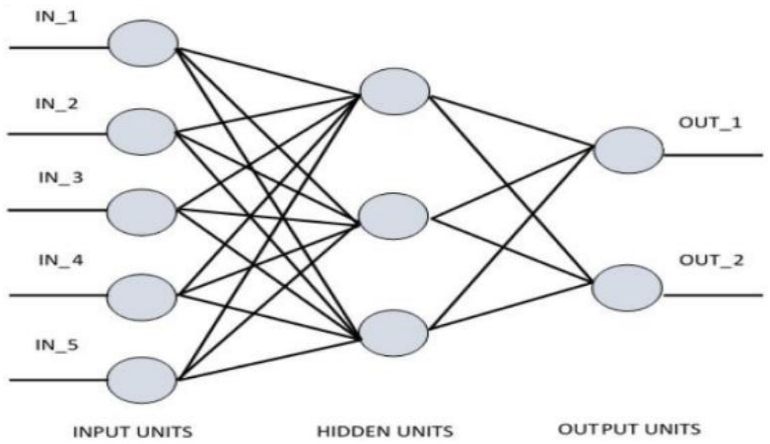
Common activation functions include:

Step Function (Hard Classification)

$$\sigma(z) = \begin{cases} 1, & z \geq 0 \\ 0, & \text{otherwise} \end{cases}$$

Sigmoid Function (Probabilistic Output)

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$



Limitations of Single-Layer Perceptrons:

Single-layer perceptrons can only learn linearly separable patterns. They fail on problems like XOR, where positive and negative examples cannot be separated by a single line.

The XOR Problem Example:

$x1$	$x2$	OR
0	0	0
0	1	1
1	0	1
1	1	0

The two classes (0s and 1s) form a checkerboard pattern in 2D space that cannot be separated by any single line. A single perceptron cannot solve XOR problem.

A **Multi-Layer Perceptron (MLP)** extends the single perceptron to a network of multiple layers of artificial neurons. This seemingly simple extension stacking layers of perceptrons gives rise to a dramatically more powerful model capable of learning complex, non-linear decision boundaries that would be impossible for a single perceptron.

The key insight is that by combining multiple linear transformations with non-linear activation functions, MLPs can approximate arbitrarily complex functions. Each layer transforms the data in a way that makes the final classification or regression task easier. Early layers might detect simple patterns, while later layers combine these patterns to recognize more complex structures.

Definition 4.11.2 (Multi-Layer Perceptron Architecture) An MLP is a feed forward neural network organized into layers:

- **Input Layer:** Receives the raw feature vector x = $(x_1, x_2, \ldots, x_d)^T$. This layer
  simply passes the input to the next layer; no computation happens here.
- **Hidden Layer(s):** One or more intermediate layers of neurons. Each neuron computes a weighted sum of its inputs, adds a bias, and applies a non linear activation function. These layers are called hidden because their values are not directly observed they are internal representations learned by the network.
- **Output Layer:** Produces the final prediction. For binary classification, this might be a single neuron with a sigmoid activation. For multi-class classification, it might have one neuron per class with a softmax

activation. For regression, it might be a single neuron with no activation (linear output).

Each neuron in a hidden or output layer computes: $a = \sigma(w^T x + b)$ where σ is a non-linear activation function, w are the weights connecting inputs to this neuron, and b is the bias.

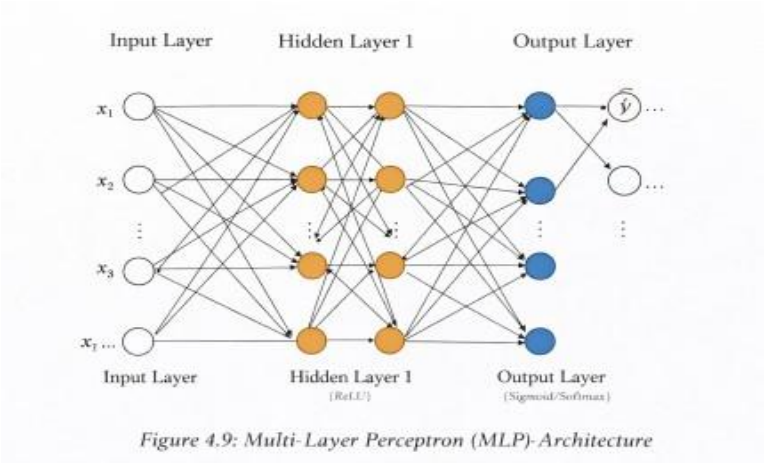


Figure 4.9: Multi-Layer Perceptron (MLP)-Architecture

Figure 4.9: Multi-Layer Perceptron (MLP) Architecture: Input layer receives features, hidden layers learn intermediate representations using non-linear activations (like ReLU), and the output layer produces final predictions.

The Importance of Non-Linear Activations: The activation function introduces non-linearity, which is crucial for the power of MLPs. Without non-linear activations, stacking multiple layers would be pointless any sequence of linear transformations is equivalent to a single linear transformation. The non-linearity allows the network to learn complex patterns.

Common Activation Functions:

Name	Formula	Output Range	Common Use
Sigmoid	$\sigma(z) = \frac{1}{1+e^{-z}}$	$(0, 1)$	Output layer (binary classification)
Tanh	$\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$	$(-1, 1)$	Hidden layers (older networks)
ReLU	$\text{ReLU}(z) = \max(0, z)$	$[0, \infty)$	Hidden layers (modern standard)
Softmax	$\text{softmax}(z_i) = \frac{e^{z_i}}{\sum_j e^{z_j}}$	$(0, 1)$	Output layer (multi-class)

ReLU (Rectified Linear Unit) has become the default choice for hidden layers because it is simple, computationally efficient, and helps avoid the "vanishing gradient" problem that can slow down training with sigmoid or tanh.

Forward Propagation: Computing the output of an MLP involves passing information forward through the network, layer by layer. For a network with one hidden layer:

$z^{(1)} = W_1 x + b_1$ (linear transformation) (4.35)

$h = \sigma(z^{(1)})$ (hidden layer activations) (4.36)

$z^{(2)} = W_2 h + b_2$ (linear transformation) (4.37)

$\hat{y} = f(z^{(2)})$ (output layer, with appropriate activation f) (4.38)

Each layer first computes a weighted sum (linear transformation), then applies the activation function (non-linear transformation).

Example 4.11.1 (How an MLP Solves the XOR Problem) The Challenge:
The XOR function cannot be computed by a single perceptron because the positive and negative examples are not linearly separable. But an MLP with one hidden layer can solve it!
Architecture: We use 2 input neurons, 2 hidden neurons (with step activation),

and 1 output neuron.
Hidden layer weights (each hidden neuron computes a different linear function):

$$\mathbf{W}_1 = \begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix}, \quad \mathbf{b}_1 = \begin{pmatrix} -0.5 \\ -1.5 \end{pmatrix} \tag{4.39}$$

The first hidden neuron computes h1 = step(x1+x2−0.5), which outputs 1 if at least
one input is 1 (OR gate). The second hidden neuron computes h2 = step(x1+x2−1.5),
which outputs 1 only if both inputs are 1 (AND gate).
Output layer weights: w2 = (1,−2)T , b2 = 0
Computing XOR(1, 1) step by step:

$$z_1^{(1)} = 1(1) + 1(1) - 0.5 = 1.5 \geq 0 \implies h_1 = 1 \quad \textit{(OR: at least one input is 1)} \tag{4.40}$$

$$z_2^{(1)} = 1(1) + 1(1) - 1.5 = 0.5 \geq 0 \implies h_2 = 1 \quad \textit{(AND: both inputs are 1)} \tag{4.41}$$

$$\hat{y} = 1 \times h_1 + (-2) \times h_2 + 0 = 1 - 2 = -1 < 0 \implies \textit{output 0 (False)} \tag{4.42}$$

These matches XOR (1,1) = 0! The network computes: XOR = OR − 2 × AND,
which is equivalent to OR ∧ ¬ AND, the definition of XOR.

Universal Approximation Theorem: A remarkable theoretical result states that an MLP with a single hidden layer containing enough neurons can approximate any continuous function to arbitrary accuracy. This means MLPs are **universal approximators** given enough hidden neurons, they can learn to represent any reasonable function. However, finding the right weights through training is a separate challenge, which is where backpropagation comes in.

4.12 Backpropagation for Training an MLP

Backpropagation (short for "backward propagation of errors") is the fundamental algorithm used to train neural networks. It provides an efficient method for computing how much each weight in the network contributed to the overall prediction error, enabling us to update weights in a way that reduces this error. Backpropagation is essentially a clever application of the chain rule from calculus, applied systematically through the layers of the network.

The key insight behind backpropagation is that we can compute the gradient of the loss function with respect to any weight in the network by working backward from the output layer to the input layer. At each layer, we determine how sensitive the loss is to changes in that layer's activations, then use this information to compute how sensitive the loss is to changes in the layer's weights.

Definition 4.12.1 (Backpropagation) Backpropagation computes gradients of the loss function with respect to each weight using the chain rule of calculus:

$$\frac{\partial L}{\partial w_{ij}} = \frac{\partial L}{\partial a_j} \cdot \frac{\partial a_j}{\partial z_j} \cdot \frac{\partial z_j}{\partial w_{ij}} \tag{4.43}$$

where:

- L is the loss function (measuring how wrong the prediction is)
- $a_j = \sigma(z_j)$ is the activation of neuron j after applying the activation function
- $z_j = \sum_i w_{ij} a_i + b_j$ is the pre-activation (weighted sum of inputs plus bias)
- w_{ij} is the weight connecting neuron i to neuron j

The chain rule allows us to decompose a complex gradient into a product of simpler gradients, each of which can be computed locally at each neuron.

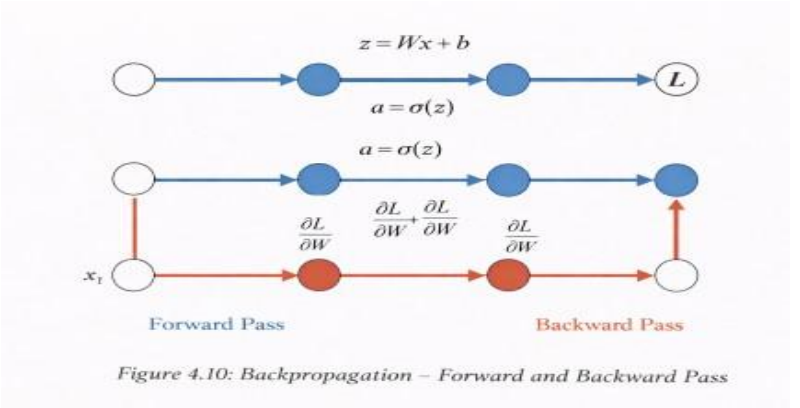


Figure 4.10: Backpropagation – Forward and Backward Pass

Figure 4.10: Backpropagation: The Forward Pass computes activations from input to output, while the Backward Pass propagates error gradients from output back to input, enabling weight updates through gradient descent.

The Complete Training Algorithm:

The training process for a neural network using backpropagation consists of four main steps, repeated many times until the network converges to a good solution:

1. **Forward Pass:** Pass the input through the network layer by layer, computing and storing the pre-activations $z^{(l)}$ and activations $a^{(l)}$ at each layer. These stored values will be needed during the backward pass. At the end of the forward pass, we have the network's prediction $\hat{y}$.
2. **Compute Loss:** Compare the network's prediction $\hat{y}$ with the true target value y using a loss function. For regression, this is typically Mean Squared Error: $L = \frac{1}{2}(y - \hat{y})^2$. For classification, this is typically Cross-Entropy Loss: $L = -[y \log \hat{y} + (1 - y) \log(1 - \hat{y})]$. The loss tells us how wrong the prediction is.
3. **Backward Pass:** Starting from the output layer, compute the gradient of the loss with respect to each layer's pre-activation (the "error signal" δ). Then propagate this error signal backward through the network, layer by layer. At each layer, compute the gradients with respect to the weights and biases using the stored activations from the forward pass.
4. **Update Weights:** Adjust each weight in the direction that reduces the loss using gradient descent: $w \leftarrow w - \eta \frac{\partial L}{\partial w}$, where η is the learning rate (a small positive number like 0.01 or 0.001).

Key Gradient Formulas for Backpropagation:

The error signal $\delta^{(l)}$ at layer l represents how much the pre-activation at that layer affects the final loss. Different formulas apply to the output layer versus hidden layers:

For the output layer (layer *L*) - the error signal is computed directly from the loss:

$$\delta^{(L)} = \frac{\partial L}{\partial a^{(L)}} \odot \sigma'(z^{(L)}) \qquad (4.44)$$

where $\odot$ denotes element-wise multiplication and $\sigma'$is the derivative of the activation function. For MSE loss with linear output, this simplifies to $\delta^{(L)} = (\hat{y} - y)$. For cross entropy with sigmoid, it's also $\delta^{(L)} = (\hat{y} - y)$ (the sigmoid derivative cancels nicely). ***For hidden layers*** the error signal is propagated backward from the next layer:

$$\delta(l) = (W(l+1))T\delta(l+1) \odot \sigma'(z^{(l)}) \qquad (4.45)$$

This formula says: multiply the error from the next layer by the weights connecting to it, then scale by the derivative of our activation function. This is where the backward propagation name comes from errors flow backward through the weight matrices

Weight and bias gradients once we have the error signals, computing the gradients is straightforward:

$$\frac{\partial L}{\partial \mathbf{W}^{(l)}} = \delta^{(l)}(a^{(l-1)})^T \quad \text{(weight gradient)} \qquad (4.46)$$

$$\frac{\partial L}{\partial \mathbf{b}^{(l)}} = \delta^{(l)} \quad \text{(bias gradient)} \qquad (4.47)$$

The weight gradient is the outer product of the error signal and the incoming activations. The bias gradient is simply the error signal itself.

Example 4.12.1 (Step-by-Step Backpropagation Calculation) Let's work through a complete backpropagation example for a tiny network.

Network Architecture: One input x, one hidden neuron h (with sigmoid acti vation), one output $\hat{y}$ (linear activation for regression).

Initial Setup: Input $x = 1$, target $y = 1$, initial weights $w_1 = 2$ (input to hidden), $w_2 = 1$

(hidden to output), biases $b_1 = b_2 = 0$, learning rate $\eta = 0.1$.

FORWARD PASS - Computing the prediction:

Step 1: Compute hidden layer pre-activation:

$$z_h = w_1 \cdot x + b_1 = 2 \times 1 + 0 = 2 \qquad (4.48)$$

Step 2: Apply sigmoid activation to get hidden layer output:

$$h = \sigma(zh) = \sigma(2) = 1 \ -21 + e = 1 \ 1 + 0.135 = 0.88 \qquad (4.49)$$

Step 3: Compute output layer pre-activation and output (linear activation):

$$z_o = w_2 \cdot h + b_2 = 1 \times 0.88 + 0 = 0.88, \hat{y} = z_o = 0.88 \qquad (4.50)$$

COMPUTE LOSS - Measuring the error:

Using Mean Squared Error:

$$L=1/2(y - \hat{y})^2 = 2(1 \ L = 1/2(1 - 0.88)^2 = 1/2(0.12)^2 = 0.0072 \qquad (4.51)$$

The network predicted 0.88 but the target is 1, so there's some error to correct.

BACKWARD PASS - Computing gradients:

Step 1: Compute output layer error (derivative of loss with respect to prediction):

$$\frac{\partial L}{\partial \hat{y}} = -(y - \hat{y}) = -(1 - 0.88) = -0.12 \qquad (4.52)$$

The negative sign indicates we need to increase $\hat{y}$ to reduce the loss.

Step 2: Compute gradient with respect to w_2 (weight connecting hidden to output):

$$\frac{\partial L}{\partial w_2} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial w_2} = -0.12 \times h = -0.12 \times 0.88 = -0.106 \qquad (4.53)$$

Step 3: Backpropagate error to hidden layer:

$$\frac{\partial L}{\partial h} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial h} = -0.12 \times w_2 = -0.12 \times 1 = -0.12 \qquad (4.54)$$

Step 4: Compute gradient with respect to w_1 using chain rule through the sigmoid:

$$\sigma'(z_h) = \sigma(z_h) \cdot (1 \quad \sigma(z_h)) = 0.88 \times (1 \quad 0.88) = 0.88 \times 0.12 = 0.105 \qquad (4.55)$$

$$\frac{\partial L}{\partial w_1} = \frac{\partial L}{\partial h} \cdot \sigma'(z_h) \cdot x = -0.12 \times 0.105 \times 1 = -0.013 \qquad (4.56)$$

UPDATE WEIGHTS - Applying gradient descent:

$$w_2^{new} = w_2 - \eta \cdot \frac{\partial L}{\partial w_2} = 1 - 0.1 \times (-0.106) = 1 + 0.0106 = 1.011 \qquad (4.57)$$

$$w_1^{new} = w_1 - \eta \ \frac{\partial L}{\partial w_1} = 2 - 0.1 \times (-0.013) = 2 + 0.0013 = 2.001 \qquad (4.58)$$

Both weights increased slightly, which will make the network's output larger (closer to the target of 1). After many such updates, the network will converge to weights that accurately predict the target.

Training Tips and Best Practices:

Training neural networks effectively requires more than just implementing backpropagation. Here are key techniques that improve training:

Gradient Descent Variants

Efficient training requires choosing the right optimization strategy for updating weights. **Batch Gradient Descent (BGD):**

$$\mathbf{w} \leftarrow \mathbf{w} - \eta \sum^{n} \nabla_{\mathbf{w}} L_i(\mathbf{w}) \qquad (4.56)$$

Update weights after seeing all n training examples. Advantage: stable convergence. Disadvantage: slow for large datasets.

Stochastic Gradient Descent (SGD):

$$\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla_{\mathbf{w}} L_i(\mathbf{w}) \qquad (4.57)$$

Update weights after each single example. Advantage: fast updates. Disadvantage: noisy, oscillatory convergence.

Mini-batch Gradient Descent(MGD):

$$\mathbf{w} \leftarrow \mathbf{w} - \eta \sum_{i=1}^{B} \nabla_{\mathbf{w}} L_i(\mathbf{w}) \qquad (4.58)$$

Update weights after a small batch of B examples (typically 32-256). Provides balance between BGD and SGD: more stable than SGD but faster than BGD.

Learning Rate Scheduling:

Instead of using a fixed learning rate throughout training, adapt it over time:

$$\eta t = \eta 0 \cdot \text{decay}(t) \qquad (4.59)$$

Common schedules:

- **Step Decay:** Halve the learning rate every k epochs .
- **Exponential Decay:** $\eta_t = \eta_0 e^{-\lambda t}$
- **Warm-up then Decay:** Start with small η, increase then decrease.

Momentum

Instead of updating based only on the current gradient, also consider the direction of previous updates:

$$\mathbf{v} \leftarrow \beta \mathbf{v} + \eta \nabla L \tag{4.60}$$

$$\mathbf{w} \leftarrow \mathbf{w} - \mathbf{v} \tag{4.61}$$

where β ≈ 0.9 is the momentum coefficient. This helps the optimization roll through shallow valleys in the loss landscape without oscillating.

Adam Optimizer

Adaptive Moment Estimation (Adam) maintains separate adaptive learning rates for each weight:

$$\mathbf{m} \leftarrow \beta_1 \mathbf{m} + (1 - \beta_1) \nabla L \tag{4.62}$$

$$\mathbf{v} \leftarrow \beta_2 \mathbf{v} + (1 - \beta_2)(\nabla L)^2 \tag{4.63}$$

$$\mathbf{w} \leftarrow \mathbf{w} - \frac{\eta \mathbf{m}}{\sqrt{\mathbf{v}} + \epsilon} \tag{4.64}$$

where **m** is the first moment (mean), v is the second moment (variance), typically β1 = 0.9, β2 = 0.999, and ϵ = 10−8.

Adam is the most commonly used optimizer in modern deep learning because it works well on diverse problems with minimal hyperparameter tuning.

AdaGrad (Adaptive Gradient)

Accumulates squared gradients to scale the learning rate:

$$\mathbf{g} \leftarrow \mathbf{g} + (\nabla L)^2 \tag{4.65}$$

$$\mathbf{w} \leftarrow \mathbf{w} - \frac{\eta}{\sqrt{\mathbf{g}} + \epsilon} \odot \nabla L \tag{4.66}$$

Parameters with larger gradients receive smaller adaptive learning rates. Advantage: reduces learning rate decay. Disadvantage: learning rate can decay too much in long training.

Nesterov Accelerated Gradient (NAG)

A variant of momentum that looks ahead before taking a step:

$$\mathbf{v} \leftarrow \beta \mathbf{v} + \eta \nabla L(\mathbf{w} - \beta \mathbf{v}) \tag{4.67}$$

$$\mathbf{w} \leftarrow \mathbf{w} - \mathbf{v} \tag{4.68}$$

NAG evaluates the gradient at the momentum step's" future" position, allowing it to correct course more quickly than standard momentum. Often converges faster than standard momentum.

RMSprop (Root Mean Square Propagation)

Divides the learning rate by an exponentially decaying average of squared gradients:

$$\mathbf{v} \leftarrow \beta \mathbf{v} + (1-\beta)(\nabla L)^2 \tag{4.69}$$

$$\mathbf{w} \leftarrow \mathbf{w} - \frac{\eta}{\sqrt{\mathbf{v}} + \epsilon} \odot \nabla L \tag{4.70}$$

where β ≈ 0.9. RMSprop adapts the learning rate per parameter while avoiding the learning rate decay problem of AdaGrad. It's particularly useful for recurrent neural networks and remains effective for feedforward networks.

Weight Initialization

Proper weight initialization is critical for successful training:

Bad practices:

- All zeros: The network cannot learn (all neurons compute identical functions)
- Large random values: Gradients explode or vanish

Good practices:

Xavier/Glorot Initialization (best for sigmoid/tanh):

$$w \sim \mathcal{U}\left(-\sqrt{\frac{6}{n_{\text{in}} + n_{\text{out}}}}, \sqrt{\frac{6}{n_{\text{in}} + n_{\text{out}}}}\right) \tag{4.71}$$

He Initialization (best for ReLU):

$$w \sim \mathcal{N}\left(0, \sqrt{\frac{2}{n_{\text{in}}}}\right) \tag{4.72}$$

where n_{in} is the number of input connections, n_{out} is the number of output connections.

Batch Normalization

Normalize activations within each mini-batch to have zero mean and unit variance:

$$\hat{a}_i^{(l)} = \frac{a_i^{(l)} - \mu_{\text{batch}}}{\sqrt{\sigma_{\text{batch}}^2 + \epsilon}} \tag{4.73}$$

Then apply learnable scale and shift:

$$\tilde{a}_i^{(l)} = \gamma \hat{a}_i^{(l)} + \beta \tag{4.74}$$

Benefits:

- Stabilizes training, allowing higher learning rates
- Reduces internal covariate shift.
- Acts as mild regularization.
- Networks converge faster.

Batch normalization has become standard in modern deep learning architectures.

Chapter Summary

This chapter provided a complete, self-contained treatment of linear discriminant models for machine learning, progressing from the simplest geometric intuition of a separating hyperplane through the mathematical sophistication of kernel methods, probabilistic classifiers, and deep neural networks.

1. Foundations of Linear Discriminants

A linear discriminant $f(x) = w^T x + b$ separates d-dimensional feature space with a $(d-1)$-dimensional hyperplane. The weight vector w is always orthogonal to this hyperplane and points toward the positive class region. The bias b shifts the hyperplane away from the origin. The signed distance from any data point x to the boundary equals $f(x)/\|w\|$, so the magnitude of $f(x)$ directly reflects classification confidence. For binary classification, points with $f(x) > 0$ are assigned to Class +1, those with $f(x) < 0$ to Class −1, and those exactly on the boundary ($f(x) = 0$) lie on the decision surface. Multi-class problems are handled by training K binary classifiers in a One-vs-All scheme, or $K(K-1)/2$ classifiers in a One-vs-One scheme.

2. Perceptron and Its Learning Algorithm

The Perceptron computes $\hat{y} = \text{sign}(w^T x + b)$ and learns by the update rule $w \leftarrow w + \eta \cdot y_i \cdot x_i$ whenever a training point is misclassified. Geometrically, this rotates the weight vector toward a positive misclassified point or away from a negative one. Worked examples demonstrated that a perceptron with $w_1=1$, $w_2=1$, $b=-1.5$ correctly implements the AND gate, and $w_1=1$, $w_2=1$, $b=-0.5$ implements OR. The Perceptron Convergence Theorem guarantees termination in at most $(R/\gamma)^2$ updates when the data is linearly separable, where γ is the margin of the best separator and R bounds the norm of training points. The fundamental limitation — inability to learn XOR and other non-linearly separable functions — motivated the development of all subsequent algorithms in the chapter.

3. Support Vector Machines

SVMs select among all valid separating hyperplanes the unique one that maximizes the geometric margin $2/\|w\|$. This is achieved by minimizing $\frac{1}{2}\|w\|^2$ subject to the constraint $y_i(w^T x_i + b) \geq 1$ for all training points — the hard-margin formulation. Only the support vectors (points lying exactly on the margin boundaries $w^T x + b = \pm 1$) determine the solution; removing all other training

points leaves it unchanged. This sparsity property makes SVMs robust and efficient at prediction time.

The soft-margin SVM relaxes the hard-margin constraint by introducing per-point slack variables $\xi_i \geq 0$, penalizing them with the regularization parameter C: minimize $\frac{1}{2}\|w\|^2 + C \cdot \Sigma \xi_i$. A large C approximates the hard-margin case (narrow margin, low training error, overfitting risk), while a small C prioritizes margin width over training accuracy (wider margin, some misclassifications, underfitting risk if too small). Cross-validation is the standard approach to selecting C.

Key Insight	The SVM margin $2/\|w\|$ has a direct connection to generalization: Vapnik-Chervonenkis theory shows that the generalization error is bounded by a term involving $1/\text{margin}^2$. Maximizing the margin thus minimizes the VC bound on test error, providing theoretical justification for why SVMs often outperform the Perceptron on unseen data.

4. Kernel Methods for Non-Linear Classification

When no linear hyperplane can adequately separate the classes, the kernel trick extends SVMs to non-linear boundaries. A feature map $\varphi: R^d \rightarrow R^D$ ($D \gg d$, potentially infinite) transforms inputs to a higher-dimensional space where classes become linearly separable. The kernel function $K(x, z) = \varphi(x)^T \varphi(z)$ computes the inner product in feature space from the original inputs, without ever computing φ explicitly. This is computationally equivalent to (but far cheaper than) forming the high-dimensional representations.

Four standard kernels were covered: the Linear kernel ($K = x^T z$) for already-separable data; the Polynomial kernel ($K = (x^T z + c)^d$) for polynomial decision boundaries; the Radial Basis Function (RBF/Gaussian) kernel $K(x,z) = \exp(-\|x-z\|^2/(2\sigma^2))$ — the most widely used, corresponding to an infinite-dimensional feature space — for complex local patterns; and the Sigmoid kernel for neural network-like similarity. The XOR example demonstrated how adding the product feature $x_1 \cdot x_2$ lifts 2D non-separable data to a 3D separable configuration.

5. Logistic Regression

Logistic regression models $P(Y=1|x) = \sigma(w^Tx + b)$ where $\sigma(z) = 1/(1+e^{-z})$ is the sigmoid function. Unlike the Perceptron's hard ±1 output, logistic regression produces calibrated probabilities: values near 0 or 1 reflect high confidence, while values near 0.5 reflect maximum uncertainty. The decision boundary is the set of points where $w^Tx + b = 0$, identical to a linear discriminant, but the sigmoid output enables nuanced threshold adjustment — critical in high-stakes applications (e.g., a cancer screener might lower the decision threshold to minimize missed positives).

Training minimizes the cross-entropy loss $L = -\Sigma[y_i\log(\hat{p}_i) + (1-y_i)\log(1-\hat{p}_i)]$, equivalent to maximizing the log-likelihood of the observed class labels. The sigmoid's smooth, everywhere-differentiable form enables gradient descent. Its derivative $\sigma'(z) = \sigma(z)(1-\sigma(z))$ is particularly convenient for implementation.

6. Linear Regression

Linear regression extends the discriminant framework to continuous target prediction: $\hat{y} = w^Tx + b$. The Ordinary Least Squares (OLS) criterion minimizes the sum of squared residuals $\Sigma(y_i - \hat{y}_i)^2$, a convex objective with the closed-form normal equation solution $w^* = (X^TX)^{-1}X^Ty$. The house price example illustrated fitting: with training data from 1000–2500 sq ft, the fitted model $\hat{y} = 0.147x + 55$ predicts $319,600 for a 1800 sq ft house, with the weight 0.147 directly interpretable as price-per-square-foot.

Regularization addresses overfitting with many features or limited data. Ridge (L2) regression adds $\lambda\|w\|^2$ to the loss, shrinking all weights toward zero without eliminating any. Lasso (L1) regression adds $\lambda\|w\|_1$, which can drive weights exactly to zero and thus performs automatic feature selection. Elastic Net combines both penalties, inheriting Ridge's stability and Lasso's sparsity.

7. Multi-Layer Perceptrons

A Multi-Layer Perceptron (MLP) extends the single perceptron to a feedforward network with an input layer, one or more hidden layers, and an output layer. Each hidden neuron computes $a = \sigma(w^Tx + b)$ using a non-linear activation. Without non-linear activations, stacking layers would be equivalent to a single linear transformation the non-linearity is what gives MLPs their power to approximate arbitrary continuous functions (Universal Approximation Theorem).

The XOR solution demonstrated this concretely: one hidden neuron implements OR (threshold 0.5), another implements AND (threshold 1.5), and the output

neuron computes OR − 2×AND = XOR. Common activation functions are Sigmoid (output layer, binary classification), Tanh (hidden layers, older architectures), ReLU = max(0, z) (hidden layers, modern standard; avoids vanishing gradients), and Softmax (output layer, multi-class). ReLU has become the default choice for hidden layers due to its computational simplicity and gradient-preservation properties.

8. Backpropagation and Optimization

Backpropagation computes gradients of the loss function with respect to every weight in the network by applying the chain rule layer by layer, working backward from the output. The error signal $\delta^{(L)} = \partial L/\partial a^{(L)} \odot \sigma'(z^{(L)})$ at the output layer propagates to hidden layers as $\delta^{(l)} = (W^{(l+1)})^T\delta^{(l+1)} \odot \sigma'(z^{(l)})$. Weight gradients are then $\partial L/\partial W^{(l)} = \delta^{(l)}(a^{(l-1)})^T$. A complete numerical example traced all four steps — forward pass, loss computation, backward pass, and weight update — for a single-hidden-neuron network.

Modern training employs several enhancements: Mini-batch gradient descent balances the stability of batch updates with the speed of stochastic updates. Learning rate scheduling (step decay, exponential decay, warm-up) adapts the learning rate over training. Momentum accumulates gradient history to accelerate convergence and smooth oscillations. Adam (Adaptive Moment Estimation) maintains per-weight adaptive learning rates using first and second gradient moments — the most widely used optimizer in deep learning. AdaGrad adapts rates based on cumulative squared gradients. Nesterov Accelerated Gradient evaluates gradients at a lookahead position. RMSprop uses exponentially decaying squared gradients to avoid AdaGrad's learning rate decay.

Weight initialization is critical: zero initialization causes symmetry breaking failure; large random values cause gradient explosion or vanishing. Xavier/Glorot initialization (for sigmoid/tanh: uniform in $[-\sqrt{(6/(n_{in}+n_{out}))}, +\sqrt{(6/(n_{in}+n_{out}))}]$) and He initialization (for ReLU: normal with $\sigma = \sqrt{(2/n_{in})}$) are the standard choices. Batch normalization normalizes layer activations within each mini-batch, stabilizes training, permits higher learning rates, and provides mild regularization.

Key Terms

Term	Definition
Linear Discriminant	A function $f(x) = w^Tx + b$ whose sign determines class membership; the boundary $f(x)=0$ is a $(d-1)$-dimensional hyperplane
Weight Vector	The vector w in $f(x) = w^Tx + b$; perpendicular to the decision boundary and encodes the influence of each feature
Bias	The scalar b in $f(x) = w^Tx + b$; offsets the decision boundary from the origin
Decision Boundary	The hyperplane $w^Tx + b = 0$ that separates feature space into two class regions
Signed Distance	Distance of point x from the hyperplane, given by $f(x)/\|\|w\|\|$; indicates both distance and side
Perceptron	A binary classifier computing $\hat{y} = \text{sign}(w^Tx + b)$; the simplest artificial neural unit
Perceptron Update Rule	Weight adjustment $w \leftarrow w + \eta \cdot y \cdot x$ applied whenever a training point is misclassified
Learning Rate (η)	A positive scalar controlling the step size of each weight update during training
Convergence Theorem	Guarantees the Perceptron terminates in $\leq (R/\gamma)^2$ updates on linearly separable data
Linear Separability	The existence of a hyperplane that perfectly separates all training points by class
Support Vector Machine	Classifier that finds the unique maximum-margin hyperplane separating the classes

Margin	Perpendicular distance from the decision boundary to the nearest training points; SVM maximizes $2/\|\|w\|\|$
Support Vectors	Training points lying exactly on the margin boundaries $w^Tx + b = \pm 1$; they define the optimal hyperplane
Hard-Margin SVM	SVM formulation requiring all training points to be correctly classified with margin ≥ 1
Soft-Margin SVM	SVM formulation with slack variables $\xi_i \geq 0$ permitting margin violations; controlled by parameter C
Slack Variable (ξ_i)	Measures how much training point i violates the margin; $\xi_i=0$ (outside margin), $0<\xi_i<1$ (inside margin), $\xi_i>1$ (misclassified)
Regularization (C)	Trade-off parameter in soft-margin SVM; large C → narrow margin, small C → wide margin with more violations
Feature Map (φ)	Function mapping inputs to a higher-dimensional space where classes become linearly separable
Kernel Function	$K(x,z) = \varphi(x)^T\varphi(z)$; computes inner products in feature space without forming φ explicitly
RBF Kernel	Gaussian kernel $K(x,z) = \exp(-\|\|x-z\|\|^2/(2\sigma^2))$; maps to infinite-dimensional space; most widely used kernel
Kernel Trick	Computing K(x,z) directly from original features, enabling implicit high-dimensional feature spaces

Logistic Regression	Linear classifier outputting class probabilities via $\sigma(w^Tx+b)$; trained by minimizing cross-entropy loss
Sigmoid Function	$\sigma(z) = 1/(1+e^{-z})$; S-shaped function mapping any real input to (0,1); decision boundary at $\sigma(0)=0.5$
Cross-Entropy Loss	Loss function $L = -\Sigma[y \log \hat{p} + (1-y) \log(1-\hat{p})]$; equivalent to negative log-likelihood for binary classification
Linear Regression	Predicts a continuous target as $\hat{y} = w^Tx + b$ by minimizing sum of squared residuals
OLS (Normal Equation)	Closed-form solution $w^* = (X^TX)^{-1}X^Ty$ that minimizes the sum of squared errors
Ridge Regression	L2 regularization adding $\lambda\|w\|^2$ to OLS; shrinks all weights toward zero without zeroing them
Lasso Regression	L1 regularization adding $\lambda\|w\|_1$ to OLS; induces sparsity by driving some weights exactly to zero
MLP	Multi-Layer Perceptron: feedforward neural network with input, hidden, and output layers; overcomes linear separability limit
Activation Function	Non-linear function (ReLU, Sigmoid, Tanh, Softmax) applied in each neuron; provides the non-linearity needed for complex learning
ReLU	Rectified Linear Unit: max(0,z); default hidden-layer activation; avoids vanishing gradients
Forward Propagation	Layer-by-layer computation of activations from input to output of an MLP

Backpropagation	Algorithm computing gradients of the loss w.r.t. all weights using the chain rule, propagating backward from output to input
Error Signal (δ)	Gradient of loss w.r.t. pre-activation at each layer; propagated backward during backpropagation
Adam Optimizer	Adaptive Moment Estimation; maintains per-weight adaptive learning rates using first and second gradient moments
Batch Normalization	Normalizes layer activations within each mini-batch; stabilizes training and permits higher learning rates
Xavier Initialization	Weight init for sigmoid/tanh: uniform in $\pm\sqrt{(6/(n_{in}+n_{out}))}$; prevents vanishing/exploding gradients
He Initialization	Weight init for ReLU: normal with $\sigma=\sqrt{(2/n_{in})}$; prevents vanishing gradients in ReLU networks

Table 4.1: Key terms and definitions for Unit IV.

Review Questions

Section A — Conceptual Questions

1. Define a linear discriminant function $f(x) = w^T x + b$ and explain the geometric role of each component. In what sense is the weight vector w 'perpendicular' to the decision boundary? How does the bias b affect the boundary?

2. Explain why the XOR function cannot be learned by a single perceptron. Use a geometric argument. How does the Minsky-Papert critique apply, and what fundamental insight motivated the development of multi-layer networks?

3. Describe the concept of margin in Support Vector Machines. Why does maximizing the margin tend to improve generalization to unseen data? What is the role of support vectors, and how many support vectors does an SVM typically have?

4. Compare and contrast the hard-margin and soft-margin SVM formulations. Under what real-world conditions would the hard-margin SVM fail, and how does the regularization parameter C in the soft-margin SVM control the bias-variance trade-off?

5. Explain the kernel trick. Why is computing $K(x, z) = \varphi(x)^T \varphi(z)$ directly from the original features advantageous over explicitly mapping inputs to the feature space $\varphi(x)$? Give an example using the polynomial kernel.

6. Compare logistic regression and the perceptron. Both create linear decision boundaries — what are the key differences in (a) their output, (b) their training objective, and (c) their usefulness in practice? When would you prefer logistic regression over a hard-boundary classifier?

7. Explain why non-linear activation functions are essential in multi-layer perceptrons. Prove (or argue informally) that a network of purely linear layers reduces to a single linear transformation regardless of depth.

8. Describe the four steps of the complete backpropagation training cycle. What does the error signal $\delta^{(l)}$ represent at hidden layer l, and how is it computed from the error signal of the layer above?

Section B — Analytical Problems

9. Given the linear discriminant $f(x_1, x_2) = 3x_1 - 2x_2 + 4$, answer the following:

(a) Write the equation of the decision boundary as a line in slope-intercept form.

(b) Classify the points $P_1 = (1, 3)$, $P_2 = (0, 2)$, $P_3 = (-1, 1)$. Show calculations.

(c) Compute the signed distance from P_1 to the decision boundary.

(d) Which point is most confidently classified, and which is closest to the boundary?

10. Trace the Perceptron Learning Algorithm on the following training dataset with learning rate $\eta = 1$. Start with $w = (0, 0)^T$ and $b = 0$, and show all epoch iterations until convergence:

(a) Data: $(2, 3) \rightarrow +1$; $(1, 1) \rightarrow +1$; $(3, 1) \rightarrow -1$; $(4, 3) \rightarrow -1$.

(b) For each misclassified point, explicitly show the weight update.

(c) Verify that the final weights correctly classify all four points.

11. An SVM finds the optimal hyperplane $2x_1 + 3x_2 - 6 = 0$ (so $w = (2,3)^T$, $b = -6$).

(a) Compute the width of the margin.

(b) The support vectors from Class +1 are at (1, 2) and from Class −1 at (3, 0). Verify that they lie on the margin boundaries $w^Tx + b = \pm 1$.

(c) A new point $x = (2, 2)$ is to be classified. Compute $f(x)$ and its signed distance to the boundary.

12. Using the polynomial kernel $K(x, z) = (x^Tz)^2$, derive the implicit feature map $\varphi(x_1, x_2)$ that this kernel corresponds to for 2D inputs. Verify your answer by confirming that $\varphi(x)^T\varphi(z) = K(x, z)$.

13. A logistic regression model has weights $w_1 = 1.5$, $w_2 = -2.0$ and bias $b = 0.8$.

(a) For input $x = (1.2, 0.8)$, compute the linear combination z, the probability $P(Y=1|x)$, and the predicted class.

(b) At what input value of x_2 (holding $x_1 = 1.2$ fixed) would the model output exactly $P(Y=1|x) = 0.5$?

(c) Compare the logistic output to the perceptron output for the same x. What information does logistic regression provide that the perceptron cannot?

14. Perform one complete iteration of backpropagation on the following network: one input $x = 2$, one hidden neuron (sigmoid activation, $w_1 = 0.5$, $b_1 = 0$), one output (linear, $w_2 = 1.0$, $b_2 = 0$), target $y = 0$, learning rate $\eta = 0.1$. Use MSE loss. Show the forward pass, loss, backward pass, and updated weights.

15. A regression dataset has $n = 4$ points: $(x, y) = \{(1, 2), (2, 3), (3, 5), (4, 6)\}$.

(a) Set up the design matrix X (with bias column) and the target vector y.

(b) Apply the normal equation $w^* = (X^TX)^{-1}X^Ty$ to find the optimal weights.

(c) Predict $\hat{y}$ for $x = 2.5$ and compute the residual.

Section C — Applied and Design Problems

16. You are building an email spam classifier using SVM. The feature vector contains 500 binary word-presence indicators (1 = word present, 0 = absent).

(a) Which kernel would you choose and why? Discuss Linear vs. RBF in this context.

(b) How would you select the regularization parameter C? Describe the complete cross-validation procedure.

(c) Your deployed model has precision = 0.95 and recall = 0.70. A product manager asks you to increase recall. What would you change in the model, and what trade-off are you making?

17. A medical imaging system must classify tissue samples as malignant or benign. Training data is highly imbalanced: 950 benign samples and 50 malignant samples.

(a) Explain why a linear discriminant trained with standard cross-entropy loss may predict 'benign' for everything and still achieve ~95% accuracy. What is wrong with this?

(b) Propose two concrete strategies — one involving the loss function and one involving data sampling — to address the imbalance.

(c) Should the classification threshold be set at 0.5 for this application? Justify your answer in terms of the relative costs of false positives and false negatives.

18. You are designing an MLP to predict house prices (a regression task). You have 15 input features including size, location, age, and number of rooms.

(a) Design the network architecture: how many layers, neurons per layer, and which activation functions would you use? Justify each choice.

(b) Which weight initialization scheme would you apply and why?

(c) Compare three optimization strategies: SGD with constant learning rate, SGD with momentum, and Adam. In what scenario might each be preferred?

(d) Describe how batch normalization modifies the training process and what benefits it provides for this regression task.

19. Compare the Perceptron, SVM, and Logistic Regression on the following dataset properties. For each combination, state which algorithm you would choose and provide a brief justification:

(a) Data: 10,000 features, 500 training examples, classes appear linearly separable.

(b) Data: 2 features, 10,000 training examples, classes form concentric rings.

(c) Data: 50 features, 2,000 training examples, 5% of Class 1 points are outliers in the Class 2 region.

(d) Data: 8 features, 300 training examples, downstream system requires calibrated probability estimates.

20. An MLP with two hidden layers is trained on image data (28×28 grayscale pixels → 784 inputs) to classify digits 0–9 (10 classes). Architecture: 784 → 256 (ReLU) → 128 (ReLU) → 10 (Softmax).

(a) Count the total number of trainable parameters in this network.

(b) During training, you observe that training loss decreases but validation loss increases after epoch 15. What is this phenomenon called, and name three regularization techniques to address it.

(c) Explain why He initialization is appropriate for ReLU hidden layers in this network. What problem would zero initialization cause, and what would large random initialization cause?

(d) If you replaced all ReLU activations with Sigmoid, what potential training problem might arise in the deep layers? Explain the mechanism.

CHAPTER V
Clustering

Chapter Overview

This chapter introduces one of the most fundamental paradigms in unsupervised machine learning: clustering. Unlike supervised methods that learn from labeled examples, clustering algorithms discover the inherent structure in unlabeled data by grouping objects such that similar items fall within the same cluster and dissimilar items are separated across clusters. The chapter builds systematically from conceptual foundations through a rich spectrum of algorithms, providing both mathematical rigor and intuitive worked examples for each technique.

The chapter begins by establishing why clustering matters, grounding the discussion in real-world domains such as customer segmentation, image analysis, document organization, anomaly detection, and bioinformatics. It then formalizes the notion of a good partition using objective functions such as Within-Cluster Sum of Squares (WCSS) and Between-Cluster Sum of Squares (BCSS), giving the reader precise tools to compare competing clustering's. A bridge to linear algebra is provided through Non-negative Matrix Factorization (NMF), which frames clustering as a matrix decomposition problem and naturally supports soft, probabilistic membership.

The chapter then presents the two major families of hierarchical clustering. Divisive (top-down) methods start with the full dataset and recursively split clusters, while agglomerative (bottom-up) methods start with individual points and progressively merge the nearest clusters. Four linkage criteria — single, complete, average, and Ward's method — are analyzed alongside their behavioral tradeoffs and visualized via dendrograms. The chapter then transitions to partitional clustering, where K-Means is covered in depth including its convergence guarantees, initialization sensitivity, and choice of k via the Elbow Method and Silhouette Score, and known limitations (sensitivity to outliers and restriction to spherical clusters).

Three advanced clustering paradigms address K-Means' limitations. Fuzzy C-Means introduces soft membership degrees, allowing boundary points to partially belong to multiple clusters. Rough K-Means brings in Rough Set Theory

to partition data into definite lower approximations and uncertain boundary regions without requiring graded membership values. Expectation-Maximization (EM) clustering adopts a fully probabilistic framework through Gaussian Mixture Models, performing soft assignment via Bayes' theorem and updating distributional parameters in alternating E and M steps. Finally, Spectral Clustering is introduced as a graph-theoretic technique that uses eigenvectors of the graph Laplacian to transform non-linearly separable data into a spectral embedding space where standard clustering algorithms succeed.

Learning Objectives	By the end of this chapter, the reader will be able to: (1) define clustering formally and distinguish it from supervised classification; (2) evaluate partition quality using WCSS, BCSS, Silhouette Score, and the Elbow Method; (3) apply NMF to discover latent cluster structure; (4) implement and compare the divisive among them and agglomerative hierarchical clustering with the appropriate linkage criteria; (5) run K-Means and select k intelligently; (6) apply soft and rough clustering to handle boundary uncertainty; (7) use EM-based Gaussian Mixture Models for probabilistic cluster assignment; and (8) recognize when Spectral Clustering is required and construct the spectral embedding pipeline.

Section-by-Section Roadmap

Section	Topic	Key Concepts
5.1	Introduction to Clustering	Unsupervised learning, intra/inter-cluster similarity, distance measures (Euclidean, Manhattan, Cosine), types of clustering
5.2	Partitioning of Data	Partition definition, WCSS, BCSS, cluster assignment function, evaluating partition quality
5.3	Matrix Factorization for Clustering	NMF, basis matrix W, coefficient matrix H, multiplicative update rules, soft cluster assignments
5.4	Clustering of Patterns	Feature space representation, centroid, medoid, prototype, statistical model
5.5	Divisive Clustering	Top-down hierarchy, splitting strategies (KMeans, principal direction, farthest point, graph-based), dendrogram
5.6	Agglomerative Clustering	Bottom-up hierarchy, single/complete/average linkage, Ward's method, dendrogram cutting

5.7	Partitional Clustering	Non-overlapping partitions, k must be specified, WCSS minimization, K-Means variants
5.8	K-Means Clustering	Assignment-update iteration, convergence, Elbow Method, Silhouette Score, limitations
5.9–5.10	Soft & Fuzzy C-Means	Membership degrees, fuzzifier m, weighted centroid update, soft vs. hard comparison
5.11–5.12	Rough Clustering	Rough Set Theory, lower/upper approximation, boundary region, Rough K-Means, threshold ε
5.13	EM-Based Clustering (GMM)	Gaussian Mixture Model, E-Step responsibilities, M-Step parameter updates, log-likelihood convergence
5.14	Spectral Clustering	Similarity graph, graph Laplacian, Fiedler vector, spectral embedding, non-convex cluster shapes

5.1 Introduction to Clustering

Clustering is one of the most fundamental tasks in **unsupervised learning**. Unlike supervised learning methods (such as classification and regression) where we train models using labeled data, clustering works with **unlabeled data**. The goal is to discover natural groupings or patterns that exist within the data without any prior knowledge of what those groups should be.

Clustering:

Clustering is the process of partitioning a set of data objects into subsets called clusters, such that:

- Objects within a cluster are highly similar to one another (high intra-cluster similarity)
- Objects in different clusters are highly dissimilar from each other (low inter-cluster similarity)

Formally, given a dataset$X = \{x_1, x_2, x_3, \dots, x_n\}$, clustering produces a partition $C = \{C_1, C_2, C_3, \dots, C_k\}$ where:

$$\bigcup_{i=1}^{k} C_i = X \text{ and } C_i \cap C_j = \emptyset \text{ for } i \neq j \text{ (in hard clustering)} \qquad (5.1)$$

Imagine you have a dataset of customer purchasing behaviors, but you don't know how to categorize your customers. Clustering algorithms can automatically identify groups of customers with similar behaviors perhaps one group buys luxury items, another focuses on discounts, and a third buys only essentials. This kind of automatic pattern discovery is incredibly valuable in many real-world applications.

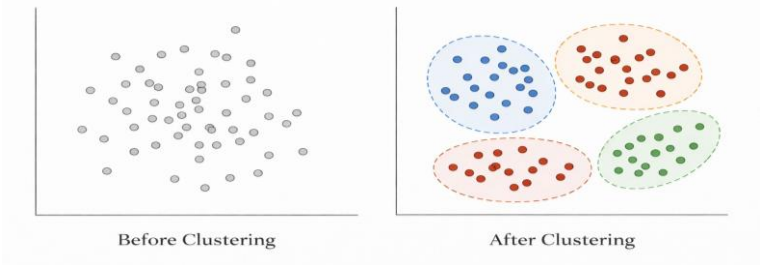


Figure 1: Introduction to Clustering

Figure 5.1: Introduction to Clustering: Unlabeled data points (left) are grouped into natural clusters (right) based on similarity. The algorithm discovers these groupings without prior labels.

Why is Clustering Important?

Clustering serves as a powerful tool for understanding the structure of data and has numerous practical applications:

- **Customer Segmentation:** Businesses use clustering to group customers based on purchasing behavior, demographics, or preferences. This enables targeted marketing strategies sending relevant promotions to each customer segment rather than generic messages to everyone.
- **Image Segmentation**: In computer vision, clustering helps partition an image into meaningful regions. Each cluster might represent a different object or part of the scene, which is essential for tasks like autonomous driving or medical image analysis.
- **Document Organization:** Search engines and digital libraries cluster documents by topic, allowing users to browse related content easily. News aggregators use clustering to group articles about the same event from different sources.
- **Anomaly Detection:** By identifying what normal clusters look like, clustering can help detect outliers data points that don't belong to any cluster. This is crucial for fraud detection, network intrusion detection, and quality control.
- **Biological Applications:** Clustering genes or proteins with similar expression patterns helps biologists understand biological processes, identify disease subtypes, and discover drug targets.

Similarity and Distance Measures:

The foundation of any clustering algorithm is the notion of similarity or distance between data points. Common distance measures include:

- **Euclidean Distance:** The Euclidean distance is the straight-line distance between two points in space.
For points $x = (x_1, \dots, x_d)$ and $y = (y_1, \dots, y_d)$:

$$d(x, y) = \sqrt{\sum_{i=1}^{d} (x_i - y_i)^2} = \| x - y \|_2 \qquad (5.2)$$

This is the most commonly used distance measure and works well when all features have similar scales.

- **Manhattan Distance:** The Manhattan distance is the sum of absolute differences along each dimension, like walking along city blocks:

$$d(x, y) = \sum_{i=1}^{d} | x_i - y_i | = \| x - y \|_1 \qquad (5.3)$$

More robust to outliers than Euclidean distance.

- **Cosine Similarity:** Cosine similarity measures the angle between two vectors, ignoring their magnitudes:

$$\text{similarity}(x, y) = \frac{x \cdot y}{\| x \| \ \| y \|} = \frac{\sum_{i=1}^{d} x_i y_i}{\sqrt{\sum_{i=1}^{d} x_i^2} \sqrt{\sum_{i=1}^{d} y_i^2}} \qquad (5.4)$$

Particularly useful for text data and high-dimensional sparse data where the direction matters more than the magnitude.

Interpretation: Customer A is much closer to Customer B (distance 24.5) than to Customer C (distance 218). A clustering algorithm would likely place A and B Computing Distance Between Data Points:

Consider two customers represented by their monthly spending on different categories:

Customer A: $\mathbf{x}_A$ = (200,50,100) (Electronics, Clothing, Food)

Customer B: $\mathbf{x}_B$ = (180,60,110)

Customer C: $\mathbf{x}_C$ = (50,200,150)

Euclidean Distance between A and B:

$$\begin{aligned} d(A, B) &= \sqrt{(200 - 180)^2 + (50 - 60)^2 + (100 - 110)^2} \\ &= \sqrt{400 + 100 + 100} = \sqrt{600} \approx 24.5 \end{aligned} \qquad (5.5)$$

Euclidean Distance between A and C:

$$\begin{aligned} d(A, C) &= \sqrt{(200 - 50)^2 + (50 - 200)^2 + (100 - 150)^2} \\ &= \sqrt{22500 + 22500 + 2500} = \sqrt{47500} \approx 218 \end{aligned} \qquad (5.6)$$

in the same cluster, while C would be in a different cluster. This makes intuitive sense A and B have similar spending patterns (high electronics, low clothing), while C has a very different pattern (low electronics, high clothing).

Types of Clustering Approaches:

Clustering algorithms can be broadly categorized into several families:

1. **Partitional Clustering:** Divides data into non-overlapping clusters in a single step. Each data point belongs to exactly one cluster. Examples: KMeans, K-Medoids.
2. **Hierarchical Clustering:** Creates a tree-like hierarchy of clusters. Can be agglomerative (bottom-up) or divisive (top-down). The result is a dendrogram showing nested cluster structures.
3. **Density-Based Clustering:** Forms clusters based on dense regions of points separated by sparse regions. Can discover clusters of arbitrary shapes. Examples: DBSCAN, OPTICS.
4. **Soft (Fuzzy) Clustering:** Allows data points to belong to multiple clusters with varying degrees of membership. Examples: Fuzzy C-Means, Gaussian Mixture Models.
5. **Spectral Clustering:** Uses eigenvalues of similarity matrices to reduce dimensionality before clustering. E ective for non-convex clusters.

5.2 Partitioning of Data

Partitioning is the process of dividing a dataset into distinct, non-overlapping groups. In the context of clustering, a partition assigns each data point to exactly one cluster (in hard clustering) or distributes membership across multiple clusters (in soft clustering).

The quality of a partition depends on how well it captures the natural structure of the data. A good partition should maximize the similarity within clusters while minimizing the similarity between clusters.

Partition of a Dataset:

A **partition** of a dataset $X = \{x_1, x_2, \dots, x_n\}$ into k clusters is a collection into

$\mathcal{P} = \{C_1, C_2, \dots, C_k\}$ satisfying:

1. **Coverage**: Every data point belongs to at least one cluster: $\bigcup_{i=1}^{k} C_i = X$
2. **Non-emptiness:** No cluster is empty: $C_i \neq \emptyset$ for all i
3. **Non-overlap** (for hard partitions): Clusters don't share points:

$$C_i \cap C_j = \emptyset \text{ for } i \neq j$$

The partition can be represented by a **cluster assignment function**:

$$\gamma: X \to \{1,2, \dots, k\}$$

where $\gamma(x_i) = j$ means that point $x_i$is assigned to cluster C_j.

Measuring Partition Quality:

How do we know if one partition is better than another? Several objective functions quantify partition quality:

Within-Cluster Sum of Squares (WCSS): Also called inertia, this measures the total squared distance from each point to its cluster center:

$$\text{WCSS} = \sum_{i=1}^{k} \sum_{x \in C_i} \| x - \mu_i \|^2 \qquad (5.7)$$

where $\mu_i = \frac{1}{|C_i|} \sum_{x \in C_i} x$ is the centroid (mean) of cluster C_i. A lower WCSS indicates tighter, more compact clusters.

Between-Cluster Sum of Squares (BCSS): Measures how spread out the cluster centers are from the global mean:

$$\text{BCSS} = \sum_{i=1}^{k} | C_i | \ \| \mu_i - \mu \|^2 \qquad (5.8)$$

where μ is the global mean of all data points. A higher BCSS indicates well-separated clusters.

The Clustering Objective: The ideal partition minimizes WCSS (compact clusters) while maximizing BCSS (separated clusters). Since total variance is constant (Total SS = WCSS + BCSS), minimizing WCSS is equivalent to maximizing BCSS.

Evaluating Two Partitions:

Consider 6 data points in 1D: X = {1,2,3,10,11,12}. Compare two partitions into 2 clusters:

Partition A: C_1 = {1,2,3}, C_2 = {10,11,12}

Centroid of C_1: $\mu_1 = (1 + 2 + 3)/3 = 2$

Centroid of C_2: $\mu_2 = (10 + 11 + 12)/3 = 11$

WCSS = $[(1-2)^2 + (2-2)^2 + (3-2)^2] + [(10-11)^2 + (11-11)^2 + (12-11)^2]$

WCSS = [1 + 0 + 1] + [1 + 0 + 1] = 4

Partition B: C_1 = {1,2,10}, C_2 = {3,11,12}

Centroid of C_1: $\mu_1 = (1 + 2 + 10)/3 = 4.33$

Centroid of C_2: $\mu_2 = (3 + 11 + 12)/3 = 8.67$

WCSS = $[(1-4.33)^2 + (2-4.33)^2 + (10-4.33)^2] + [(3-8.67)^2 + (11-8.67)^2 + (12 - 8.67)^2]$

WCSS = [11.1 + 5.4 + 32.1] + [32.1 + 5.4 + 11.1] = 97.2

Conclusion: Partition A (WCSS = 4) is vastly better than Partition B (WCSS = 97.2). Partition A correctly identifies the two natural groups of nearby numbers, while Partition B mixes points from both groups.

5.3 Matrix Factorization for Clustering

Matrix factorization provides an elegant mathematical framework for clustering. The key idea is to decompose a data matrix into lower-rank factors that reveal the underlying cluster structure. This approach connects clustering to linear algebra and provides powerful computational techniques.

The most commonly used matrix factorization for clustering is **Non-negative Matrix Factorization (NMF)**, which is particularly well-suited for data with non-negative values (like word counts, pixel intensities, or customer ratings).

Non-negative Matrix Factorization (NMF):

Given a non-negative data matrix $X \in \mathbb{R}_{\geq 0}^{m \times n}$ (where m is the number of features and n is the number of data points), NMF finds two non-negative matrices:

$$X \approx WH \qquad (5.9)$$

where:

$\boldsymbol{W} \in \mathbb{R}_{\geq \mathbf{0}}^{\boldsymbol{m} \times \boldsymbol{k}}$ is the **basis matrix** (or dictionary), with k basis vectors representing cluster centroids or prototypes

$\boldsymbol{H} \in \mathbb{R}_{\geq 0}^{k \times n}$ is the **coefficient matrix**, where each column $\mathbf{h}_j$ gives the representation of data point j in terms of the k basis vectors

$k \ll \min(m, n)$ is the target rank, typically equal to the number of clusters

The non-negativity constraints encourage parts-based representations each data point is expressed as an additive combination of non-negative components.

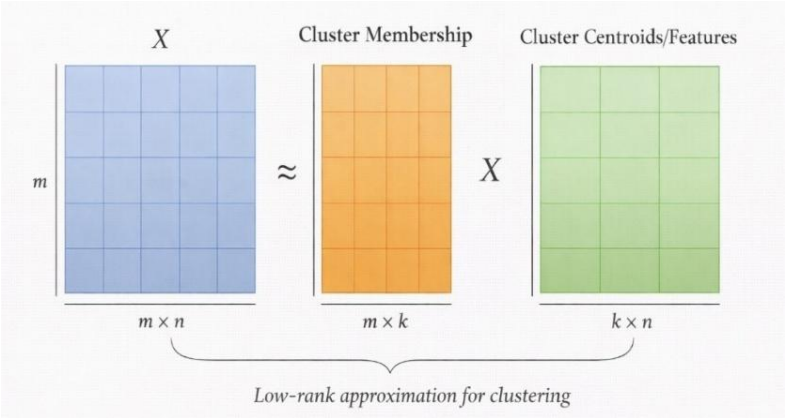


Figure 5.2: Matrix Factorization for Clustering

Figure 5.2: Matrix Factorization for Clustering: The data matrix X is approximately factorized into W (basis/centroids) and H (cluster membership weights). Each column of H indicates how strongly each data point belongs to each cluster.

How NMF Relates to Clustering:

In the context of clustering, NMF can be interpreted as follows:

- Each column of **W** represents a cluster centroid or prototype
- Each column of **H** represents the soft cluster membership of a data point
- To get hard cluster assignments, assign each point to the cluster with the highest weight:

$$\gamma(j) = \arg\max_{i} H_{ij}$$

NMF Objective Function:

NMF finds **W** and **H** by minimizing the reconstruction error:

$$\min_{W \geq 0,\, H \geq 0} \| X - WH \|_F^2 = \sum_{i,j} \left(X_{ij} - (WH)_{ij}\right)^2 \qquad (5.10)$$

where $\|\cdot\|_F$ is the Frobenius norm. This optimization is typically solved using multiplicative update rules or alternating least squares.

Multiplicative Update **Rules:**

$$H_{ij} \leftarrow H_{ij} \cdot \frac{(W^T X)_{ij}}{(W^T W H)_{ij}} \qquad (5.11)$$

$$W_{ij} \leftarrow W_{ij} \cdot \frac{(X H^T)_{ij}}{(W H H^T)_{ij}} \qquad (5.12)$$

These updates are applied iteratively until convergence, guaranteeing that the objective function decreases at each step.

NMF for Document Clustering: Consider a term-document matrix with 3 documents and 4 terms (words):

$$X = \begin{pmatrix} 5 & 0 & 4 \\ 3 & 0 & 2 \\ 0 & 4 & 1 \\ 0 & 5 & 0 \end{pmatrix} \quad \begin{matrix} \leftarrow \text{“algorithm”} \\ \leftarrow \text{“data”} \\ \leftarrow \text{“music”} \\ \leftarrow \text{“song”} \end{matrix} \qquad (5.13)$$

Columns represent documents; entries are word frequencies.

Applying NMF with k=2 clusters, we might Obtain:

$$W = \begin{pmatrix} 5 & 0 \\ 3 & 0 \\ 0 & 4 \\ 0 & 5 \end{pmatrix}, H = \begin{bmatrix} 1 & 0 & 0.8 \\ 0 & 1 & 0.2 \end{bmatrix} \qquad (5.14)$$

Interpretation:

Column 1 of **W**: Topic about computing (high algorithm, data ; zero music , song)

Column 2 of **W**: Topic about music (zero algorithm , data ; high music , song)

From **H**: Document 1 is 100% computing topic, Document 2 is 100% music topic, Document 3 is 80% computing + 20% music (a mixed document!)

Cluster Assignments: Assign each document to its dominant topic Documents 1 and 3 to the computing cluster, Document 2 to the music cluster.

5.4 Clustering of Patterns

In pattern recognition and machine learning, **clustering of patterns** refers to organizing data samples (patterns) into groups based on their feature representations. This is the practical application of clustering theory to real-world datasets where each data point is described by a set of features.

A **pattern** is a representation of an object in feature space. For example, an image might be represented as a vector of pixel intensities, a document as a vector of word frequencies, or a customer as a vector of behavioral attributes.

The goal of pattern clustering is to discover natural groupings among these feature vectors.

Feature Space Representation:

Every pattern is represented as a point in a d-dimensional feature space:

$$\mathbf{x} = (x_1, x_2, \ldots, x_d)^T \in \mathrm{R}^d \qquad (5.15)$$

The choice of features and their representation signi cantly impacts clustering quality.

Good features should:

- Capture the essential characteristics that distinguish different patterns
- Be invariant to irrelevant transformations (e.g., lighting changes for images)
- Have similar scales across dimensions (or be normalized appropriately)

Cluster Representation:

A cluster can be represented in several ways:

- **Centroid:** The mean of all patterns in the cluster: $\mu = \frac{1}{|C|}\sum_{x \in C} x$
- **Medoid:** An actual pattern that minimizes total distance to other patterns in the cluster
- **Prototype Set:** Multiple representative patterns that together characterize the cluster
- **Statistical Model:** A probability distribution (e.g., Gaussian) that describes the cluster

Clustering Iris Flower Patterns:

The famous Iris dataset contains 150 flowers with 4 features each: sepal length, sepal width, petal length, and petal width.

Sample patterns:

Pattern	Sepal L	Sepal W	Petal L	Petal W
x_1	5.1	3.5	1.4	0.2
x_2	4.9	3.0	1.4	0.2
x_{75}	6.4	2.9	4.3	1.3
x_{125}	6.7	3.3	5.7	2.1

After clustering into k=3 groups, we might find:

Cluster 1 (small flowers): Patterns like $\mathbf{x}_1$, $\mathbf{x}_2$ with small petals

Cluster 2 (medium flowers): Patterns like $\mathbf{x}_{75}$ with medium petals

Cluster 3 (large flowers): Patterns like $\mathbf{x}_{125}$ with large petals

Interestingly, these clusters correspond closely to the three Iris species (Setosa, Versicolor, Virginica), demonstrating that clustering can discover meaningful biological groupings from the feature data alone!

5.5 Divisive Clustering

Divisive clustering (also called **top-down clustering**) is a hierarchical approach that starts with all data points in a single cluster and recursively splits it into smaller clusters. At each step, the algorithm selects a cluster to divide and determines the best way to split it into two (or more) sub-clusters.

This approach is intuitive when thinking about taxonomy: start with all items as one category, then progressively divide into subcategories based on distinguishing characteristics.

Divisive Hierarchical Clustering:

Divisive clustering constructs a hierarchy by repeatedly dividing clusters:

1. **Initialize:** Start with all *n* data points in a single cluster $C = X$
2. **Select**: Choose a cluster to split (usually the largest or most heterogeneous)
3. **Divide**: Split the selected cluster into two sub-clusters using some criterion
4. **Repeat**: Continue steps 2-3 until each data point is its own cluster (or stopping criterion is met)

The result is a binary tree (dendrogram) where the root contains all points and leaves contain individual points.

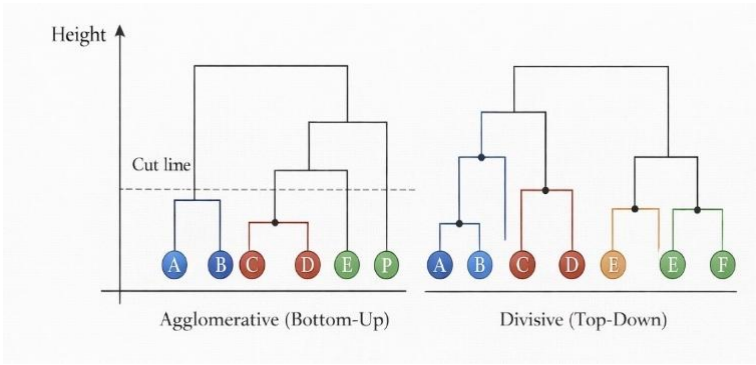


Figure 5.3: Hierarchical Clustering Dendrograms

Figure 5.3: Hierarchical Clustering Dendrograms: (Left) Agglomerative clustering builds bottom-up by merging similar points. (Right) Divisive clustering works top-down by splitting clusters. The horizontal cut line determines the nal number of clusters.

Splitting Strategies:

The key decision in divisive clustering is how to split a cluster. Common approaches include:

- **Use a Partitional Algorithm:** Apply K-Means with k = 2 to split each cluster into two parts. This is simple and effective but requires multiple runs to find a good split.
- **Principal Direction Split:** Find the principal component (direction of maximum variance) and split the cluster by a hyperplane perpendicular to it. Points on one side form one sub-cluster, points on the other side form another.
- **Farthest Point Split**: Find the two points farthest apart in the cluster. Use these as seeds and assign remaining points to the nearest seed, creating two sub-clusters.
- **Graph-Based Split:** Construct a graph where edges connect similar points. Find a minimum cut that separates the graph into two components with minimal edge weight between them.

Selecting Which Cluster to Split:

When multiple clusters exist, the algorithm must choose which one to split next.

Common criteria:

- **Largest Cluster:** Split the cluster with the most points
- **Highest Variance:** Split the cluster with the highest internal variance (most heterogeneous)
- **Largest Diameter:** Split the cluster where the farthest pair of points is most distant

Divisive Clustering Process:

Consider 6 points: A(1,1), B(2,1), C(1,2), D(8,8), E(9,8), F(8,9)

Step 1: Start with all points in one **cluster:**

$$C_1 = \{A, B, C, D, E, F\}$$

Step 2: Split C_1 (nd two groups). The farthest points are C(1,2) and E(9,8).

Assigning points to nearest seed:

Near **C:** $A(1,1)$, $B(2,1)$, $C(1,2) \rightarrow C_2$

Near **E:** $D(8,8)$, $E(9,8)$, $F(8,9) \rightarrow C_3$

Step 3: Choose to split C_2 (or C_3). Split C_2 using farthest points A and C:

Near **A:** $A(1,1)$, $B(2,1) \rightarrow C_4$

Near **C:** $C(1,2) \rightarrow C_5$

Continue until each point is its own cluster (or stop earlier for desired k).

Dendrogram: The tree shows: at the top is {*A,B,C,D,E,F*}; it splits into {*A,B,C*} and {*D,E,F*}; then {*A,B,C*} splits into {*A,B*} and {*C*}; and so on.

Advantages and Disadvantages of Divisive Clustering:

Advantages:

- More efficient when you want a small number of clusters (fewer splits needed)
- Can use global information about cluster structure when making splits
- Natural for applications where you want to understand the major divisions first

Disadvantages:

- Splitting decisions are difficult and computationally expensive
- Once a split is made, it cannot be undone (greedy approach)
- Less commonly used than agglomerative methods due to complexity

5.6 Agglomerative Clustering

Agglomerative clustering (also called **bottom-up clustering**) is the opposite of divisive clustering. It starts with each data point as its own cluster and progressively merges the most similar pairs of clusters until only one cluster remains (or a stopping criterion is met).

This is the most widely used hierarchical clustering method due to its conceptual simplicity and availability of efficient algorithms.

Agglomerative Hierarchical Clustering:

Agglomerative clustering builds a hierarchy by iteratively merging clusters:

1. **Initialize:** Each data point starts as its own singleton cluster:
   $$C_i = \{x_i\} \text{ for } i = 1,...,n$$
2. **Compute Distances:** Calculate the distance (or dissimilarity) between all pairs of clusters
3. **Merge:** Find the two closest clusters and merge them into a single cluster
4. **Update:** Recalculate distances involving the new merged cluster
5. **Repeat:** Continue steps 3-4 until only one cluster remains
   The algorithm produces a dendrogram showing the sequence of merges and the distances at which they occurred.

Linkage Criteria:

The crucial decision in agglomerative clustering is how to measure the distance between two clusters (each potentially containing multiple points). Different linkage criteria lead to different clustering behaviours:

Single Linkage (Minimum): Distance between two clusters is the distance between their closest points:

$$d(C_i, C_j) = \min_{x \in C_i,\, y \in C_j} d(x, y) \qquad (5.16)$$

Tends to create long, chain-like clusters. Good for discovering clusters of arbitrary shapes but sensitive to noise.

Complete Linkage (Maximum): Distance between two clusters is the distance between their farthest points:

$$d(C_i, C_j) = \max_{x \in C_i,\, y \in C_j} d(x, y) \qquad (5.17)$$

Tends to create compact, roughly spherical clusters. More robust to outliers than single linkage.

Average Linkage: Distance is the average of all pairwise distances between points in the two clusters:

$$d(C_i, C_j) = \frac{1}{|\, C_i \,| \cdot |\, C_j \,|} \sum_{x \in C_i} \sum_{y \in C_j} d(x, y) \qquad (5.18)$$

A compromise between single and complete linkage; produces moderately compact clusters.

Ward's Method: Merges clusters that result in the minimum increase in total within-cluster variance:

$$d(C_i, C_j) = \frac{|\, C_i \,| \cdot |\, C_j \,|}{|\, C_i \,| + |\, C_j \,|} \; \| \mu_i - \mu_j \|^2 \qquad (5.19)$$

Tends to create clusters of similar sizes; minimizes the same objective as KMeans.

Agglomerative Clustering Step-by-Step:

Consider 5 points with the following Euclidean distance matrix:

	A	B	C	D	E
A	0	2	6	10	9
B	2	0	5	9	8
C	6	5	0	4	5
D	10	9	4	0	3
E	9	8	5	3	0

Using single linkage:

Step 1: Initial clusters: $\{A\},\{B\},\{C\},\{D\},\{E\}$.

Find minimum distance: $d(A,B) = 2$

Step 2: Merge A and B → $\{AB\}$. Update distances to $\{AB\}$:

$d(\{AB\}, C) = \min\big(d(A,C), d(B,C)\big) = \min(6,5) = 5$

$d(\{AB\}, D) = \min\,(d(A,D), d(B,D)) = \min\,(10,9) = 9$

$d(\{AB\}, E) = \min\big(d(A,E), d(B,E)\big) = \min(9,8) = 8$

Clusters: $\{AB\},\{C\},\{D\},\{E\}$. Next minimum: $d(D,E) = 3$

Step 3: Merge D and E → $\{DE\}$. Clusters: $\{AB\},\{C\},\{DE\}$. Next minimum: $d(C,DE) = \min(4,5) = 4$

Step 4: Merge C and DE → $\{CDE\}$. Clusters: $\{AB\},\{CDE\}$. Next minimum: $d(AB,CDE) = \min(5,8,9) = 5$

Step 5: Merge AB and CDE → $\{ABCDE\}$. Done!

Dendrogram heights: 2 (merge AB), 3 (merge DE), 4 (merge CDE), 5 (merge all).

To get 2 clusters: Cut the dendrogram at height between 4 and 5 → $\{A,B\}$ and $\{C,D,E\}$.

Cutting the Dendrogram:

The dendrogram represents all possible clusterings from n clusters (each point alone) to 1 cluster (all points together). To obtain k clusters, we cut the dendrogram at an appropriate height this height determines which merges to accept and which to reject.

All clusters that exist below the cut line become the final clusters.

5.7 Partitional Clustering

Partitional clustering algorithms directly partition the dataset into k non-overlapping clusters in a single step (rather than creating a hierarchy). These methods require specifying the number of clusters k in advance and aim to optimize some objective function that measures cluster quality.

The most famous partitional algorithm is K-Means, but the family includes many variants designed for different data types and objectives.

Partitional Clustering:

A partitional clustering algorithm takes a dataset $X = \{x_1,...,x_n\}$ and a target number of clusters k, and produces a partition $P = \{C_1,...,C_k\}$ that optimizes some objective function $J(P)$.

Common objective: Minimize the within-cluster sum of squares **WCSS**

$$J(P) = \sum_{i=1}^{k} \sum_{x \in C_i} \| x - \mu_i \|^2 \qquad (5.20)$$

where μ_i is the centroid of cluster C_i.

Characteristics of Partitional Methods:

- **Require *k* in advance:** The user must specify the number of clusters before running the algorithm. Choosing the right k is often challenging and may require running the algorithm multiple times with di erent values.
- **Iterative refinement:** Most partitional methods start with an initial partition (often random) and iteratively improve it until convergence. They typically nd a local optimum, not necessarily the global optimum.
- **Scalability:** Generally more scalable than hierarchical methods, especially for large datasets. K-Means, for example, has time complexity $O(nkdi)$ where n is the number of points, k is the number of clusters, d is the dimensionality, and i is the number of iterations.
- **Sensitivity to initialization:** The final result often depends on the initial cluster centers. Running the algorithm multiple times with different initializations and keeping the best result is a common practice.

Variants of Partitional Clustering:

- **K-Means:** Uses cluster centroids (means); sensitive to outliers
- **K-Medoids:** Uses actual data points (medoids) as cluster representatives; more robust to outliers
- **K-Modes:** For categorical data; uses modes instead of means

- **K-Medians:** Uses median along each dimension; more robust to outliers than K-Means

5.8 K-Means Clustering

K-Means is the most widely used clustering algorithm due to its simplicity, efficiency, and effectiveness. It partitions data into k clusters by iteratively assigning points to the nearest cluster center and then updating the centers to be the mean of assigned points.

K-Means Algorithm:

Given a dataset $X = \{\mathbf{x}_1,...,\mathbf{x}_n\}$ and the number of clusters k:

Objective: Minimize the Within-Cluster Sum of Squares **WCSS**

$$J = \sum_{i=1}^{k} \sum_{x \in C_i} \| x - \mu_i \|^2 \qquad (5.21)$$

Algorithm:

Initialize: Randomly select k data points as initial cluster centroids $\mu_1,...,\mu_k$
Assign: For each data point $\mathbf{x}_j$, assign it to the nearest centroid

$$\gamma(j) = \arg \min_{i \in \{1,2,...,k\}} \| x_j - \mu_i \|^2 \qquad (5.22)$$

Update: Recalculate each centroid as the mean of its assigned it:

$$\mu_i = \frac{1}{| C_i |} \sum_{x \in C_i} x \qquad (5.23)$$

Repeat: Go to step 2 until centroids no longer change (or change is below a threshold)

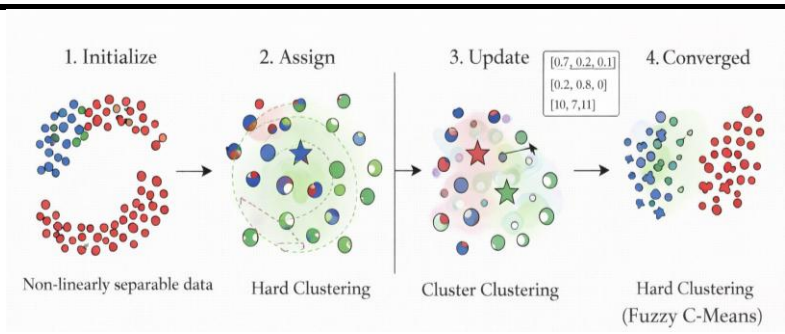


Figure 5.4: K-Means Clustering Algorithm

Figure 5.4: K-Means Clustering Algorithm: (1) Initialize random centroids, (2) Assign each point to nearest centroid, (3) Update centroids to cluster means, (4) Repeat until convergence. The algorithm alternates between assignment and update steps.

K-Means Clustering Step-by-Step: Consider 6 points in 2D: $A(1,1), B(1,2), C(2,1), D(8,8), E(9,8), F(8,9)$. Cluster into $k = 2$ groups.

Iteration 1:

Step 1 - Initialize: Randomly choose $A(1,1)$ and $D(8,8)$ as initial centroids.

Step 2 - Assign: Calculate distances from each point to both centroids:

Point	d to $\mu_1(1,1)$	d to $\mu_2(8,8)$	Assignment
A(1,1)	0	9.90	Cluster 1
B(1,2)	1	9.22	Cluster 1
C(2,1)	1	9.22	Cluster 1
D(8,8)	9.90	0	Cluster 2
E(9,8)	10.63	1	Cluster 2
F(8,9)	10.63	1	Cluster 2

$C_1 = \{A,B,C\}$, $C_2 = \{D,E,F\}$

Step 3 - Update Centroids

$$\mu_1 = \frac{1}{3}[(1,1) + (1,2) + (2,1)] = \left(\frac{4}{3}, \frac{4}{3}\right) \approx (1.33,1.33) \qquad (5.24)$$

$$\mu_2 = \frac{1}{3}[(8,8) + (9,8) + (8,9)] = \left(\frac{25}{3}, \frac{25}{3}\right) \approx (8.33,8.33) \qquad (5.25)$$

Iteration 2:

Step 2 - Assign: With new centroids (1.33,1.33) and (8.33,8.33):

All points in Cluster 1 are still closer to μ_1

All points in Cluster 2 are still closer to μ_2

Assignments unchanged!

Step 3 - Update: Centroids remain the same since assignments didn't change.

Convergence: The algorithm has converged. Final clusters:

Cluster 1: $\{A,B,C\}$ with centroid (1.33,1.33)

Cluster 2: $\{D,E,F\}$ with centroid (8.33,8.33)

Final WCSS

$$\begin{aligned} J &= [(1-1.33)^2 + (1-1.33)^2] + [(1-1.33)^2 + (2-1.33)^2] \\ &+[(2-1.33)^2 + (1-1.33)^2] + [(8-8.33)^2 + (8-8.33)^2] \\ &+[(9-8.33)^2 + (8-8.33)^2] + [(8-8.33)^2 + (9-8.33)^2] \\ &= 0.22 + 0.56 + 0.56 + 0.22 + 0.56 + 0.56 = 2.68 \end{aligned} \qquad (5.28)$$

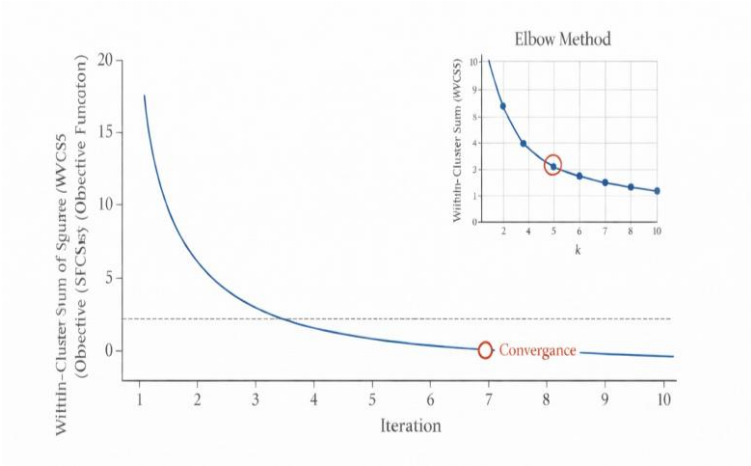


Figure 5.5: K-Means Convergence

Figure 5.5: K-Means Convergence: The objective function (WCSS) decreases with each iteration until convergence. The inset shows the Elbow Method for choosing *k*look for the elbow where adding more clusters provides diminishing returns.

Choosing the Number of Clusters *k*:

One of the main challenges with K-Means is selecting the appropriate number of clusters. Several methods can help:

- **Elbow Method:** Plot WCSS against *k* for *k* = 1,2,3,…. The WCSS always decreases as *k* increases, but look for an elbow a point where the rate of decrease sharply changes. The *k* at the elbow is often a good choice.
- **Silhouette Score:** For each point, compute how similar it is to its own cluster compared to other clusters. The silhouette coe cient ranges from -1 to 1; higher values indicate better-de ned clusters. Choose *k* that maximizes the average silhouette score.
- **Gap Statistic:** Compares the within-cluster dispersion to that expected under a null reference distribution. Choose *k* that maximizes the gap.

K-Means Limitations:

- **Assumes spherical clusters:** Works best when clusters are roughly spherical and similar in size
- **Sensitive to outliers**: A single outlier can significantly shift a centroid
- **Requires specifying *k*:** Must know the number of clusters in advance

- **Sensitive to initialization:** Different starting points can lead to different results
- **Finds local optima:** Not guaranteed to find the global minimum of WCSS

K-Means++ Initialization:

To address initialization sensitivity, **K-Means++** selects initial centroids more carefully:

1. Choose the first centroid uniformly at random from the data points
2. For each subsequent centroid, choose a point with probability proportional to its squared distance from the nearest existing centroid
3. Repeat until *k* centroids are selected

This spreads out the initial centroids and typically leads to better and more consistent results.

5.9 Soft Partitioning and Soft Clustering

In **hard clustering** (like K-Means), each data point belongs to exactly one cluster. However, in many real-world scenarios, data points may genuinely belong to multiple clusters to varying degrees. Soft clustering (also called fuzzy clustering) allows this by assigning each point a membership degree in every cluster, rather than a single hard assignment.

Soft Clustering:

In soft clustering, each data point $\mathbf{x}_j$ has a membership **value** $u_{ij} \in [0,1]$ for each cluster C_i, where C_i represents the **degree to which point** x_j belongs to cluster

$u_{ij} = 1 \rightarrow$ full membership

$u_{ij} = 0 \rightarrow$ no membership

Membership values for each data point satisfy:

$$\sum_{i=1}^{k} u_{ij} = 1 \text{ for all } j$$

This allows points near cluster boundaries to have partial membership in multiple clusters, reflecting uncertainty in their assignment.

Why Use Soft Clustering?

Hard clustering forces every point into exactly one cluster, but this may not re ect reality:

- A news article about tech companies and stock markets could belong to both the Technology cluster and the Finance cluster
- A pixel at the boundary between sky and mountain in an image could belong partially to both regions
- A customer might have characteristics of multiple market segments

Soft clustering acknowledges this ambiguity by providing graded memberships. A point with membership (0.7,0.3,0.0) clearly belongs mostly to cluster 1, somewhat to cluster 2, and not at all to cluster 3.

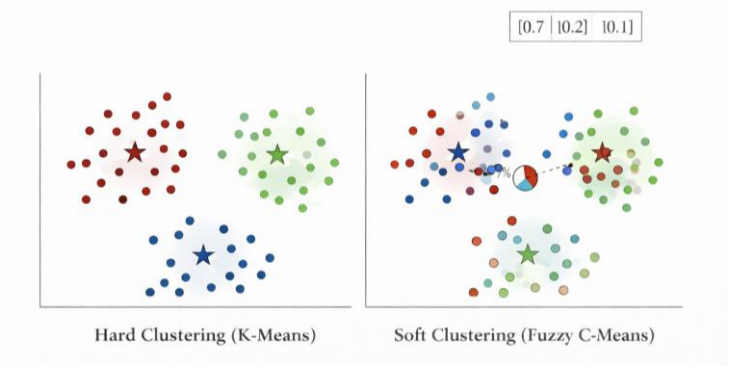


Figure 5.6: Hard vs. Soft Clustering

Figure 5.6: Hard vs. Soft Clustering: In hard clustering (left), each point belongs to exactly one cluster. In soft clustering (right), points have graded memberships points near boundaries have mixed memberships shown by colour blending or pie charts.

Membership Matrix:

The result of soft clustering is often represented as a membership **matrix** U of size $k \times n$:

$$U = \begin{pmatrix} u_{11} & u_{12} & \cdots & u_{1n} \\ u_{21} & u_{22} & \cdots & u_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ u_{k1} & u_{k2} & \cdots & u_{kn} \end{pmatrix} \qquad (5.29)$$

Each column sums to 1, and u_{ij} tells us how much point j belongs to cluster i.

5.10 Fuzzy C-Means Clustering

Fuzzy C-Means (FCM) is the most widely used soft clustering algorithm. It generalizes K-Means by allowing soft membership values, controlled by a **fuzziness parameter** m that determines how fuzzy the cluster boundaries are.

Fuzzy C-Means Algorithm:

Given data $X = \{\mathbf{x}_1,...,\mathbf{x}_n\}$, number of clusters k, and fuzziness parameter $m > 1$:

Objective: Minimize the fuzzy within-cluster sum of squares:

$$J_m = \sum_{i=1}^{k} \sum_{j=1}^{n} u_{ij}^m \parallel x_j - \mu_i \parallel^2 \qquad (5.30)$$

Subject to:

$$\sum_{i=1}^{k} u_{ij} = 1 \; for \; all \; j, u_{ij} \geq 0$$

Algorithm:

1. **Initialize:** Randomly initialize the membership matrix **U** (satisfying constraints)
2. **Update Centroids:** Calculate cluster centers as weighted means:

$$\mu_i = \frac{\sum_{j=1}^{n} u_{ij}^m \; x_j}{\sum_{j=1}^{n} u_{ij}^m} \qquad (5.31)$$

3. **Update Memberships:** For each point and cluster:

$$u_{ij} = \frac{1}{\sum_{\ell=1}^{k} \left(\frac{\parallel x_j - \mu_i \parallel}{\parallel x_j - \mu_\ell \parallel} \right)^{\frac{2}{m-1}}} \qquad (5.32)$$

4. **Repeat:** Continue steps 2-3 until convergence (memberships stop changing significantly)

The **Fuzziness Parameter** m:

The parameter $m > 1$ controls how fuzzy the clustering is:

- As m → 1: FCM approaches hard K-Means clustering. Memberships become 0 or 1.

Fuzzy C-Means Clustering:

Consider 4 points: $P_1(1,1)$, $P_2(2,1)$, $P_3(6,6)$, $P_4(7,6)$ with $k = 2$ clusters and $m = 2$.

Initial (random) centroids: $\mu_1 = (1.5,1)$, $\mu_2 = (6.5,6)$

Update Memberships:

For $P_1(1,1)$:

$$d_1 = \| P_1 - \mu_1 \| = \| (1,1) - (1.5,1) \| = 0.5 \quad (5.33)$$

$$d_2 = \| P_1 - \mu_2 \| = \| (1,1) - (6.5,6) \| = 7.43 \quad (5.34)$$

$$u_{11} = \frac{1}{1 + \left(\frac{0.5}{7.43}\right)^2} = \frac{1}{1 + 0.0045} \approx 0.996 \quad (5.35)$$

$$u_{21} = 1 - u_{11} \approx 0.004 \quad (5.36)$$

P_1 is 99.6% in cluster 1, 0.4% in cluster 2 (very crisp assignment it's far from cluster 2).

For P_3(6,6) (similarly):

$$u_{13} \approx 0.004, u_{23} \approx 0.996 \quad (5.37)$$

P_3 is 99.6% in cluster 2 (equally crisp).

For a point at (3.5,3.5) (hypothetically, equidistant from both clusters):

$$u_1 = u_2 = 0.5 \quad (5.38)$$

This point would have 50% membership in each cluster maximum fuzziness.

Membership Matrix

$$U \approx \begin{pmatrix} 0.996 & 0.98 & 0.004 & 0.02 \\ 0.004 & 0.02 & 0.996 & 0.98 \end{pmatrix} \quad (5.39)$$

Update Centroids: Using the weighted formula with these memberships, we get new centroids close to the means of each group.

To get hard assignments: Assign each point to the cluster with highest membership: P_1, $P_2 \rightarrow$ Cluster 1; P_3, $P_4 \rightarrow$ Cluster 2.

FCM vs. K-Means:

Aspect	K-Means	Fuzzy C-Means
Membership type	Hard (0 or 1)	Soft (0 to 1)
Boundary handling	Points forced to one cluster	Partial membership at boundaries
Centroid calculation	Simple mean	Weighted mean (by u^m_{ij})
Output	Cluster labels	Membership matrix
Interpretability	Simpler	Richer (shows uncertainty)
Computation	Slightly faster	Slightly slower

5.11 Rough Clustering

Rough clustering is based on Rough Set Theory, developed by Zdzislaw Pawlak in 1982. It provides a way to handle uncertainty and vagueness in cluster boundaries by defining clusters using two approximations: a **lower approximation** (definitely in the cluster) and an **upper approximation** (possibly in the cluster).

Unlike fuzzy clustering which assigns graded memberships, rough clustering divides the data space into three regions for each cluster: de nitely inside, de nitely outside, and uncertain (boundary region).

Rough Set Approximations in Clustering:

For a cluster C, rough set theory defines:

Lower Approximation $\bar{C}$: The set of points that **definitely belong** to cluster C. These points are close to the cluster center and far from other clusters.

Upper Approximation $\bar{C}$: The set of points that **possibly belong** to cluster C. This includes the lower approximation plus boundary points.

Boundary Region $BR(C) = \bar{C} - \underline{C}$: Points that may or may not belong to Cthey are in the uncertain zone between clusters.

Properties:

$$\underline{C} \subseteq C \subseteq \bar{C} \qquad (5.40)$$

$$BR(C) = \bar{C} - \underline{C} \qquad (5.41)$$

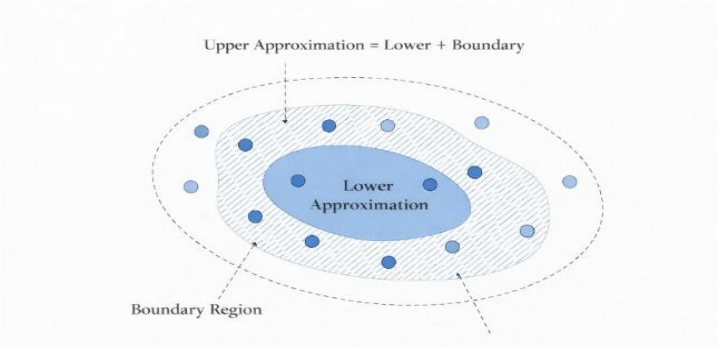


Figure 5.7: Rough Clustering Concept

Figure 5.7: Rough Clustering Concept: The lower approximation (dark inner region) contains points definitely in the cluster. The boundary region (hatched area) contains uncertain points. The upper approximation includes both regions.

Intuition Behind Rough Clustering:

Imagine you're classifying students into high performers and low performers based on exam scores:

- **Lower Approximation of High Performers:** Students with scores above 90% definitely high performers
- **Boundary Region**: Students with scores between 70% and 90% could be classified either way depending on the context
- **Lower Approximation of Low Performers:** Students with scores below 70% definitely in this category

Rough clustering acknowledges that some points genuinely belong to multiple clusters or are too ambiguous for a single assignment.

5.12 Rough K-Means Clustering Algorithm

The **Rough K-Means** algorithm combines the iterative refinement of K-Means with the approximation concepts of rough sets. Each cluster has a lower and upper approximation, and centroids are calculated considering both.

Rough K-Means Algorithm:

Given dataset X, number of clusters k, and threshold ϵ (for defining boundary region):

Algorithm:

- **Initialize:** Select k random centroids $\mu_1,...,\mu_k$
- **Classify Points into Approximations:**

For each point $\mathbf{x}_j$, compute distance to all centroids

Let $d_{\min} = \min_i \| x_j - \mu_i \|$ (distance to nearest centroid)

If only one cluster i *satisfies* $\| x_j - \mu_i \| \leq d_{min} + \epsilon$: Point is in **lower approximation** of cluster i

If multiple clusters satisfy this: Point is in **boundary region** (upper approximation of all those clusters, but lower approximation of none)

- **Update Centroids:** For each cluster i, compute centroid using weighted combination:

$$\mu_i = w_{lower} \cdot \bar{x}_{C_i} + w_{upper} \cdot \bar{x}_{\bar{C}_i - C_i} \qquad (5.42)$$

where $\bar{\mathbf{x}}_{Ci}$ is the mean of points in lower approximation, $\bar{\mathbf{x}}_{Ci-Ci}$ is the mean of boundary points, and $w_{\text{lower}} + w_{\text{upper}} = 1$.

A common simplified formula when lower approximation is non-empty:

$$\mu_i = \frac{w_{\text{lower}} \sum_{x \in C_i} x + w_{\text{upper}} \sum_{x \in \bar{C}_i - C_i} x}{w_{\text{lower}} \mid C_i \mid + w_{\text{upper}} \mid \bar{C}_i - C_i \mid} \qquad (5.43)$$

4. **Repeat:** Continue until convergence

Parameters:

- ϵ: Threshold for boundary region. Larger ϵ means wider boundaries (more uncertainty).
- $w_{\text{lower}}, w_{\text{upper}}$: Weights for lower and upper approximations.

Typical values: w_{lower} = 0.75, w_{upper} = 0.25 (give more weight to definite members).

Rough K-Means Classification:

Consider points $A(0,0)$, $B(1,0)$, $C(4,0)$, $D(5,0)$, $E(2.5,0)$ with $k = 2$ and $\epsilon = 1$.

Initial centroids: $\mu_1 = (0.5,0)$, $\mu_2 = (4.5,0)$

Classify each point:

For $A(0,0)$:

$d(A,\mu_1) = 0.5$, $d(A,\mu_2) = 4.5$

$d_{\min} = 0.5$, threshold = 0.5 + 1 = 1.5

Only cluster 1 within threshold ⇒ $A \in \underline{C_1}$ (lower approximation)

For $B(1,0)$:

$d(B,\mu_1) = 0.5$, $d(B,\mu_2) = 3.5$

$d_{\min} = 0.5$, threshold = 1.5

Only cluster 1 within threshold ⇒ $B \in \underline{C_1}$

For $E(2.5,0)$ (the point in the middle):

$d(E,\mu_1) = 2$, $d(E,\mu_2) = 2$

$d_{\min} = 2$, threshold = 2 + 1 = 3

Both clusters within threshold ⇒ E is in boundary region

$E \in C_1$ and $E \in C_2$ (but in neither lower approximation)

Similarly: $C,D \in \underline{C_2}$

Result:

$\underline{C_1} = \{A,B\}$, $C_1 = \{A,B,E\}$

$\underline{C_2} = \{C,D\}$, $C_2 = \{C,D,E\}$

Boundary region: $\{E\}$ (belongs to upper approximation of both clusters)

Point E is genuinely ambiguous and rough K-Means captures this uncertainty rather than forcing a decision.

5.13 Expectation-Maximization (EM) Based Clustering

Expectation-Maximization (EM) clustering takes a probabilistic approach to clustering. Instead of assigning points to clusters directly, it models the data as being generated by a mixture of probability distributions typically Gaussian distributions. The algorithm finds the parameters of these distributions that best explain the observed data.

This approach is more principled than K-Means because it explicitly models uncertainty and provides a probabilistic framework for cluster membership.

Gaussian Mixture Model (GMM)

A Gaussian Mixture Model assumes that data is generated from a mixture of kGaussian distributions:

$$p(x) = \sum_{i=1}^{k} \pi_i \ \mathcal{N}(x \mid \mu_i, \Sigma_i) \qquad (5.44)$$

Where:

- $\rightarrow$ mixing coefficient (prior probability) of cluster i, with

$$\sum_{i=1}^{k} \pi_i = 1$$

- $\mu_i \rightarrow$ mean (center) of the i-th Gaussian
- $\Sigma_i \rightarrow$ covariance matrix of the i-th Gaussian
- $\mathcal{N}(x \mid \mu, \Sigma) \rightarrow$ Gaussian probability density function

The multivariate Gaussian density is:

$$\mathcal{N}(x \mid \mu, \Sigma) = \frac{1}{(2\pi)^{d/2} \mid \Sigma \mid^{1/2}} \exp\left(-\frac{1}{2}(x-\mu)^T \Sigma^{-1}(x-\mu)\right) \qquad (5.45)$$

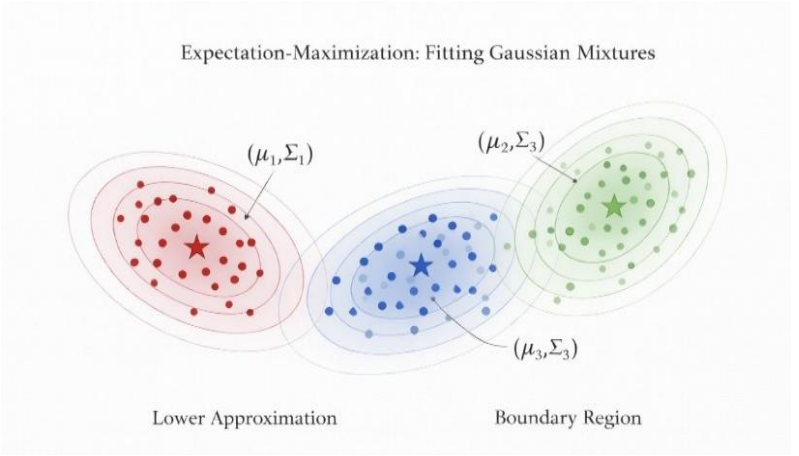


Figure 5.8: Gaussian Mixture Model

Figure 5.8: Gaussian Mixture Model: Data is modeled as coming from multiple Gaussian distributions. Elliptical contours show the probability density of each component. Points are colored by their most likely cluster, but each has a probability of belonging to any cluster.

The EM Algorithm for GMM:

The EM algorithm iteratively refines the model parameters by alternating between two steps:

M-Step (Maximization): Update the model parameters using the responsibilities:

$$N_i = \sum_{j=1}^{n} \gamma_{ij} \text{ (effective number of points in cluster } i) \qquad (5.47)$$

$$\mu_i^{\text{new}} = \frac{1}{N_i} \sum_{j=1}^{n} \gamma_{ij}\ x_j \qquad (5.48)$$

EM Algorithm (for GMM)

Goal: Find parameters $\{\pi_i, \mu_i, \Sigma_i\}_{i=1}^{k}$ that maximize the likelihood of the observed data.

E-Step (Expectation):

Compute the responsibility γ_{ij} the probability that point x_j was generated by cluster i:

$$\gamma_{ij} = \frac{\pi_i \cdot \mathcal{N}(x_j \mid \mu_i, \Sigma_i)}{\sum_{\ell=1}^{k} \pi_\ell \cdot \mathcal{N}(x_j \mid \mu_\ell, \Sigma_\ell)} \qquad (5.46)$$

This is a soft assignment based on Bayes' theorem.

M-Step (Maximization): Update the model parameters using the responsibilities:

$$N_i = \sum_{j=1}^{n} \gamma_{ij} \text{ (effective number of points in cluster } i) \qquad (5.47)$$

$$\mu_i^{\text{new}} = \frac{1}{N_i} \sum_{j=1}^{n} \gamma_{ij}\ x_j \qquad (5.48)$$

$$\Sigma_i^{\text{new}} = \frac{1}{N_i} \sum_{j=1}^{n} \gamma_{ij}\ (x_j - \mu_i^{\text{new}})(x_j - \mu_i^{\text{new}})^T \qquad (5.49)$$

$$\pi_i^{\text{new}} = \frac{N_i}{n} \qquad (5.50)$$

Repeat: Alternate E and M steps until convergence (log-likelihood stops improving).

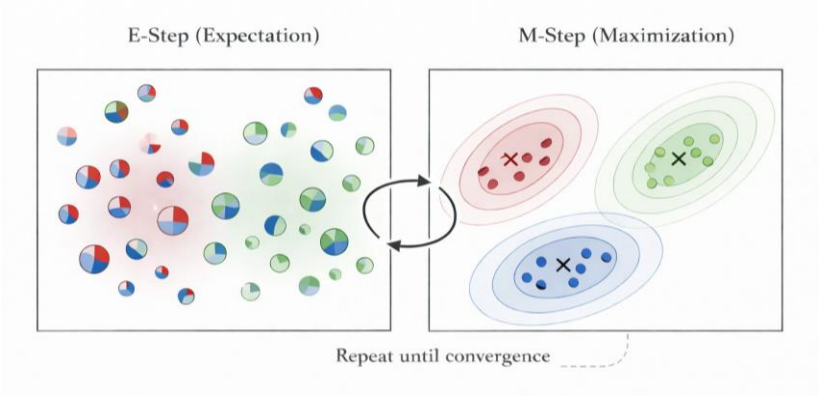


Figure 5.9: EM Algorithm Steps

Figure 5.9: EM Algorithm Steps: E-Step computes soft assignments (responsibilities) of points to clusters. M-Step updates the Gaussian parameters (means, covariances, mixing weights) based on these assignments. The process repeats until convergence.

EM Clustering Intuition:

Suppose we have data that appears to come from two overlapping Gaussians.

Initial guess: Two Gaussians with means μ_1 = (0,0), μ_2 = (3,0), equal covariances, and mixing coefficients $\pi_1 = \pi_2 = 0.5$.

E-Step: For a point at (1, 0):

- Distance to μ1: 1, Distance to μ2: 2
- The point is more likely from cluster 1, so γ1j > γ2j
- Perhaps γ1j = 0.8, γ2j = 0.2 For a point at (1.5, 0) (exactly between):
- Equal distances to both means
- γ1j ≈ γ2j ≈ 0.5 (equally likely from either cluster) M-Step: Update means using weighted averages:
- μ1 moves toward points with high γ1j
- μ2 moves toward points with high γ2j
- Covariances adjust to match the spread of points assigned to each cluster

After convergence: The algorithm finds the Gaussian parameters that best explain the data distribution. Final cluster assignments can be made by assigning each point to its most probable cluster (highest γij).

GMM vs. K-Means:

Aspect	K-Means	GMM (EM)
Cluster shape	Spherical only	Elliptical (any orientation)
Assignment	Hard (0 or 1)	Soft (probabilities)
Cluster sizes	Assumes equal	Can model di erent sizes
Model type	Distance-based	Probabilistic
Output	Cluster labels	Probabilities + parameters

K-Means is actually a special case of GMM where all covariances are equal and spherical ($\mathbf{\Sigma}_i = \sigma^2\mathbf{I}$), and we take the limit as $\sigma \to 0$ (hard assignments).

5.14 Spectral Clustering

Spectral clustering uses concepts from linear algebra and graph theory to cluster data. It is particularly effective for discovering clusters that are not linearly separable such as concentric circles, interleaved crescents, or other complex shapes that would confuse K-Means.

The key idea is to transform the data into a new representation using the eigenvectors (spectrum) of a similarity matrix or graph Laplacian, then apply a standard clustering algorithm (like K-Means) in this transformed space.

Spectral Clustering:

Spectral clustering works by:

- Constructing a **similarity graph** from the data
- Computing the **graph Laplacian** matrix
- Finding the **eigenvectors** of the Laplacian
- Using these eigenvectors as a new representation of the data
- Applying K-Means (or another algorithm) in the spectral embedding space

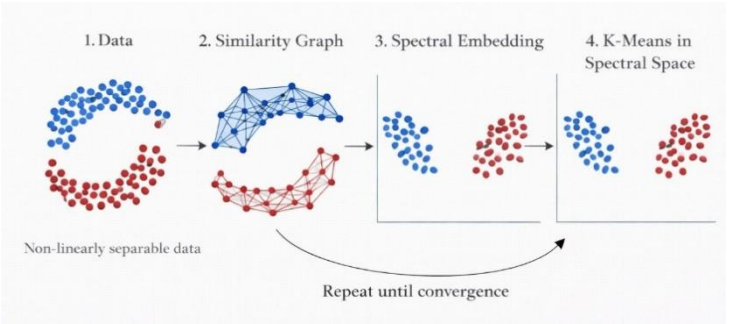


Figure 5.10: Spectral Clustering Process

Figure 5.10: Spectral Clustering Process: (1) Original non-linearly separable data (two moons), (2) Construct similarity graph with edges connecting nearby points, (3) Transform to spectral embedding where clusters become separable, (4) Apply K-Means to get final clusters.

Step 1: Construct the Similarity Graph

From the data points, construct a graph where:

- Each data point is a node
- Edges connect similar points (nearby in feature space)
- Edge weights re ect the strength of similarity

Common approaches:

- ϵ-neighborhood graph: Connect points within distance ϵ (unweighted)
- k-nearest neighbors graph: Connect each point to its k nearest neighbors
- Fully connected graph: Connect all points with weights based on Gaussian kernel:

$$w_{ij} = \exp\left(\frac{-\| x_i - x_j \|^2}{2\sigma^2}\right) \quad (5.51)$$

Step 2: Compute the Graph Laplacian

The **similarity matrix W** contains edge weights: $W_{ij} = w_{ij}$.

The **degree matrix D** is diagonal with $D_{ii} = {}^{P}{}_{j} W_{ij}$ (sum of edge weights for node i).

The **Laplacian matrix** has several variants:

- Unnormalized: $\mathbf{L} = \mathbf{D} - \mathbf{W}$
- Normalized (symmetric): $\mathbf{L}_{sym} = \mathbf{I} - \mathbf{D}^{-1/2}\mathbf{W}\mathbf{D}^{-1/2}$
- Normalized (random walk): $\mathbf{L}_{rw} = \mathbf{I} - \mathbf{D}^{-1}\mathbf{W}$

Step 3: Compute Eigenvectors

Compute the k smallest eigenvectors of the Laplacian (corresponding to smallest eigenvalues). For the unnormalized Laplacian, solve:

$$\mathbf{Lv} = \lambda\mathbf{v} \tag{5.52}$$

Let $\mathbf{v}_1,\mathbf{v}_2,\ldots,\mathbf{v}_k$ be the eigenvectors for the k smallest eigenvalues.

Step 4: Form the Spectral Embedding

Create a new representation of the data where each point i is represented by:

$$\mathbf{y}_i = (v_1(i),v_2(i),\ldots,v_k(i))^T \tag{5.53}$$

where $v_j(i)$ is the i-th component of eigenvector $\mathbf{v}_j$.

Step 5: Cluster in the Embedded Space

Apply K-Means (or another clustering algorithm) to the points $\{\mathbf{y}_1,\ldots,\mathbf{y}_n\}$ in the k-dimensional spectral embedding space.

Why Spectral Clustering Works: The Two Moons Example:

Consider the two moons dataset two crescent-shaped clusters that interleave.

K-Means fails here because the clusters are not convex.

What happens in spectral clustering:

1. **Similarity graph:** Points within each moon are connected by short paths (high similarity). Points in different moons are connected only by long paths through the gap.
2. **Graph Laplacian:** Captures the connectivity structure. The two moons form two loosely connected components.
3. **Eigenvectors:** The second smallest eigenvector (Fiedler vector) naturally separates the graph into two components. Points in one moon have positive values; points in the other have negative values.
4. **Spectral embedding:** In this 1D representation (using just the Fiedler vector), the two moons become two well-separated groups on a line!
5. **K-Means in 1D:** Trivially separates the two clusters.

Key insight: Spectral clustering transforms the problem from separating nonconvex shapes in original space to separating convex (even linear) structures in spectral space.

Properties of Graph Laplacian Eigenvalues:

- The smallest eigenvalue is always 0 (corresponding to the constant eigenvector)
- The number of zero eigenvalues equals the number of connected components in the graph
- The spectral gap (difference between first and second eigenvalue) indicates how strongly connected the graph is
- The Fiedler vector (second eigenvector) optimally bipartitions the graph

Spectral Clustering Algorithm Summary:

1. Input: Data points $\mathbf{x}_1,\ldots,\mathbf{x}_n$, number of clusters k
2. Construct similarity matrix $\mathbf{W}$ using Gaussian kernel or k-NN
3. Compute degree matrix $\mathbf{D}$ and Laplacian $\mathbf{L} = \mathbf{D} - \mathbf{W}$
4. Find first k eigenvectors of $\mathbf{L}$ (smallest eigenvalues)
5. Form matrix $\mathbf{U} \in R^{n \times k}$ with eigenvectors as columns
6. Normalize rows of $\mathbf{U}$ to have unit length (for normalized spectral clustering)
7. Cluster the rows of $\mathbf{U}$ using K-Means
8. Assign original point $\mathbf{x}_i$ to the cluster of row i

When to Use Spectral Clustering:

- When clusters have non-convex shapes
- When the number of clusters is small (computing eigenvectors is expensive for large k)
- When you have a natural notion of similarity between points
- For image segmentation, community detection in networks, and other graph structured problems

Chapter Summary

This chapter presented a comprehensive treatment of clustering the core unsupervised learning task of discovering natural groups in unlabeled data. The discussion progressed from foundational concepts through increasingly sophisticated algorithms, unified by the central question: how do we define and measure similarity, and how do we exploit it to reveal meaningful structure?

Foundations: Similarity, Partitions, and Objectives

Clustering was introduced as a partition problem: given a dataset X = {x1, ..., xn}, find a collection of disjoint, exhaustive subsets C = {C1, ..., Ck} that maximizes intra-cluster similarity and minimizes inter-cluster similarity. Three canonical distance measures Euclidean, Manhattan, and Cosine similarity were reviewed as the computational backbone that determines what 'closeness' means for a given dataset.

The quality of any partition is measured objectively. The Within-Cluster Sum of Squares (WCSS) quantifies compactness by summing squared distances from every point to its cluster centroid; lower WCSS means tighter clusters. The Between-Cluster Sum of Squares (BCSS) measures separation between cluster centers. Since total variance is fixed, minimizing WCSS is mathematically equivalent to maximizing BCSS, reducing the multi-objective trade-off to a single optimization target.

Matrix Factorization and Pattern Representation

Non-negative Matrix Factorization (NMF) reframes clustering as a low-rank decomposition: $X \approx WH$, where W encodes cluster prototypes and H encodes each data point's membership weights. The non-negativity constraint promotes parts-based, additive representations ideal for word-count matrices, pixel intensities, and user ratings. Cluster assignments follow by selecting the column of H with the highest weight for each data point, yielding a natural bridge between linear algebra and unsupervised classification.

Feature-space pattern representation was formalized alongside four cluster representations: the centroid (cluster mean), the medoid (most central actual data point), a prototype set (multiple representative patterns), and a statistical model (a probability distribution fitted to the cluster). The choice of representation influences both algorithmic design and the interpretability of the results.

Hierarchical Clustering: Divisive and Agglomerative

Hierarchical methods produce a full dendrogram a tree of nested clustering's from n singletons to one all-inclusive cluster without requiring k to be specified

in advance. The desired number of clusters is obtained by cutting the dendrogram at an appropriate height.

Divisive (top-down) clustering begins with the complete dataset and recursively splits the most heterogeneous cluster. Splitting strategies include running K-Means with k=2 internally, splitting along the principal direction of variance, selecting the two farthest points as seeds, or finding a minimum graph cut. Divisive methods are computationally expensive but advantageous when only a small number of top-level clusters are needed.

Agglomerative (bottom-up) clustering starts with n singleton clusters and iteratively merges the closest pair. The linkage criterion determines inter-cluster distance: Single linkage (minimum pairwise distance) creates elongated, chain-like clusters and is sensitive to noise; Complete linkage (maximum pairwise distance) produces compact spherical clusters; Average linkage balances the two; Ward's method minimizes the increase in total WCSS at each merge, creating similarly-sized, compact clusters and mirroring K-Means' objective.

Partitional Clustering: K-Means and Its Variants

K-Means is the most widely deployed clustering algorithm. It iterates two steps until convergence: (1) Assign each point to the nearest centroid; (2) Recompute each centroid as the mean of its assigned points. This alternating minimization monotonically decreases WCSS, guaranteeing convergence to a local optimum. The algorithm runs in O(nkdi) time, making it scalable even for large datasets.

Selecting k is the primary practical challenge. The Elbow Method plots WCSS against k and identifies the point of diminishing returns. The Silhouette Score measures how similar each point is to its own cluster relative to the nearest competing cluster, ranging from -1 (misassigned) to +1 (well-clustered). The Gap Statistic compares observed WCSS against a null distribution. K-Means' main limitations are its restriction to spherical, equally-sized clusters, sensitivity to outliers (means are non-robust), and dependence on initialization addressed in practice by multiple random restarts or the K-Means++ seeding strategy.

Key Insight	K-Means is a special case of both EM clustering (GMM with spherical, equal covariances as covariance shrinks to zero) and Fuzzy C-Means (as the fuzzifier m → 1, soft memberships harden to crisp assignments). Understanding K-Means deeply thus illuminates the entire family of partition-based methods.

Handling Uncertainty: Fuzzy and Rough Clustering

Standard K-Means forces every point into exactly one cluster, which is unrealistic for boundary points that genuinely resemble multiple groups. Two complementary paradigms address this.

Fuzzy C-Means (FCM) assigns every point a continuous membership degree uij ∈ [0,1] to every cluster i, with the constraint that memberships sum to 1 across clusters for each point. A fuzzifier parameter m > 1 controls the softness of assignments; as m → 1 the solution hardens to K-Means, and as m → ∞ memberships equalize to 1/k. Centroids are computed as weighted means using membership degrees raised to the m-th power. FCM is particularly valuable in medical imaging and sensor fusion where crisp boundaries are artificially imposed by hard clustering.

Rough K-Means, grounded in Pawlak's Rough Set Theory, takes a different approach to uncertainty. Each cluster C is characterized by a lower approximation (points definitively in C), an upper approximation (points possibly in C), and a boundary region (the difference — points that are genuinely ambiguous). A threshold ε controls boundary width: if a point's distance to its nearest centroid is within ε of its distance to any other centroid, it enters the boundary region of all competing clusters. Centroids are updated as weighted combinations of definite and boundary members, with higher weight given to definite members. Rough clustering makes no probabilistic assumptions and handles uncertainty through set-theoretic approximation rather than graded membership.

Probabilistic Clustering: Expectation-Maximization and GMMs

Expectation-Maximization clustering models the data as samples from a Gaussian Mixture Model (GMM): a weighted sum of k multivariate Gaussians, each parameterized by a mean μi, covariance matrix Σi, and mixing coefficient πi. The EM algorithm finds maximum-likelihood estimates of these parameters through two alternating steps applied until the log-likelihood converges.

In the E-Step, responsibilities γij are computed for each point xj and cluster i via Bayes' theorem — the probability that xj was generated by Gaussian i. In the M-Step, parameters are updated: the new mean is the responsibility-weighted average of all data points; the new covariance is the responsibility-weighted scatter matrix; and the new mixing coefficient is the average responsibility. Because covariances are unconstrained, GMMs can model elliptical clusters of any orientation and size, overcoming K-Means' spherical restriction. K-Means emerges as a limiting special case when all covariances are set to $\sigma^2 I$ and σ → 0.

Spectral Clustering: Graph-Theoretic Methods for Complex Shapes

When clusters have non-convex shapes such as concentric rings or interleaved crescents all distance-based algorithms fail because they implicitly assume convex cluster boundaries. Spectral Clustering resolves this by operating in a transformed representation derived from the graph structure of the data.

The algorithm constructs a weighted similarity graph in which nodes are data points and edge weights reflect pairwise similarity (typically via a Gaussian kernel or k-NN connectivity). The graph Laplacian L = D − W (where D is the diagonal degree matrix) encodes the graph's connectivity structure. The k smallest eigenvectors of L form a spectral embedding in which points from the same cluster appear close together, even if they were interleaved in the original space. Standard K-Means is then applied to these embedded representations to produce the final cluster assignments.

The mathematical justification lies in the eigenvalue spectrum of L: the number of zero eigenvalues equals the number of connected components, and the Fiedler vector (second-smallest eigenvector) optimally bipartitions the graph. Spectral clustering is the method of choice for image segmentation, social network community detection, and any domain where cluster geometry is non-convex.

Algorithm Comparison at a Glance

Algorithm	Cluster Shape	Assignment	Requires k?	Best For
K-Means	Spherical	Hard	Yes	Large, well-separated data
K-Medoids	Spherical	Hard	Yes	Noisy / outlier-prone data
Agglomerative	Flexible	Hard	No	Hierarchical structure needed

Fuzzy C-Means	Spherical	Soft (degrees)	Yes	Overlapping, boundary-heavy data
Rough K-Means	Spherical	Rough sets	Yes	Uncertain, ambiguous boundaries
GMM / EM	Elliptical	Soft (probabilities)	Yes	Probabilistic, non-spherical data
Spectral	Arbitrary	Hard (after embed)	Yes	Non-convex, graph-structured data

Key Takeaways

- Clustering is an unsupervised task: no labels are used. The algorithm must rely entirely on the geometric or probabilistic structure of the input data to discover groupings.
- Distance measure selection is foundational. Euclidean distance suits continuous, normalized features; Manhattan is more robust to outliers; Cosine similarity serves high-dimensional text and sparse data.
- WCSS is the canonical objective for hard clustering. Minimizing it is equivalent to maximizing BCSS and is the common thread linking K-Means, Ward's linkage, and GMM in its spherical limit.
- Hierarchical methods offer a full spectrum of clustering's without pre-specifying k, at the cost of $O(n^2 \log n)$ time and the inability to revise earlier decisions.
- K-Means is fast and widely applicable but assumes spherical clusters of similar size and is sensitive to initialization and outliers.
- Fuzzy and Rough clustering extend K-Means to handle uncertainty at cluster boundaries: FCM through graded membership values, Rough K-Means through set-theoretic lower and upper approximations.

- GMM/EM is the principled probabilistic generalization of K-Means: it accommodates elliptical clusters, provides calibrated uncertainty estimates, and has a sound maximum-likelihood foundation.
- Spectral Clustering is uniquely able to handle non-convex cluster shapes by transforming the problem into a graph-theoretic embedding before applying standard partitioning.

Review Questions

Section A — Conceptual Questions

1. Define clustering and explain in what way it differs from classification and regression.
2. Describe the role of similarity and distance measures in clustering. Why must features be scaled before computing Euclidean distance?
3. Distinguish between partitional and hierarchical clustering. Give one example algorithm from each family.
4. Compare divisive and agglomerative hierarchical clustering approaches. Mention one advantage and one disadvantage of each.
5. Explain the purpose of the linkage criterion in agglomerative clustering. Contrast single linkage, complete linkage, average linkage, and Ward's method.
6. Discuss the meaning of the Within-Cluster Sum of Squares and Between-Cluster Sum of Squares (BCSS). How do they indicate cluster quality?
7. List the major types of clustering algorithms and briefly state the kind of data or application each is best suited for.
8. Explain why K-Means requires prior knowledge of k. How would you choose an appropriate number of clusters?
9. What are the limitations of K-Means clustering? Under what conditions might it fail?
10. Define soft clustering. How does it differ from hard clustering conceptually and mathematically?
11. Describe the fuzziness parameter m in Fuzzy C-Means. What happens as m approaches 1 or ∞?
12. Explain the concepts of lower approximation, upper approximation, and boundary region in Rough Clustering.
13. What problem does the Rough K-Means algorithm aim to solve compare to classic K-Means?
14. Summarize the key idea of Expectation–Maximization (EM) based Gaussian Mixture Model clustering.
15. Why is spectral clustering effective for non-convex cluster shapes such as crescents or concentric circles?
16. Briefly explain the significance of the graph Laplacian and its eigenvectors in spectral clustering.

Section B — Analytical Problems

17. Given six 1-D data points X = {1, 2, 3, 10, 11, 12}, compute the WCSS for (a) clusters {1,2,3} and {10,11,12}, and (b) clusters {1,10,11} and {2,3,12}. Which partition is better and why?
18. For the distance matrix below, perform the first three merging steps of agglomerative clustering using single linkage, and draw the partial dendrogram.

	A	**B**	**C**	**D**	**E**
A	0	2	6	10	9
B	2	0	5	9	8
C	6	5	0	4	5
D	10	9	4	0	3
E	9	8	5	3	0

19. Apply one iteration of the K-Means algorithm to the following points, using $k = 2$ and find the Euclidean distance. Points: (1,1), (1,2), (2,1), (8,8), (9,8), (8,9).Initial centroids :(1,1) and (8,8). Determine cluster assignments and new centroids.
20. A dataset of four points $P_1(1,1)$, $P_2(2,1)$, $P_3(6,6)$, $P_4(7,6)$ is clustered using Fuzzy C-Means with $m = 2$ and initial centroids $\mu_1(1.5,1)$, $\mu_2(6.5,6)$. Compute the good membership values u_{11} and u_{21} for P_1 and interpret the result.
21. In the Rough K-Means algorithm with $\epsilon = 1$, two clusters have centroids $\mu_1 = 0$ and $\mu_2 = 5$ along a 1-D line. For a point $x = 2.5$, determine whether it lies in the lower or upper approximation of each cluster.
22. For a 2-cluster Gaussian Mixture Model with equal mixing coefficients and shared spherical covariance $\sigma^2 I$, show that maximizing the log-likelihood is equivalent to minimizing the squared Euclidean distances between points and centroids—that is, equivalent to K-Means.

Section C — Applied and Design Problems

23. Design a customer segmentation analysis using K-Means. Specify how you would: (1) select features, (2) normalize them, (3) choose k, and (4) validate the resulting clusters from a business perspective.

24. For an image-segmentation task with irregular object boundaries, explain why spectral clustering may outperform K-Means. Describe how you would construct the similarity graph.
25. You are working on a network-intrusion dataset. Compare the use of density-based clustering vs. Rough K-Means for identifying abnormal connections. Discuss expected advantages and limitations.
26. Consider applying EM-based GMM clustering to sensor data where each cluster corresponds to a physical mode of operation. Describe how you would interpret the Gaussian parameters μ_i and Σ_i and decide the optimal number of components.

ABOUT THE BOOK

This book offers a systematic and in-depth exploration of the machine learning, designed to help the readers build a strong foundation while progressing toward advanced applications. It begins by introducing the core principles of machine learning, including data representation, and statistical thinking, and the fundamental paradigms of supervised, and unsupervised, and reinforcement learning. The book emphasizes the integration of theory with implementation.

ABOUT AUTHORS

Dr. Halavath Balaji

Dr. Halavath Balaji has developed an exceptional expertise in delivering of high-quality education, and guiding research, and mentoring students toward academic and professional success. As a Professor at Sreenidhi Institute of Science and Technology.

Jogu Saritha

Jogu Saritha is an accomplished academician currently serving as an Assistant Professor at Sreenidhi Institute of Science and Technology. She is also a dedicated Research Scholar at the National Institute of Technology, Jalandhar (NITJ).

Pallavi B

Pallavi.B is an experienced academician working as an Assistant Professor at Sreenidhi Institute of Science and Technology. She is also a research scholar at SR University, Warangal, where she is actively involved in the field of Machine Learning.

MRP. ₹700

ISBN 978-81-685077-3-9

9 788168 507739

Lurnexa Publications

130–187, Ramulavari Gudi Centre, Gorantla,
Guntur 522034, Andhra Pradesh, India
lurnexapublication@Gmail.com
lurnexa.in